

Advance Java Programming (Spring Boot)

Practical Assignment

Practical No. 1: Introduction to JPA – Entity Class and Persisting a Single Entity

Definition / Objective

Entity

Textbook Reference: Chapter 5 — Working with Spring Data JPA

Objective:

To understand the requirements of a JPA entity class (no-arg constructor, @Entity, @Id, @GeneratedValue),

configure a relational datasource in application.properties, and persist/retrieve an entity using Spring Data JPA's

CrudRepository.

Class Name(s) with Attributes:

Employee (@Entity)

- empId : int (@Id @GeneratedValue)

- empName : String

- department : String

- salary : double

EmployeeRepository (interface extends CrudRepository<Employee, Integer>)

- Inherited: save(), findAll(), findById(), deleteById()

EmployeeRestController (@RestController)

- employeeRepository : EmployeeRepository

Step-by-Step Task Instructions:

1. Add spring-boot-starter-data-jpa and a database driver (MySQL/H2) dependency in pom.xml.
2. Configure spring.datasource.url, username, password and spring.jpa.hibernate.ddl-auto=update in application.properties.
3. Create the Employee entity annotated with @Entity, @Id, and @GeneratedValue(strategy = GenerationType.IDENTITY).
4. Create EmployeeRepository extending CrudRepository<Employee, Integer> (the base Spring Data repository)

interface introduced in this chapter).

5. Create `EmployeeRestController` with POST `/api/employees` (save) and GET `/api/employees` (findAll).

6. Add GET `/api/employees/{id}` (findById) to fetch a single employee, PUT `/api/employees/{id}` to update an

existing employee (fetch, modify fields, save again), and DELETE `/api/employees/{id}` (deleteById) to remove an employee.

7. Run the application, verify the employee table is auto-created in the database, insert a record, and query it

Code

`EmployeeRestController.java`

```
package com.example.empapi.controller;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.example.empapi.entity.Employee;
import com.example.empapi.repository.EmployeeRepository;

@RestController
@RequestMapping("/api/employees")
public class EmployeeRestController {

    private final EmployeeRepository employeeRepository;

    public EmployeeRestController(EmployeeRepository employeeRepository) {
        super();
        this.employeeRepository = employeeRepository;
    }

    @PostMapping
    public ResponseEntity<Employee> createEmployee(
        @RequestBody Employee employee){

        Employee savedEmployee = employeeRepository.save(employee);

        return ResponseEntity
            .status(HttpStatus.CREATED)
            .body(savedEmployee);
    }

    @GetMapping
    public ResponseEntity<Iterable<Employee>> getAllEmployee(){

        Iterable<Employee> employees = employeeRepository.findAll();

        return ResponseEntity.ok(employees);
    }

    @GetMapping("/{id}")
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
public ResponseEntity<Employee> getEmployeeById(
    @PathVariable int id) {

    return employeeRepository.findById(id)
        .map(ResponseEntity::ok)
        .orElseGet(() -> ResponseEntity.notFound().build());
}

@PutMapping("/{id}")
public ResponseEntity<Employee> updateEmployee(
    @PathVariable int id,
    @RequestBody Employee employeeDetails) {

    return employeeRepository.findById(id)
        .map(existingEmployee -> {

            existingEmployee.setEmpName(
                employeeDetails.getEmpName()
            );

            existingEmployee.setDepartment(
                employeeDetails.getDepartment()
            );

            existingEmployee.setSalary(
                employeeDetails.getSalary()
            );

            Employee updatedEmployee =
                employeeRepository.save(existingEmployee);

            return ResponseEntity.ok(updatedEmployee);
        })
        .orElseGet(() -> ResponseEntity.notFound().build());
}

>DeleteMapping("/{id}")
public ResponseEntity<Void> deleteEmployee(
    @PathVariable int id) {

    if (!employeeRepository.existsById(id)) {
        return ResponseEntity.notFound().build();
    }

    employeeRepository.deleteById(id);

    return ResponseEntity.noContent().build();
}
}
```

Employee.java

```
package com.example.empapi.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.*;

@Entity
public class Employee {

    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private int empId;

    private String empName;
    private String department;
    private double salary;
}
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
public Employee() {
    super();
    // TODO Auto-generated constructor stub
}

public Employee(String empName, String department, double salary) {
    super();
    this.empName = empName;
    this.department = department;
    this.salary = salary;
}

public int getEmpId() {
    return empId;
}
public void setEmpId(int empId) {
    this.empId = empId;
}
public String getEmpName() {
    return empName;
}
public void setEmpName(String empName) {
    this.empName = empName;
}
public String getDepartment() {
    return department;
}
public void setDepartment(String department) {
    this.department = department;
}
public double getSalary() {
    return salary;
}
public void setSalary(double salary) {
    this.salary = salary;
}
}
```

P1Application.java

```
package com.example.empapi;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class P1Application {

    public static void main(String[] args) {
        SpringApplication.run(P1Application.class, args);
    }

}
```

EmployeeRepository.java

```
package com.example.empapi.repository;

import org.springframework.data.repository.CrudRepository;
import com.example.empapi.entity.Employee;

public interface EmployeeRepository extends CrudRepository<Employee, Integer>{
```

```
}
```

P1ApplicationTests.java

```
package com.example.empapi;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

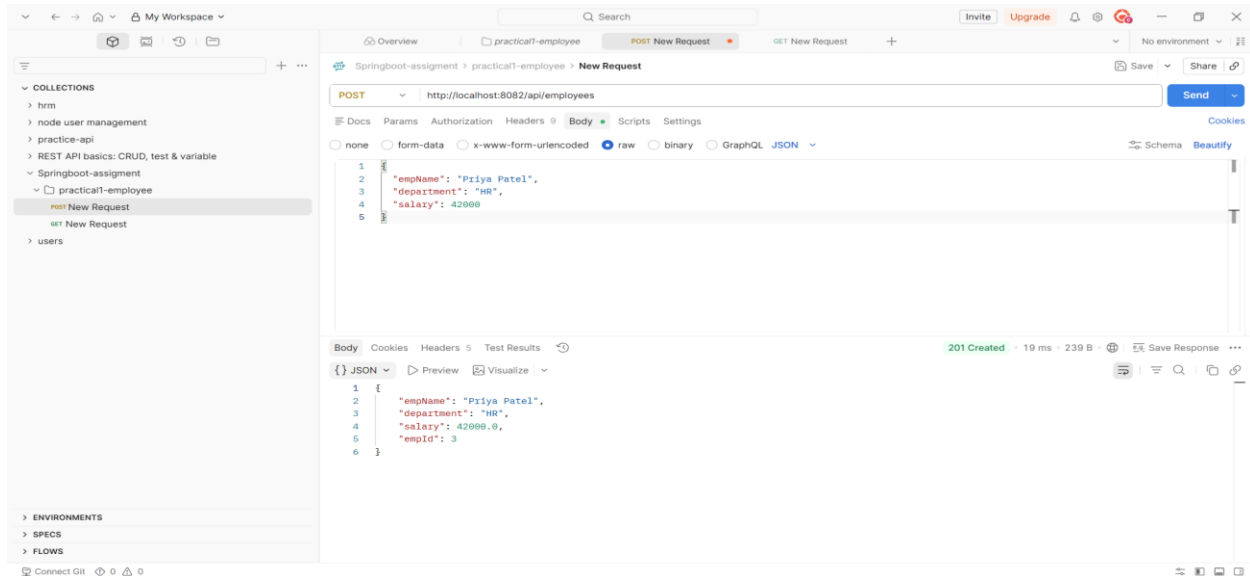
@SpringBootTest
class P1ApplicationTests {

    @Test
    void contextLoads() {
    }

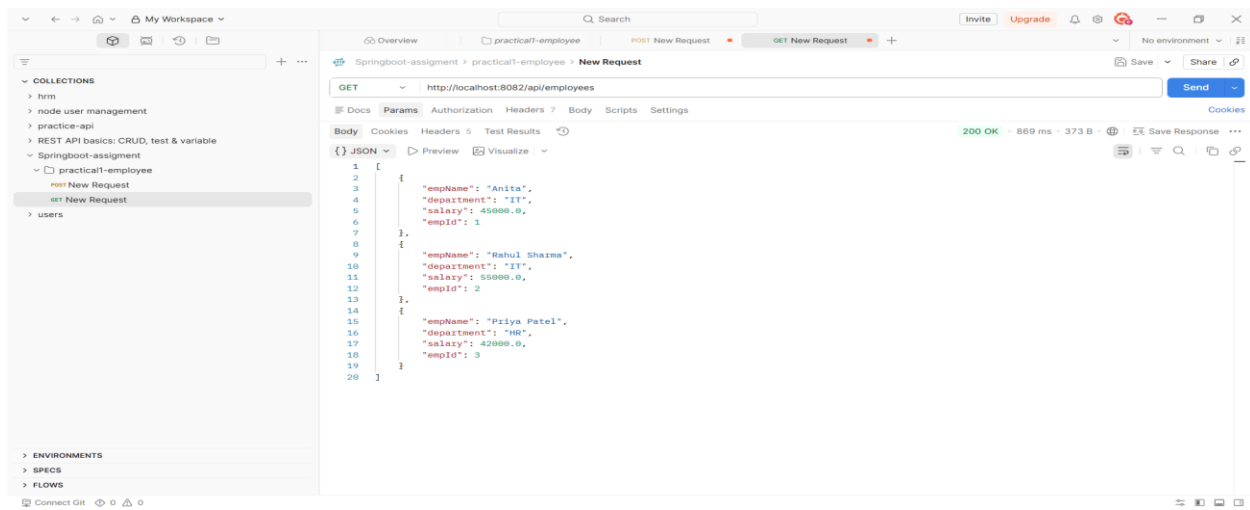
}
```

POST Method

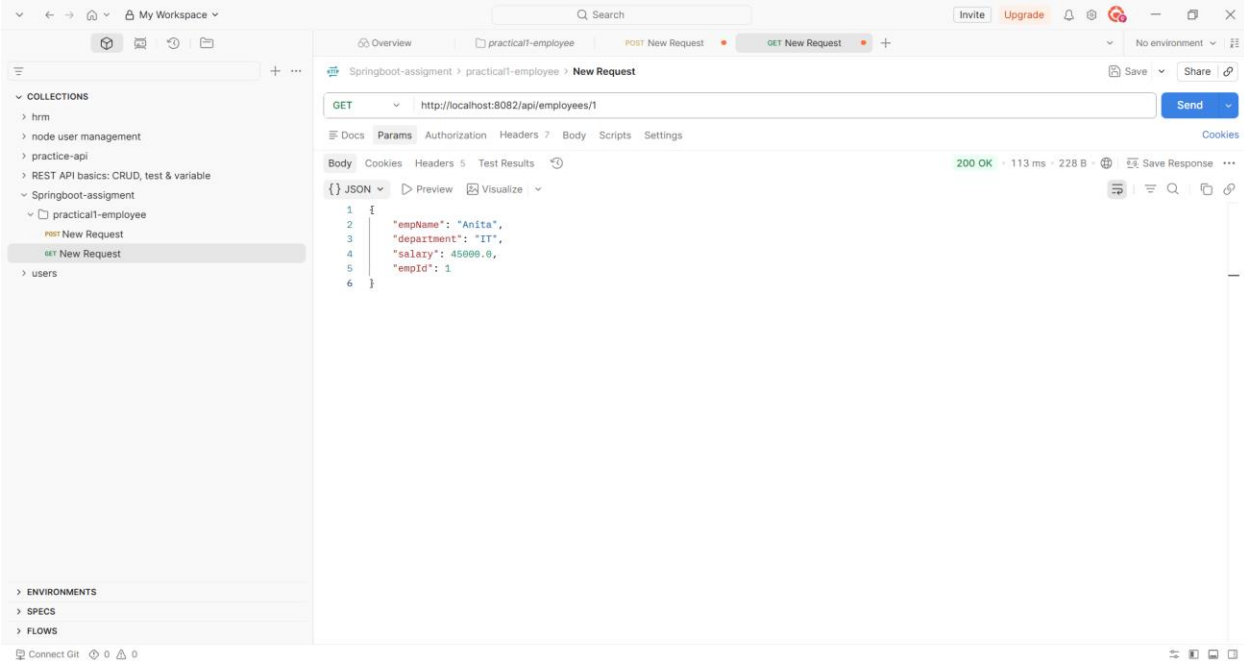
Enroll – 25CI2110039 | Name – Kalariya Heli



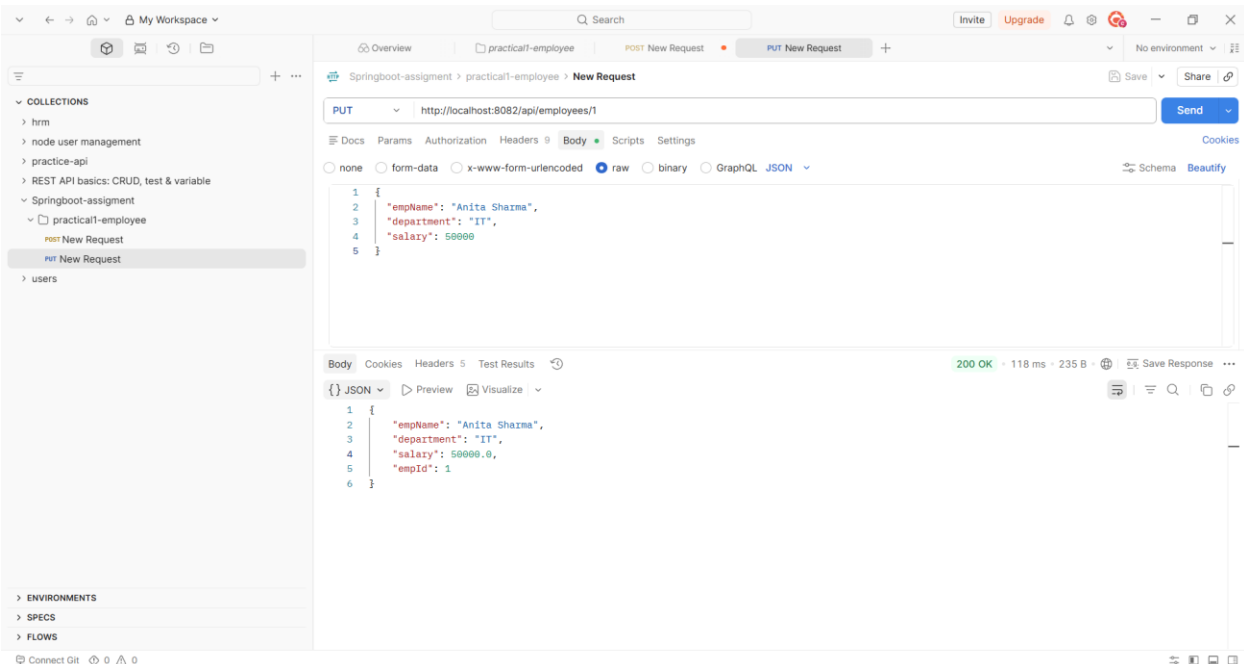
GET Method



GET Method with ID

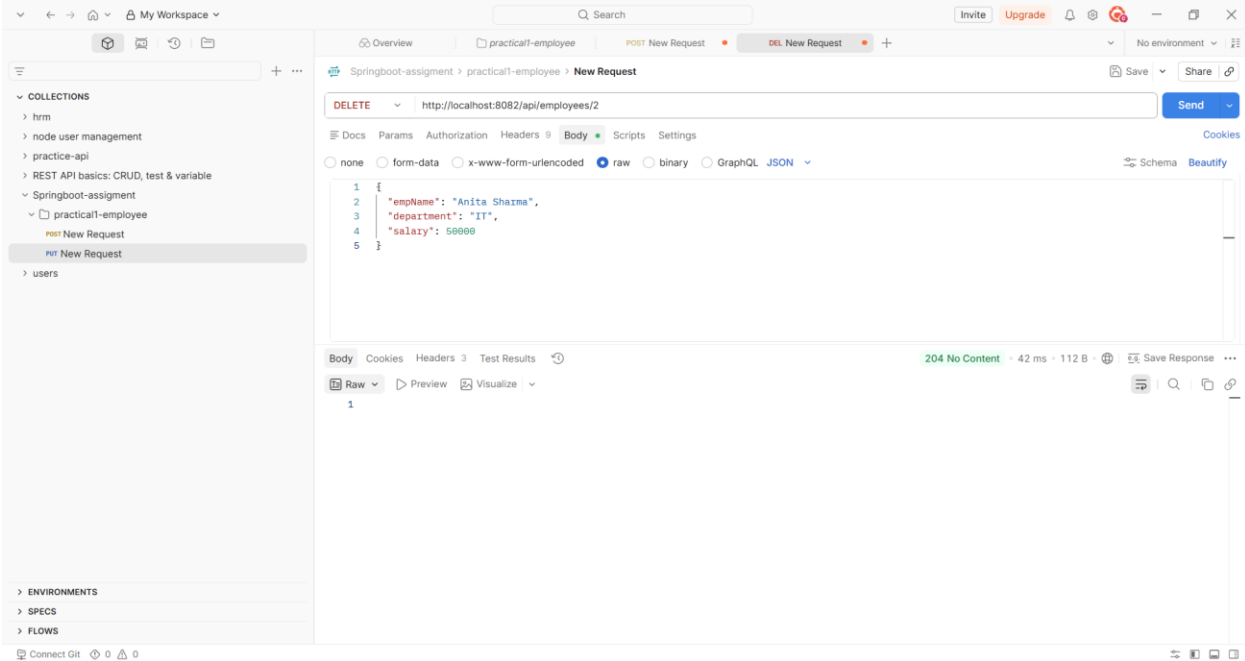


PUT Method

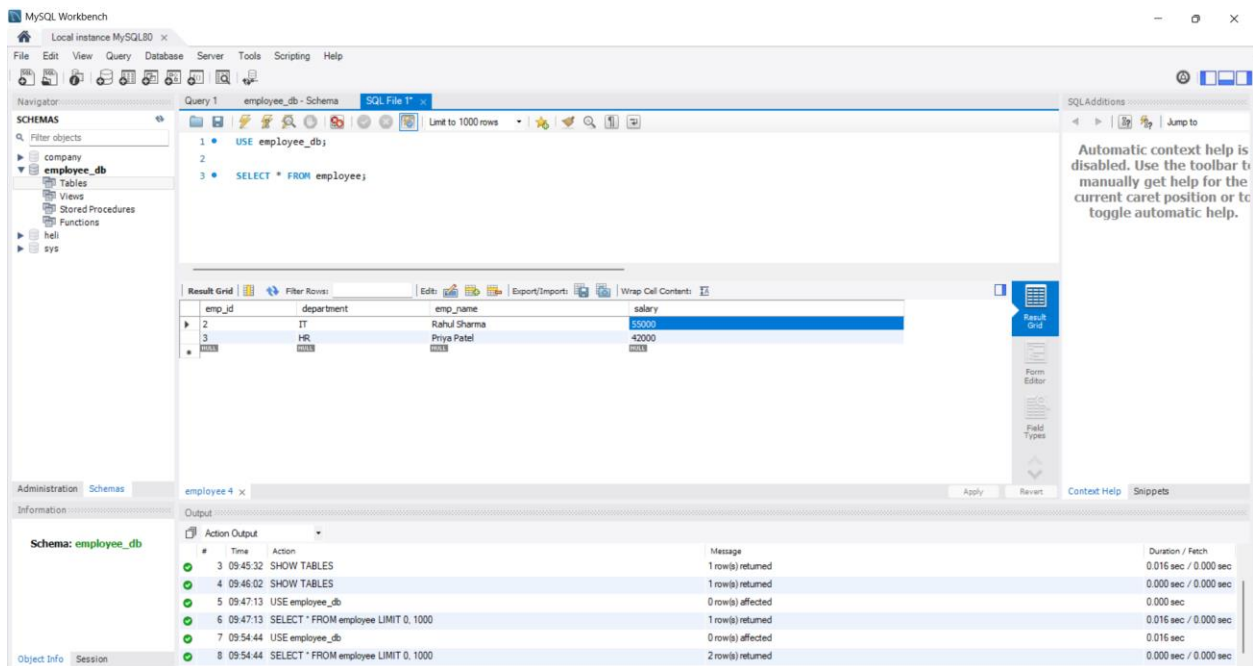


DELETE Method

Enroll – 25CI2110039 | Name – Kalariya Heli



Employee TABLE In SQL Workbench



Meth od	Endpoint (Postman)	Request Body / Params	Expecte d
------------	--------------------	-----------------------	--------------

			Response
POST	http://localhost:8082/api/employees	{"empName":"Anita","department":"IT","salary":45000}	201 Created — Employee record added successfully
GET	http://localhost:8082/api/employees	None	200 OK — Displays all employee records
GET	http://localhost:8082/api/employees/1	Path Variable: id = 1	200 OK — Displays employee with ID 1
PUT	http://localhost:8082/api/employees/1	{"empName":"Anita Sharma","department":"IT","salary":50000}	200 OK — Employee record updated successfully
DELETE	http://localhost:8082/api/employees/1	Path Variable: id = 1	204 No Content — Employee record deleted

			successfully
--	--	--	--------------

Practical No. 2: Spring Data Repository Hierarchy – CrudRepository vs PagingAndSortingRepository vs JpaRepository

Definition / Objective

PagingAndSortingRepository vs JpaRepository

Textbook Reference: Chapter 5 — Working with Spring Data JPA

Objective:

To compare the three levels of the Spring Data repository hierarchy covered in this chapter — CrudRepository

(basic CRUD), PagingAndSortingRepository (adds paging/sorting), and JpaRepository (adds JPA-specific batch

operations) — by rewriting the same repository at each level and observing the additional methods available.

Class Name(s) with Attributes:

Product (@Entity)

- productId : int (@Id @GeneratedValue)
- productName : String
- category : String
- price : double
- quantity : int

ProductRepository (progressively extends CrudRepository → PagingAndSortingRepository → JpaRepository)

ProductRestController (@RestController)

- productRepository : ProductRepository

Step-by-Step Task Instructions:

1. Create the Product entity with the listed attributes.
2. First declare ProductRepository extending CrudRepository<Product, Integer> and expose basic save/findAll endpoints.
3. Change the interface to extend PagingAndSortingRepository<Product, Integer> and add a GET /api/products/sorted endpoint using findAll(Sort.by("price")).
4. Finally change it to extend JpaRepository<Product, Integer> and demonstrate a JPA-specific method such as

saveAll() (batch insert) and flush().

5. Add a plain GET /api/products (findAll), PUT /api/products/{id} (fetch, modify, save) and DELETE /api/products/{id} (deleteById) so every level of the hierarchy is exercised through full CRUD.

6. Note in comments which methods became newly available at each stage of the hierarchy.

7. Test each version's endpoints in Postman before moving to the next stage.

Code

ProductRestController.java

```
package com.example.productapi.controller;

import java.util.List;

import org.springframework.data.domain.Sort;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import com.example.productapi.entity.Product;
import com.example.productapi.repository.ProductRepository;

@RestController
@RequestMapping("/api/products")
public class ProductRestController {

    private final ProductRepository productRepository;

    public ProductRestController(ProductRepository productRepository) {
        super();
        this.productRepository = productRepository;
    }

    @PostMapping
    public ResponseEntity<Product> createProduct(@RequestBody Product product) {

        Product savedProduct = productRepository.save(product);

        return ResponseEntity.status(HttpStatus.CREATED).body(savedProduct);
    }

    @PostMapping("/batch")
    public ResponseEntity<List<Product>> createProductsBatch(@RequestBody List<Product> products) {

        List<Product> savedProducts = productRepository.saveAll(products);

        productRepository.flush();

        return ResponseEntity.status(HttpStatus.CREATED).body(savedProducts);
    }

    @GetMapping
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
public ResponseEntity<List<Product>> getAllProducts() {
    return ResponseEntity.ok(productRepository.findAll());
}

@GetMapping("/sorted")
public ResponseEntity<List<Product>> getSortedProducts(@RequestParam(defaultValue = "price") String
sortBy) {
    List<Product> products = productRepository.findAll(Sort.by(sortBy));
    return ResponseEntity.ok(products);
}

@GetMapping("/{id}")
public ResponseEntity<Product> getProductById(@PathVariable int id) {
    return productRepository.findById(id).map(ResponseEntity::ok)
        .orElseGet(() -> ResponseEntity.notFound().build());
}

@PutMapping("/{id}")
public ResponseEntity<Product> updateProduct(@PathVariable int id, @RequestBody Product
productDetails) {
    return productRepository.findById(id).map(existingProduct -> {
        if (productDetails.getProductName() != null) {
            existingProduct.setProductName(productDetails.getProductName());
        }
        if (productDetails.getCategory() != null) {
            existingProduct.setCategory(productDetails.getCategory());
        }
        if (productDetails.getPrice() > 0) {
            existingProduct.setPrice(productDetails.getPrice());
        }
        if (productDetails.getQuantity() > 0) {
            existingProduct.setQuantity(productDetails.getQuantity());
        }
        Product updatedProduct = productRepository.save(existingProduct);
        return ResponseEntity.ok(updatedProduct);
    }).orElseGet(() -> ResponseEntity.notFound().build());
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteProduct(@PathVariable int id) {
    if (!productRepository.existsById(id)) {
        return ResponseEntity.notFound().build();
    }
    productRepository.deleteById(id);
    return ResponseEntity.noContent().build();
}
}
```

Product.java

```
package com.example.productapi.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
import jakarta.persistence.Id;

@Entity
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int productId;

    private String productName;
    private String category;
    private double price;
    private int quantity;

    public Product() {
        super();
        // TODO Auto-generated constructor stub
    }

    public Product(String productName, String category, double price, int quantity) {
        super();
        this.productName = productName;
        this.category = category;
        this.price = price;
        this.quantity = quantity;
    }

    public int getProductId() {
        return productId;
    }
    public void setProductId(int productId) {
        this.productId = productId;
    }
    public String getProductName() {
        return productName;
    }
    public void setProductName(String productName) {
        this.productName = productName;
    }
    public String getCategory() {
        return category;
    }
    public void setCategory(String category) {
        this.category = category;
    }
    public double getPrice() {
        return price;
    }
    public void setPrice(double price) {
        this.price = price;
    }
    public int getQuantity() {
        return quantity;
    }
    public void setQuantity(int quantity) {
        this.quantity = quantity;
    }

}
```

P2Application.java

```
package com.example.productapi;
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
@SpringBootApplication
public class P2Application {

    public static void main(String[] args) {
        SpringApplication.run(P2Application.class, args);
    }

}
```

ProductRepository.java

```
package com.example.productapi.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.example.productapi.entity.Product;

/*
 * Stage 1: CrudRepository<Product, Integer>
 *          Basic CRUD methods.
 *
 * Stage 2: PagingAndSortingRepository<Product, Integer>
 *          Adds paging and sorting.
 *
 * Stage 3: JpaRepository<Product, Integer>
 *          Adds JPA methods such as flush() and batch operations.
 */

public interface ProductRepository extends JpaRepository<Product,Integer> {

}
```

P2ApplicationTests.java

```
package com.example.productapi;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class P2ApplicationTests {

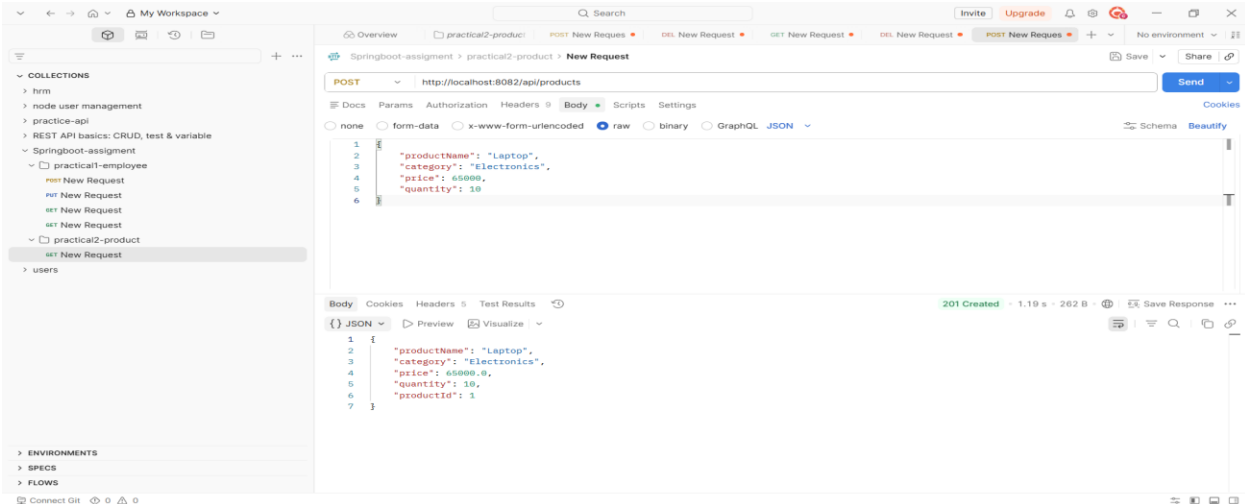
    @Test
    void contextLoads() {
    }

}
```

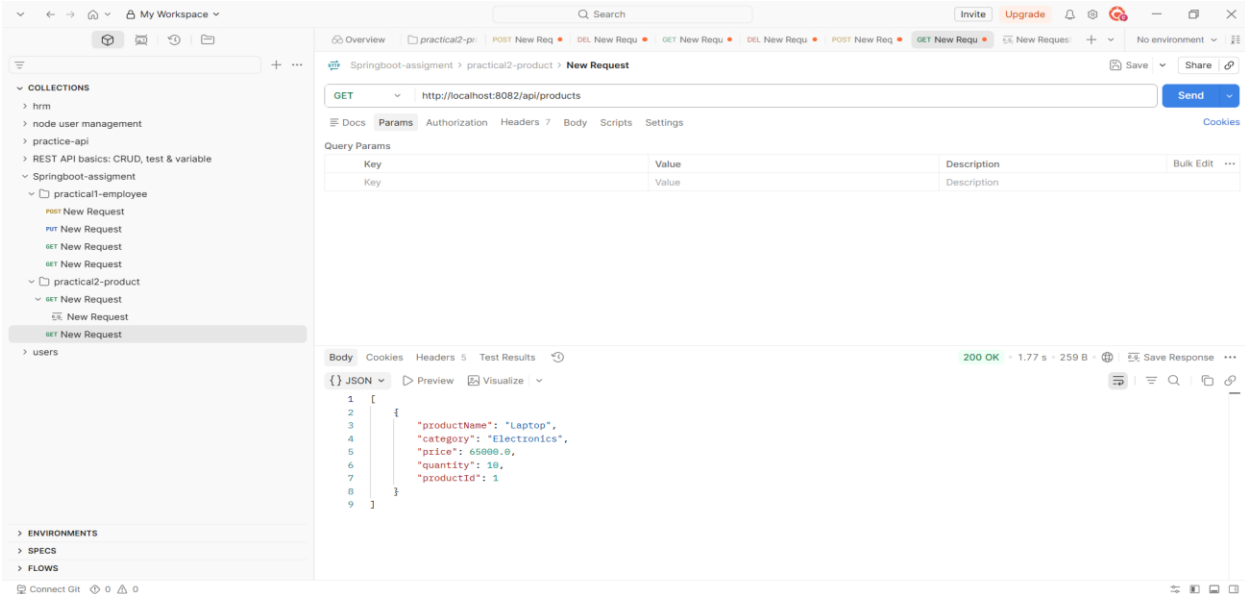
CrudRepository

Post – one product

Enroll – 25CI2110039 | Name – Kalariya Heli



Get Method



PagingAndSortingRepository

GET Method - sortBy

Enroll – 25CI2110039 | Name – Kalariya Heli

The screenshot shows a Postman interface with a GET request to `http://localhost:8082/api/products/sorted?sortBy=price`. The response is a 200 OK status with a JSON array of two products:

```
[{"productName": "Phone", "category": "Electronics", "price": 20000.0, "quantity": 10, "productId": 3}, {"productName": "Laptop1", "category": "Electronics", "price": 60000.0, "quantity": 10, "productId": 2}]
```

JpaRepository

Post – multiple products

The screenshot shows a Postman interface with a POST request to `http://localhost:8082/api/products/batch`. The request body is a JSON array of two products:

```
[{"productName": "Keyboard", "category": "Electronics", "price": 1200, "quantity": 15, "productId": 9}, {"productName": "Monitor", "category": "Electronics", "price": 9500, "quantity": 8, "productId": 8}]
```

The response is a 201 Created status.

Get - all

Springboot-assignment > practical2-product > New Request

GET Send

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 5 Test Results 200 OK · 338 ms · 916 B Save Response

```
1 [
2   {
3     "productName": "Laptop",
4     "category": "Electronics",
5     "price": 65000.0,
6     "quantity": 10,
7     "productId": 1
8   },
9   {
10    "productName": "Laptop1",
11    "category": "Electronics",
12    "price": 60000.0,
13    "quantity": 10,
14    "productId": 2
15  }
```

Get - sort by price

Springboot-assignment > practical2-product > New Request

GET Send

Query Params

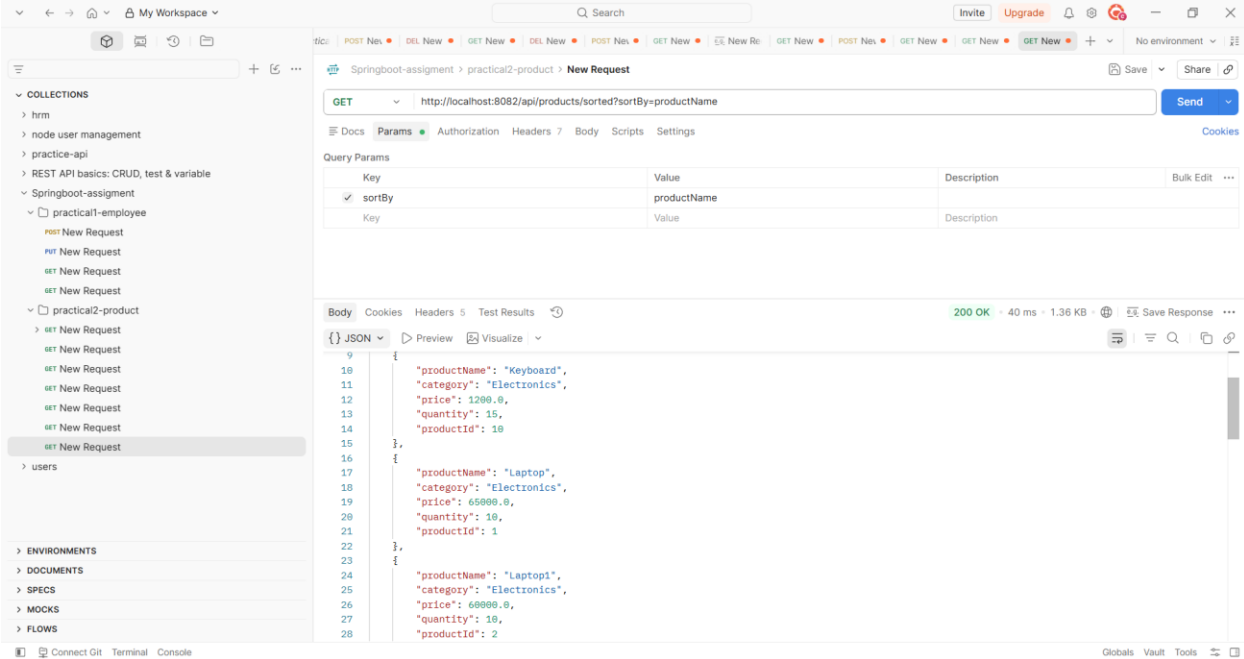
Key	Value	Description	Bulk Edit
sortBy	price		
Key	Value	Description	

Body Cookies Headers 5 Test Results 200 OK · 184 ms · 1.36 KB Save Response

```
16 {
17   "productName": "Mouse",
18   "category": "Electronics",
19   "price": 500.0,
20   "quantity": 20,
21   "productId": 4
22 },
23 {
24   "productName": "Mouse",
25   "category": "Electronics",
26   "price": 500.0,
27   "quantity": 20,
28   "productId": 9
29 },
30 {
31   "productName": "Keyboard",
32   "category": "Electronics",
33   "price": 1200.0,
34   "quantity": 15,
35   "productId": 5
36 },
37 {
38   "productName": "Keyboard",
```

Get - sort by product name

Enroll – 25CI2110039 | Name – Kalariya Heli



Springboot-assignment > practical2-product > New Request

GET http://localhost:8082/api/products/sorted?sortBy=productName

Query Params

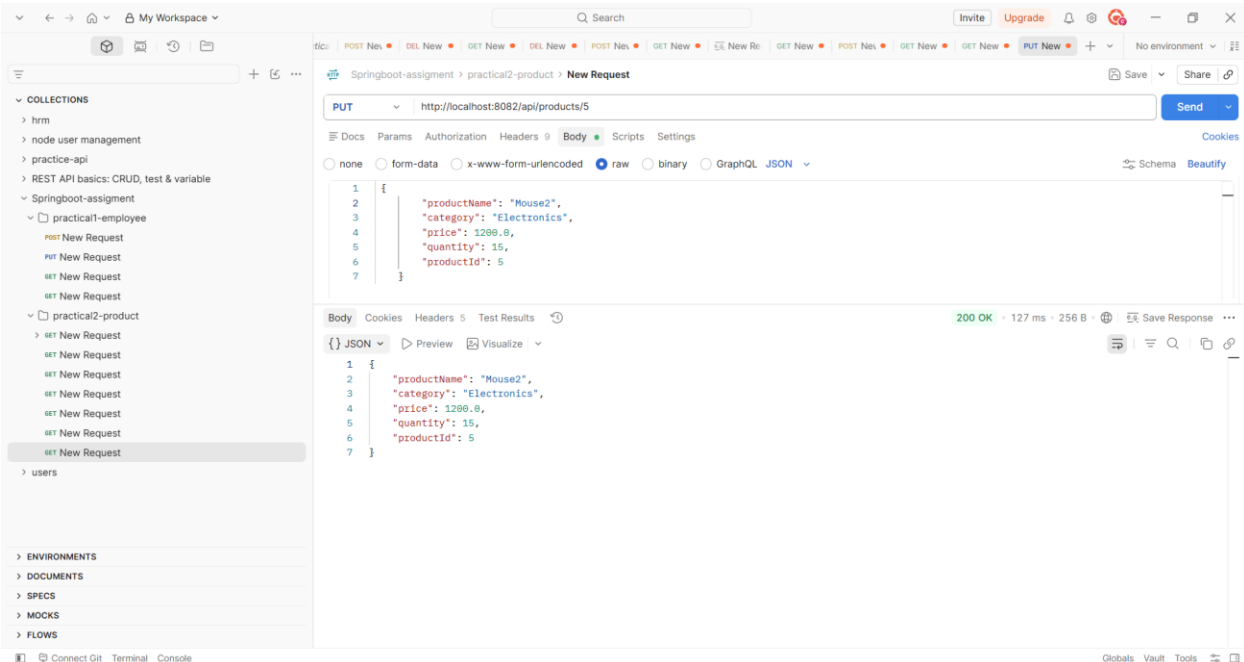
Key	Value	Description	Bulk Edit
sortBy	productName		

Body

```
200 OK - 40 ms - 1.36 KB
```

```
{
  "productName": "Keyboard",
  "category": "Electronics",
  "price": 12000.0,
  "quantity": 15,
  "productId": 10
},
{
  "productName": "Laptop",
  "category": "Electronics",
  "price": 65000.0,
  "quantity": 10,
  "productId": 1
},
{
  "productName": "Laptop1",
  "category": "Electronics",
  "price": 60000.0,
  "quantity": 10,
  "productId": 2
}
```

Put – by id



Springboot-assignment > practical2-product > New Request

PUT http://localhost:8082/api/products/5

Body

```
raw
```

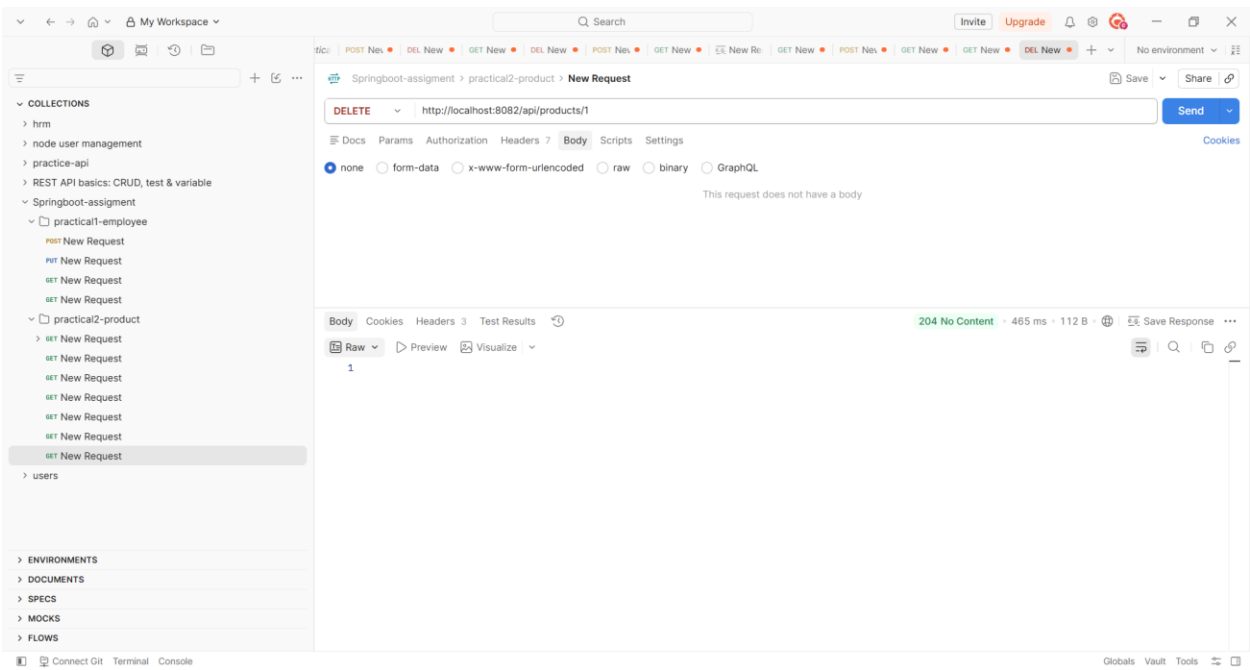
```
{
  "productName": "Mouse2",
  "category": "Electronics",
  "price": 1200.0,
  "quantity": 15,
  "productId": 5
}
```

Body

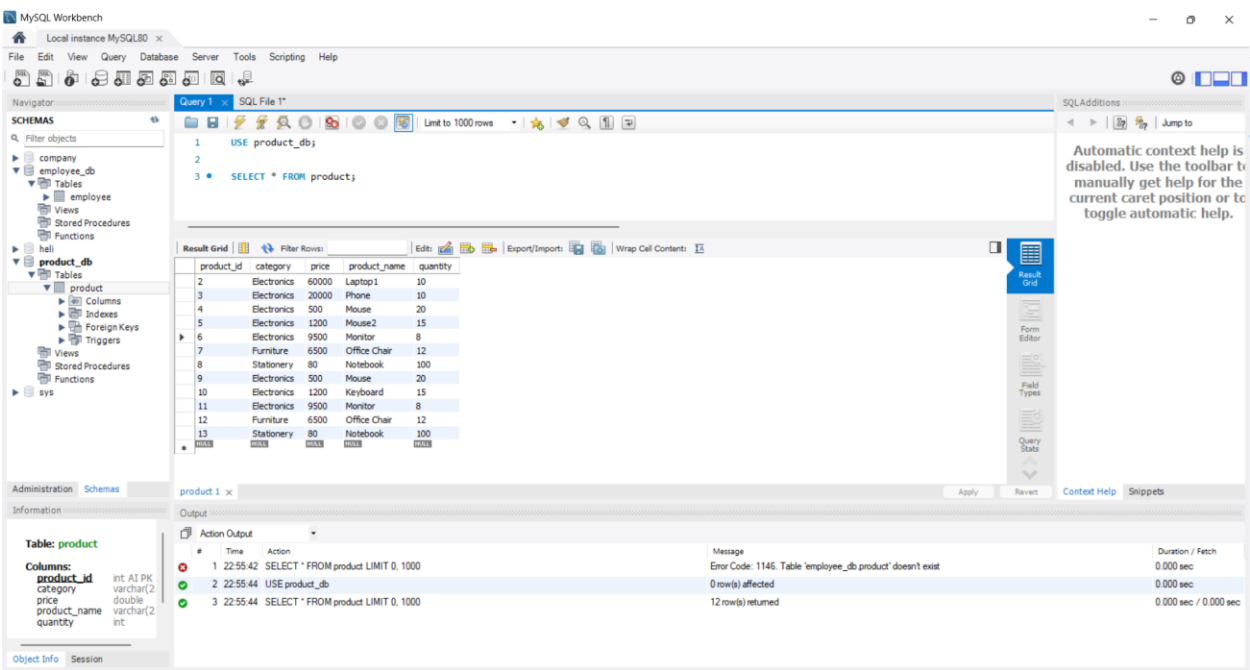
```
200 OK - 127 ms - 256 B
```

```
{
  "productName": "Mouse2",
  "category": "Electronics",
  "price": 1200.0,
  "quantity": 15,
  "productId": 5
}
```

Delete - by id



Database table - products



Method	Endpoint (Postman)	Request Body / Params	Expected Res
--------	--------------------	-----------------------	--------------

			response
P O S T	http://localhost:8083/ api/products	{"productName":"Laptop","category":"Electronics","price":65000,"quantity":10}	201 Created — Product added successfully
G E T	http://localhost:8083/ api/products	None	200 OK — Displays all product records
G E T	http://localhost:8083/ api/products/1	Path Variable: id = 1	200 OK — Displays product with

			h ID 1
GET	http://localhost:8083/ api/products/sorted?s ortBy=price	Request Param: sortBy = price	200 OK — Pro duc ts dis p lay e d i n a s c e n d i n g o r d e r o f p r i c e
GET	http://localhost:8083/ api/products/sorted?s ortBy=productName	Request Param: sortBy = productName	200 OK — Pro duc ts dis p lay e d i n a l p h a b e t i c a l o r d e r b y p r o d u c

			t na me
P O S T	http://localhost:8083/ api/products/batch	[{"productName":"Mouse","category":"Electronics","price":500,"quantity":20},{ {"productName":"Keyboard","category":"Electronics","price":1200,"quantity":15}]	201 Cre ate d — Mul tipl e pro duc ts inse rted suc cess full y usin g sav eAll ()
P U T	http://localhost:8083/ api/products/1	{"productName":"Gaming Laptop","category":"Electronics","price":75000,"quantity":8}	200 OK — Pro duc t upd ate d suc cess full y

DE LE TE	http://localhost:8083/ api/products/1	Path Variable: id = 1	204 No Con tent — Pro duc t del ete d suc cess full y
----------------	--	-----------------------	---

Practical No. 3: Custom Query Methods – findBy, countBy and @Query (JPQL)

Definition / Objective

(JPQL)

Textbook Reference: Chapter 5 — Working with Spring Data JPA

Objective:

To derive query methods automatically from method names (query derivation) and to write custom JPQL queries

using @Query for requirements that cannot be expressed by naming conventions alone, as covered in the Spring

Data JPA query-methods section of this chapter.

Class Name(s) with Attributes:

Employee (reuse from Practical 1)

- empId, empName, department, salary

EmployeeRepository (additional derived & custom methods)

- List<Employee> findByDepartment(String department)

Enroll – 25CI2110039 | Name – Kalariya Heli

- long countByDepartment(String department)
- List<Employee> findBySalaryGreaterThan(double salary)
- @Query("SELECT e FROM Employee e WHERE e.salary BETWEEN :min AND :max")
findBySalaryRange(min,max)

Step-by-Step Task Instructions:

1. Extend EmployeeRepository with derived query methods findByDepartment() and countByDepartment().
2. Add findBySalaryGreaterThan(double salary) as another derived query method.
3. Write a custom JPQL query using @Query with named parameters to fetch employees within a salary range.
4. Expose each repository method through a corresponding endpoint in EmployeeRestController, accepting path variables / request parameters.
5. Add POST /api/employees to seed sample employee records across multiple departments and salary bands,
PUT /api/employees/{id} to revise an employee's salary (so the salary-range query result changes), and
DELETE /api/employees/{id} to remove one before re-running countByDepartment().
6. Test each query endpoint from Postman and verify correctness against the seeded data.

Code

EmployeeRestController.java

```
package com.example.empqueryapi.controller;

import java.util.List;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import com.example.empqueryapi.entity.Employee;
import com.example.empqueryapi.repository.EmployeeRepository;

@RestController
@RequestMapping("/api/employees")
public class EmployeeRestController {

    private final EmployeeRepository employeeRepository;

    public EmployeeRestController(EmployeeRepository employeeRepository) {
        super();
        this.employeeRepository = employeeRepository;
    }

    @PostMapping
    public ResponseEntity<Employee> createEmployee(@RequestBody Employee employee) {
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
        Employee savedEmployee = employeeRepository.save(employee);

        return ResponseEntity.status(HttpStatus.CREATED).body(savedEmployee);
    }

    @GetMapping
    public ResponseEntity<Iterable<Employee>> getAllEmployees() {

        return ResponseEntity.ok(employeeRepository.findAll());
    }

    @GetMapping("/department/{department}")
    public ResponseEntity<List<Employee>> getByDepartment(@PathVariable String department) {

        return ResponseEntity.ok(employeeRepository.findByDepartment(department));
    }

    @GetMapping("/count/{department}")
    public ResponseEntity<Long> countByDepartment(@PathVariable String department) {

        return ResponseEntity.ok(employeeRepository.countByDepartment(department));
    }

    @GetMapping("/salary-above")
    public ResponseEntity<List<Employee>> getSalaryAbove(@RequestParam double salary) {

        return ResponseEntity.ok(employeeRepository.findBySalaryGreaterThan(salary));
    }

    @GetMapping("/salary-range")
    public ResponseEntity<List<Employee>> getSalaryRange(@RequestParam double min, @RequestParam double
max) {

        return ResponseEntity.ok(employeeRepository.findBySalaryRange(min, max));
    }

    @PutMapping("/{id}")
    public ResponseEntity<Employee> updateEmployee(@PathVariable int id, @RequestBody Employee
employeeDetails) {

        return employeeRepository.findById(id).map(existingEmployee -> {

            if (employeeDetails.getEmpName() != null) {
                existingEmployee.setEmpName(employeeDetails.getEmpName());
            }

            if (employeeDetails.getDepartment() != null) {
                existingEmployee.setDepartment(employeeDetails.getDepartment());
            }

            if (employeeDetails.getSalary() > 0) {
                existingEmployee.setSalary(employeeDetails.getSalary());
            }

            Employee updatedEmployee = employeeRepository.save(existingEmployee);

            return ResponseEntity.ok(updatedEmployee);
        }).orElseGet(() -> ResponseEntity.notFound().build());
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteEmployee(@PathVariable int id) {

        if (!employeeRepository.existsById(id)) {
            return ResponseEntity.notFound().build();
        }

        employeeRepository.deleteById(id);
    }
}
```

```
        return ResponseEntity.noContent().build();
    }
}
```

Employee.java

```
package com.example.empqueryapi.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int empId;

    private String empName;
    private String department;
    private double salary;

    public Employee() {
        super();
    }

    public Employee(String empName, String department, double salary) {
        super();
        this.empName = empName;
        this.department = department;
        this.salary = salary;
    }

    public int getEmpId() {
        return empId;
    }

    public void setEmpId(int empId) {
        this.empId = empId;
    }

    public String getEmpName() {
        return empName;
    }

    public void setEmpName(String empName) {
        this.empName = empName;
    }

    public String getDepartment() {
        return department;
    }

    public void setDepartment(String department) {
        this.department = department;
    }

    public double getSalary() {
        return salary;
    }

    public void setSalary(double salary) {
        this.salary = salary;
    }
}
```

P3Application.java

```
package com.example.empqueryapi;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class P3Application {

    public static void main(String[] args) {
        SpringApplication.run(P3Application.class, args);
    }

}
```

EmployeeRepository.java

```
package com.example.empqueryapi.repository;

import java.util.List;

import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.CrudRepository;
import org.springframework.data.repository.query.Param;

import com.example.empqueryapi.entity.Employee;

public interface EmployeeRepository
    extends CrudRepository<Employee, Integer> {

    List<Employee> findByDepartment(String department);

    long countByDepartment(String department);

    List<Employee> findBySalaryGreaterThan(double salary);

    @Query("SELECT e FROM Employee e WHERE e.salary BETWEEN :min AND :max")
    List<Employee> findBySalaryRange(
        @Param("min") double min,
        @Param("max") double max);
}
```

P3ApplicationTests.java

```
package com.example.empqueryapi;

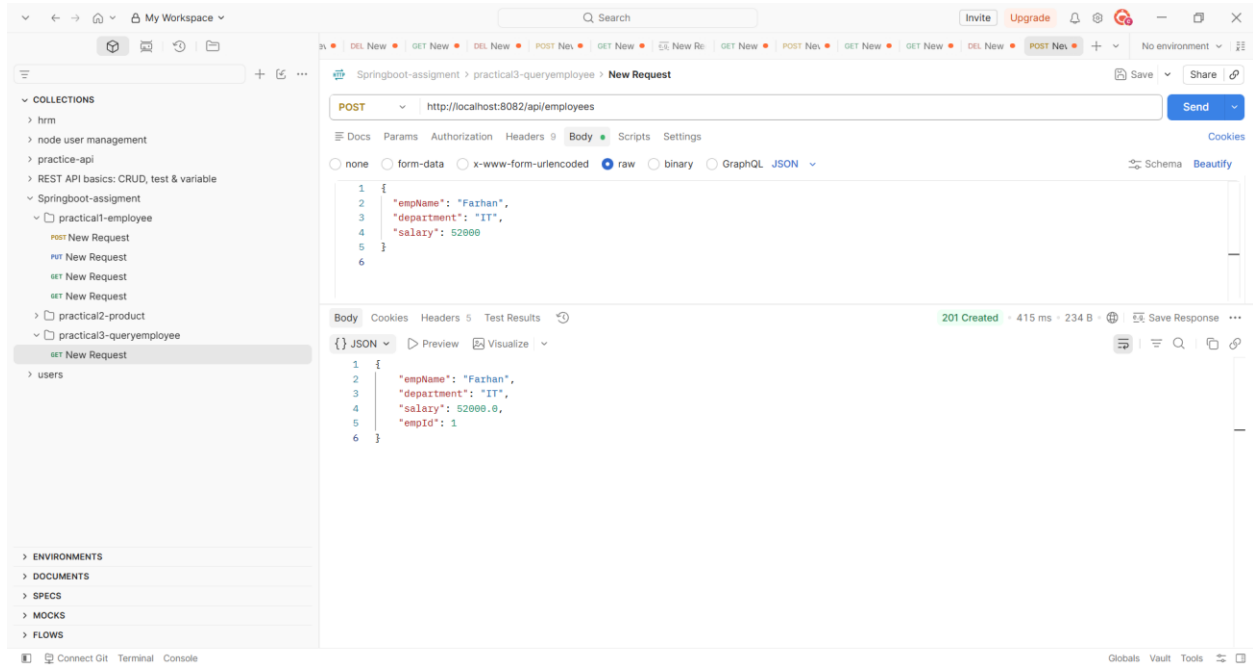
import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class P3ApplicationTests {

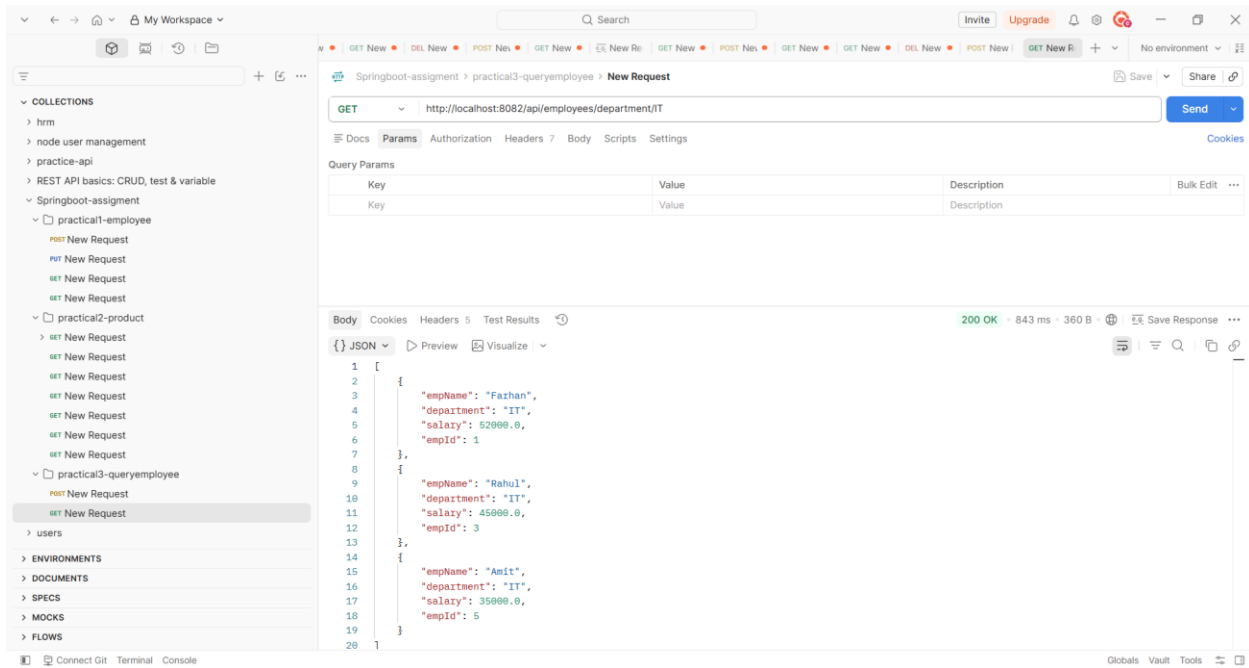
    @Test
    void contextLoads() {
    }

}
```

Post Method



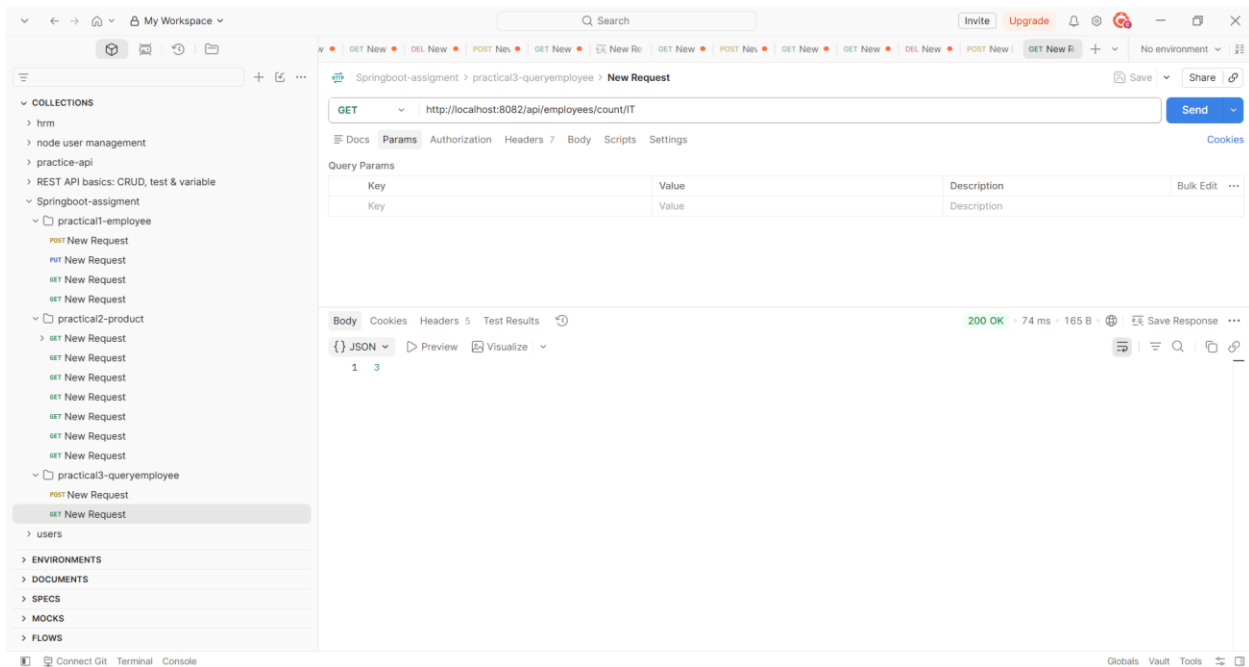
Get Method



The screenshot shows a Postman interface with a GET request to `http://localhost:8082/api/employees/department/IT`. The response is a 200 OK status with a response time of 843 ms and a body size of 360 B. The response body is a JSON array of 3 employee objects.

```
1 [{
2   {
3     "empName": "Faizan",
4     "department": "IT",
5     "salary": 52000.0,
6     "empId": 1
7   },
8   {
9     "empName": "Rahul",
10    "department": "IT",
11    "salary": 45000.0,
12    "empId": 3
13  },
14  {
15    "empName": "Amit",
16    "department": "IT",
17    "salary": 35000.0,
18    "empId": 5
19  }
20 ]
```

Get Method – countby department



The screenshot shows a Postman interface with a GET request to `http://localhost:8082/api/employees/count/IT`. The response is a 200 OK status with a response time of 74 ms and a body size of 165 B. The response body is a JSON array of 3 counts.

```
1 3
```

Get Method – salary>40000

Enroll – 25CI2110039 | Name – Kalariya Heli

Springboot-assignment > practical3-queryemployee > New Request

GET http://localhost:8082/api/employees/salary-above?salary=40000

Query Params

Key	Value	Description	Bulk Edit
salary	40000		

Body

200 OK · 42 ms · 366 B

```
{
  "employees": [
    {
      "empName": "Faizhan",
      "department": "IT",
      "salary": 52000.0,
      "empId": 1
    },
    {
      "empName": "Rahul",
      "department": "IT",
      "salary": 45000.0,
      "empId": 3
    },
    {
      "empName": "Sneha",
      "department": "Finance",
      "salary": 61000.0,
      "empId": 4
    }
  ]
}
```

Get Method - salary range JPQL query

Springboot-assignment > practical3-queryemployee > New Request

GET http://localhost:8082/api/employees/salary-range?min=30000&max=60000

Query Params

Key	Value	Description	Bulk Edit
min	30000		
max	60000		

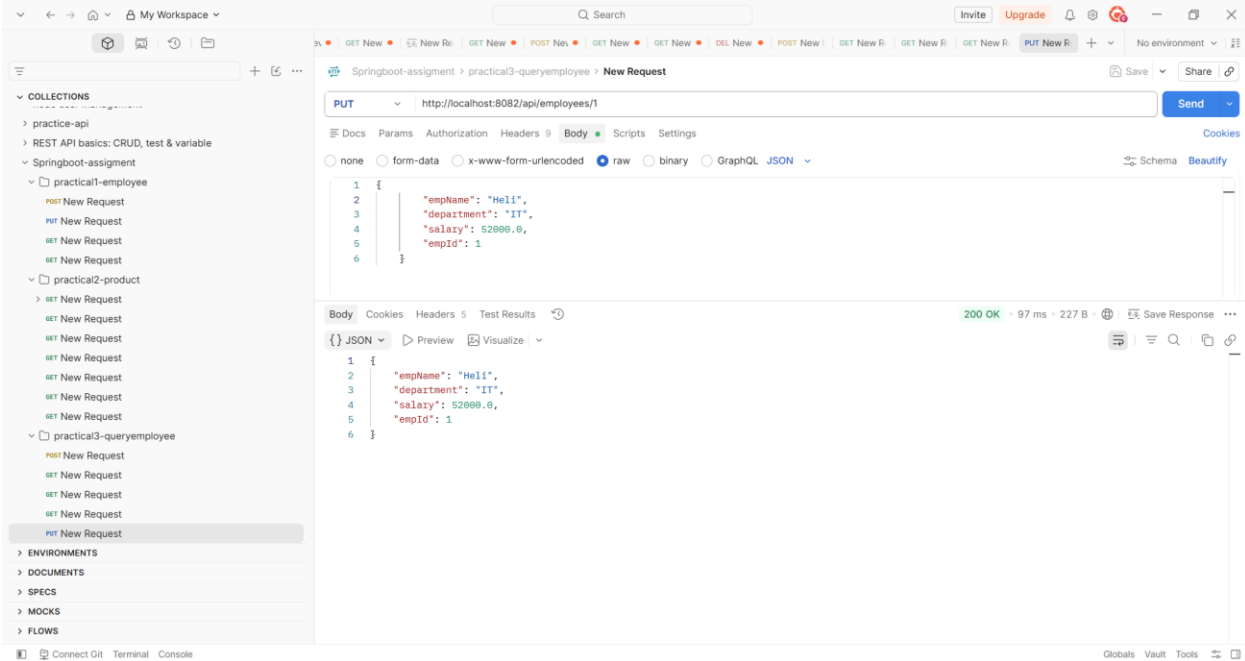
Body

200 OK · 49 ms · 425 B

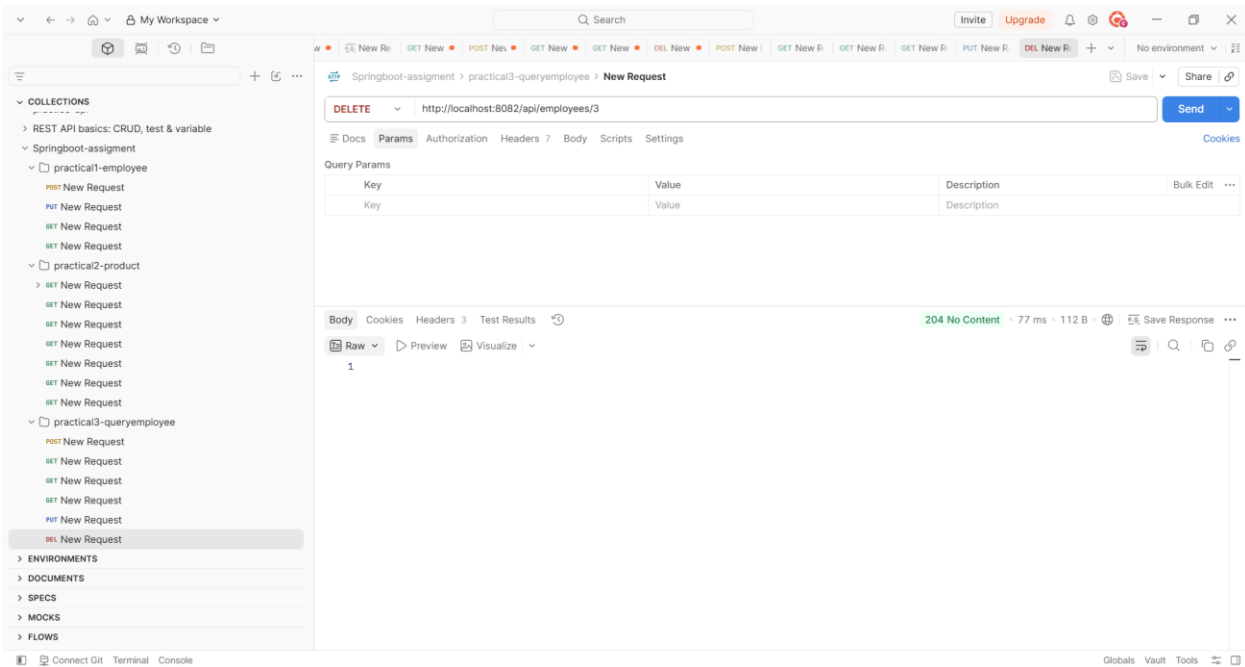
```
{
  "employees": [
    {
      "empName": "Faizhan",
      "department": "IT",
      "salary": 52000.0,
      "empId": 1
    },
    {
      "empName": "Priya",
      "department": "HR",
      "salary": 30000.0,
      "empId": 2
    },
    {
      "empName": "Rahul",
      "department": "IT",
      "salary": 45000.0,
      "empId": 3
    }
  ]
}
```

Put Method

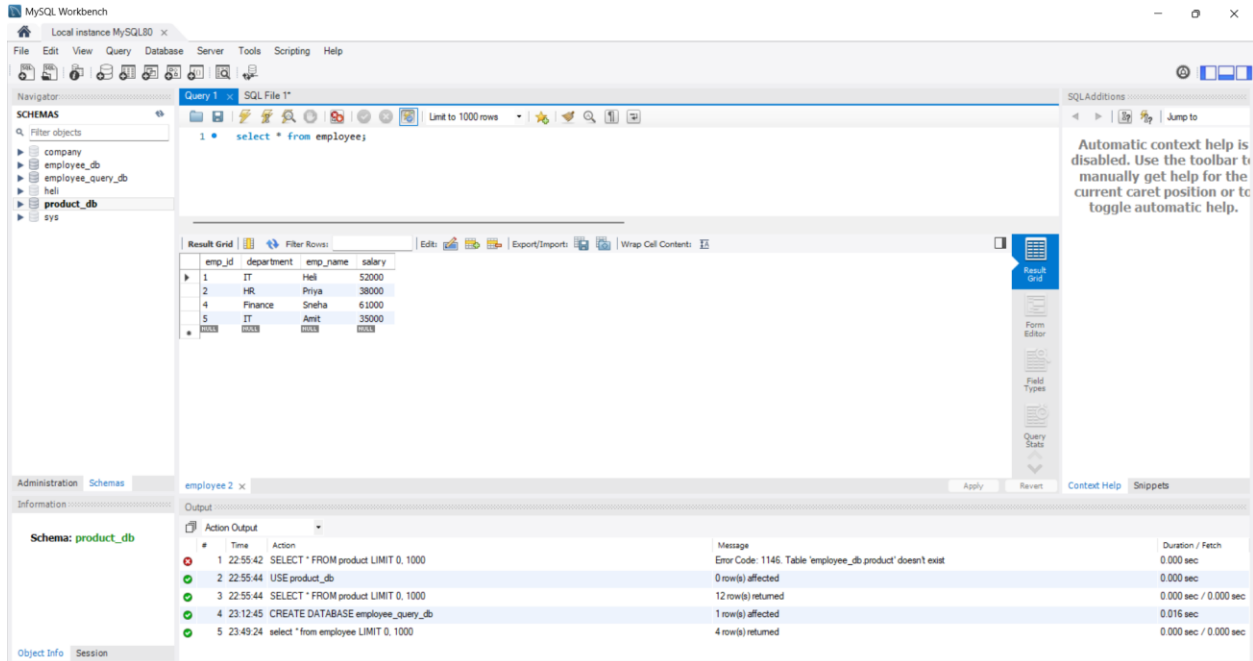
Enroll – 25CI2110039 | Name – Kalariya Heli



Delete Method



Database table of employee



Method	Endpoint (Postman)	Request Body / Params	Expected Response
POST	http://localhost:8082/api/employees	<code>{"empName":"Farhan","department":"IT","salary":52000}</code>	201 Created — Employee record saved for query testing
GET	http://localhost:8082/api/employees	None	200 OK — Displays all employees

			ee records
GET	http://localhost:8082/api/employees/department/IT	Path Variable: department = IT	200 OK — Displays all employees belonging to IT department
GET	http://localhost:8082/api/employees/count/IT	Path Variable: department = IT	200 OK — Displays numeric count of employees in IT department
GET	http://localhost:8082/api/employees/salary-above?salary=40000	Request Param: salary = 40000	200 OK — Displays employees whose salary is greater than 40000

GET	http://localhost:8082/api/employees/salary-range?min=30000&max=60000	Request Params: min = 30000, max = 60000	200 OK — Display s employ ees whose salary is between 30000 and 60000
PUT	http://localhost:8082/api/employees/1	{"salary":65000}	200 OK — Employ ee salary update d success fully
DELETE	http://localhost:8082/api/employees/3	Path Variable: id = 3	204 No Content — Employ ee deleted success fully

Practical No. 4: One-to-One Mapping – Three Worked Examples

Definition / Objective

Textbook Reference: Chapter 5 — Working with Spring Data JPA

Objective:

To implement the One-to-One relationship using `@OneToOne`, `@JoinColumn` (owning side) and `mappedBy`

(inverse side), with cascading so that saving a parent entity also persists its associated entity. Three independent

one-to-one example pairs are implemented to reinforce the pattern across different domains; full CRUD is built

and tested for the primary example (User–UserProfile).

Class Name(s) with Attributes:

User (`@Entity` — Example 1, owning side)

- `userId` : `int` (`@Id @GeneratedValue`)
- `username` : `String`
- `email` : `String`
- `userProfile` : `UserProfile` (`@OneToOne`, `cascade=ALL`, `@JoinColumn(name="profile_id")`)

UserProfile (`@Entity` — Example 1, inverse side)

- `profileId` : `int` (`@Id @GeneratedValue`)
- `address` : `String`
- `phone` : `String`
- `dob` : `LocalDate`
- `user` : `User` (`@OneToOne(mappedBy="userProfile")`)

Employee (`@Entity` — Example 2, owning side)

- `empId` : `int` (`@Id @GeneratedValue`)
- `empName` : `String`
- `department` : `String`
- `passport` : `Passport` (`@OneToOne`, `cascade=ALL`, `@JoinColumn(name="passport_id")`)

Passport (`@Entity` — Example 2, inverse side)

- `passportId` : `int` (`@Id @GeneratedValue`)
- `passportNumber` : `String`
- `issuingCountry` : `String`
- `expiryDate` : `LocalDate`
- `employee` : `Employee` (`@OneToOne(mappedBy="passport")`)

Country (`@Entity` — Example 3, owning side)

- `countryId` : `int` (`@Id @GeneratedValue`)
- `countryName` : `String`
- `capital` : `Capital` (`@OneToOne`, `cascade=ALL`, `@JoinColumn(name="capital_id")`)

Capital (@Entity — Example 3, inverse side)

- capitalId : int (@Id @GeneratedValue)
- cityName : String
- population : long
- country : Country (@OneToOne(mappedBy="capital"))

Step-by-Step Task Instructions:

1. Example 1 (User–UserProfile): Create the UserProfile entity, then the User entity holding a @OneToOne reference to UserProfile with cascade = CascadeType.ALL and @JoinColumn(name="profile_id"); add the inverse mappedBy = "userProfile" field on UserProfile.
2. Create UserRepository extending JpaRepository<User, Integer> and UserRestController exposing full CRUD:
POST /api/users (nested profile JSON saved via cascade), GET /api/users and GET /api/users/{id}, PUT /api/users/{id} (update user + nested profile), and DELETE /api/users/{id} (cascades and removes the profile too).
3. Example 2 (Employee–Passport): Repeat the identical pattern — Employee is the owning side with a @OneToOne @JoinColumn(name="passport_id") reference to Passport; Passport carries the inverse mappedBy = "passport" field. Create EmployeeRepository (or reuse) and expose the same style of CRUD endpoints under /api/employees-passport.
4. Example 3 (Country–Capital): Repeat the pattern once more — Country owns the @OneToOne @JoinColumn(name="capital_id") reference to Capital; Capital carries mappedBy = "capital". Expose analogous CRUD endpoints under /api/countries.
5. For all three examples, confirm that saving the parent (User / Employee / Country) automatically persists the associated child (UserProfile / Passport / Capital) via cascading, and that deleting the parent removes the child as well.
6. Run full CRUD Postman tests (POST, GET, PUT, DELETE) against Example 1 in detail, then repeat the POST and GET steps for Examples 2 and 3 to confirm the pattern generalizes.

Code

CountryRestController.java

```
package com.example.onetooneapi.controller;

import java.util.List;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.example.onetooneapi.Repository.CountryRepository;
import com.example.onetooneapi.entity.Country;

@RestController
@RequestMapping("/api/countries")
public class CountryRestController {

    private final CountryRepository countryRepository;

    public CountryRestController(
        CountryRepository countryRepository) {

        this.countryRepository = countryRepository;
    }

    @PostMapping
    public ResponseEntity<Country> createCountry(
        @RequestBody Country country) {

        return ResponseEntity
            .status(HttpStatus.CREATED)
            .body(countryRepository.save(country));
    }

    @GetMapping
    public ResponseEntity<List<Country>> getAllCountries() {

        return ResponseEntity.ok(
            countryRepository.findAll()
        );
    }

    @GetMapping("/{id}")
    public ResponseEntity<Country> getCountryById(
        @PathVariable int id) {

        return countryRepository.findById(id)
            .map(ResponseEntity::ok)
            .orElseGet(() ->
                ResponseEntity.notFound().build()
            );
    }
}
```

EmployeePassportController.java

```
package com.example.onetooneapi.controller;

import java.util.List;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.example.onetooneapi.Repository.EmployeeRepository;
import com.example.onetooneapi.entity.Employee;
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
@RestController
@RequestMapping("/api/employees-passport")
public class EmployeePassportController {

    private final EmployeeRepository employeeRepository;

    public EmployeePassportController(
        EmployeeRepository employeeRepository) {

        this.employeeRepository = employeeRepository;
    }

    @PostMapping
    public ResponseEntity<Employee> createEmployee(
        @RequestBody Employee employee) {

        return ResponseEntity
            .status(HttpStatus.CREATED)
            .body(employeeRepository.save(employee));
    }

    @GetMapping
    public ResponseEntity<List<Employee>> getAllEmployees() {

        return ResponseEntity.ok(
            employeeRepository.findAll()
        );
    }

    @GetMapping("/{id}")
    public ResponseEntity<Employee> getEmployeeById(
        @PathVariable int id) {

        return employeeRepository.findById(id)
            .map(ResponseEntity::ok)
            .orElseGet(() ->
                ResponseEntity.notFound().build()
            );
    }
}
```

UserRestController.java

```
package com.example.onetooneapi.controller;

import java.util.List;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.example.onetooneapi.Repository.UserRepository;
import com.example.onetooneapi.entity.User;
import com.example.onetooneapi.entity.UserProfile;

@RestController
@RequestMapping("/api/users")
public class UserRestController {

    private final UserRepository userRepository;

    public UserRestController(UserRepository userRepository) {
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
        super();
        this.userRepository = userRepository;
    }

    @PostMapping
    public ResponseEntity<User> createUser(
        @RequestBody User user) {

        User savedUser = userRepository.save(user);

        return ResponseEntity
            .status(HttpStatus.CREATED)
            .body(savedUser);
    }

    @GetMapping
    public ResponseEntity<List<User>> getAllUsers() {

        return ResponseEntity.ok(
            userRepository.findAll()
        );
    }

    @GetMapping("/{id}")
    public ResponseEntity<User> getUserById(
        @PathVariable int id) {

        return userRepository.findById(id)
            .map(ResponseEntity::ok)
            .orElseGet(() ->
                ResponseEntity.notFound().build()
            );
    }

    @PutMapping("/{id}")
    public ResponseEntity<User> updateUser(
        @PathVariable int id,
        @RequestBody User userDetails) {

        return userRepository.findById(id)
            .map(existingUser -> {

                if (userDetails.getUsername() != null) {
                    existingUser.setUsername(
                        userDetails.getUsername()
                    );
                }

                if (userDetails.getEmail() != null) {
                    existingUser.setEmail(
                        userDetails.getEmail()
                    );
                }

                if (userDetails.getUserProfile() != null) {

                    UserProfile existingProfile =
                        existingUser.getUserProfile();

                    UserProfile newProfile =
                        userDetails.getUserProfile();

                    if (existingProfile == null) {
                        existingUser.setUserProfile(newProfile);
                    } else {

                        existingProfile.setAddress(
                            newProfile.getAddress()
                        );
                    }
                }
            })
    }
```

```
                existingProfile.setPhone(
                    newProfile.getPhone()
                );

                existingProfile.setDob(
                    newProfile.getDob()
                );
            }
        }

        return ResponseEntity.ok(
            userRepository.save(existingUser)
        );
    }
    .orElseGet(() ->
        ResponseEntity.notFound().build()
    );
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteUser(
    @PathVariable int id) {

    if (!userRepository.existsById(id)) {
        return ResponseEntity.notFound().build();
    }

    userRepository.deleteById(id);

    return ResponseEntity.noContent().build();
}

}
```

Capital.java

```
package com.example.onetooneapi.entity;

import com.fasterxml.jackson.annotation.JsonBackReference;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.OneToOne;

@Entity
public class Capital {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int capitalId;

    private String cityName;
    private long population;

    @OneToOne(mappedBy = "capital")
    @JsonBackReference
    private Country country;

    public int getCapitalId() {
        return capitalId;
    }

    public void setCapitalId(int capitalId) {
        this.capitalId = capitalId;
    }
}
```

```
    public String getCityName() {
        return cityName;
    }

    public void setCityName(String cityName) {
        this.cityName = cityName;
    }

    public long getPopulation() {
        return population;
    }

    public void setPopulation(long population) {
        this.population = population;
    }

    public Country getCountry() {
        return country;
    }

    public void setCountry(Country country) {
        this.country = country;
    }
}
```

Country.java

```
package com.example.onetooneapi.entity;

import com.fasterxml.jackson.annotation.JsonManagedReference;

import jakarta.persistence.CascadeType;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.OneToOne;

@Entity
public class Country {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int countryId;

    private String countryName;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "capital_id")
    @JsonManagedReference
    private Capital capital;

    public int getCountryId() {
        return countryId;
    }

    public void setCountryId(int countryId) {
        this.countryId = countryId;
    }

    public String getCountryName() {
        return countryName;
    }

    public void setCountryName(String countryName) {
```

```
        this.countryName = countryName;
    }

    public Capital getCapital() {
        return capital;
    }

    public void setCapital(Capital capital) {
        this.capital = capital;
    }
}
```

Employee.java

```
package com.example.onetooneapi.entity;

import com.fasterxml.jackson.annotation.JsonManagedReference;

import jakarta.persistence.CascadeType;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.OneToOne;

@Entity
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int empId;

    private String empName;
    private String department;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "passport_id")
    @JsonManagedReference
    private Passport passport;

    public int getEmpId() {
        return empId;
    }

    public void setEmpId(int empId) {
        this.empId = empId;
    }

    public String getEmpName() {
        return empName;
    }

    public void setEmpName(String empName) {
        this.empName = empName;
    }

    public String getDepartment() {
        return department;
    }

    public void setDepartment(String department) {
        this.department = department;
    }

    public Passport getPassport() {
        return passport;
    }
}
```

```
    }

    public void setPassport(Passport passport) {
        this.passport = passport;
    }

}
```

Passport.java

```
package com.example.onetooneapi.entity;

import java.time.LocalDate;

import com.fasterxml.jackson.annotation.JsonBackReference;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.OneToOne;

@Entity
public class Passport {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int passportId;

    private String passportNumber;
    private String issuingCountry;
    private LocalDate expiryDate;

    @OneToOne(mappedBy = "passport")
    @JsonBackReference
    private Employee employee;

    public Passport() {
        super();
        // TODO Auto-generated constructor stub
    }

    public int getPassportId() {
        return passportId;
    }

    public void setPassportId(int passportId) {
        this.passportId = passportId;
    }

    public String getPassportNumber() {
        return passportNumber;
    }

    public void setPassportNumber(String passportNumber) {
        this.passportNumber = passportNumber;
    }

    public String getIssuingCountry() {
        return issuingCountry;
    }

    public void setIssuingCountry(String issuingCountry) {
        this.issuingCountry = issuingCountry;
    }

    public LocalDate getExpiryDate() {
        return expiryDate;
    }
}
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
    }

    public void setExpiryDate(LocalDate expiryDate) {
        this.expiryDate = expiryDate;
    }

    public Employee getEmployee() {
        return employee;
    }

    public void setEmployee(Employee employee) {
        this.employee = employee;
    }
}
```

User.java

```
package com.example.onetooneapi.entity;

import com.fasterxml.jackson.annotation.JsonManagedReference;

import jakarta.persistence.CascadeType;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.OneToOne;

@Entity
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int userId;

    private String username;
    private String email;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "profile_id")
    @JsonManagedReference
    private UserProfile userProfile;

    public User() {
        super();
        // TODO Auto-generated constructor stub
    }

    public int getUserId() {
        return userId;
    }

    public void setUserId(int userId) {
        this.userId = userId;
    }

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getEmail() {
```

```
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public UserProfile getUserProfile() {
        return userProfile;
    }

    public void setUserProfile(UserProfile userProfile) {
        this.userProfile = userProfile;
    }
}
```

UserProfile.java

```
package com.example.onetooneapi.entity;

import java.time.LocalDate;

import com.fasterxml.jackson.annotation.JsonBackReference;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.OneToOne;

@Entity
public class UserProfile {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int profileId;

    private String address;
    private String phone;
    private LocalDate dob;

    @OneToOne(mappedBy = "userProfile")
    @JsonBackReference
    private User user;

    public UserProfile() {
        super();
        // TODO Auto-generated constructor stub
    }

    public int getProfileId() {
        return profileId;
    }

    public void setProfileId(int profileId) {
        this.profileId = profileId;
    }

    public String getAddress() {
        return address;
    }

    public void setAddress(String address) {
        this.address = address;
    }
}
```

```
    public String getPhone() {
        return phone;
    }

    public void setPhone(String phone) {
        this.phone = phone;
    }

    public LocalDate getDob() {
        return dob;
    }

    public void setDob(LocalDate dob) {
        this.dob = dob;
    }

    public User getUser() {
        return user;
    }

    public void setUser(User user) {
        this.user = user;
    }
}
```

P4Application.java

```
package com.example.onetooneapi;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class P4Application {

    public static void main(String[] args) {
        SpringApplication.run(P4Application.class, args);
    }
}
```

CountryRepository.java

```
package com.example.onetooneapi.Repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.example.onetooneapi.entity.Country;

public interface CountryRepository
    extends JpaRepository<Country, Integer> {

}
```

EmployeeRepository.java

```
package com.example.onetooneapi.Repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.example.onetooneapi.entity.Employee;

public interface EmployeeRepository
    extends JpaRepository<Employee, Integer> {

}
```

UserRepository.java

Enroll – 25CI2110039 | Name – Kalariya Heli

```
package com.example.onetooneapi.Repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.example.onetooneapi.entity.User;

public interface UserRepository
    extends JpaRepository<User, Integer> {

}
```

P4ApplicationTests.java

```
package com.example.onetooneapi;

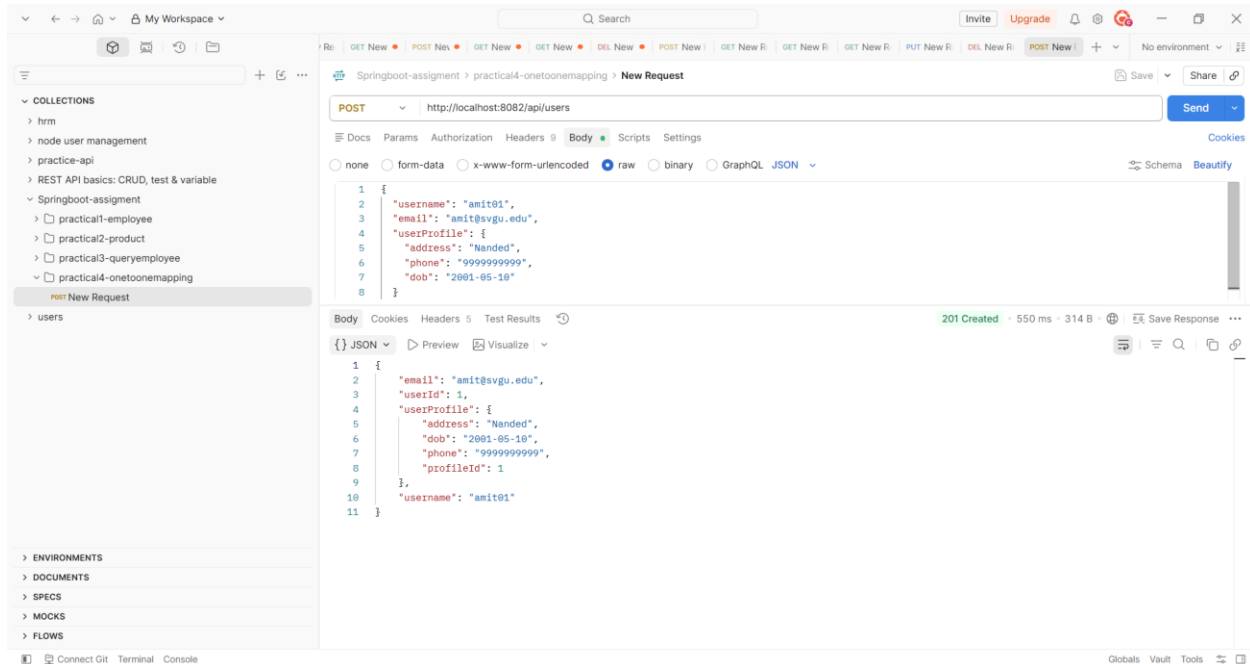
import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class P4ApplicationTests {

    @Test
    void contextLoads() {
    }

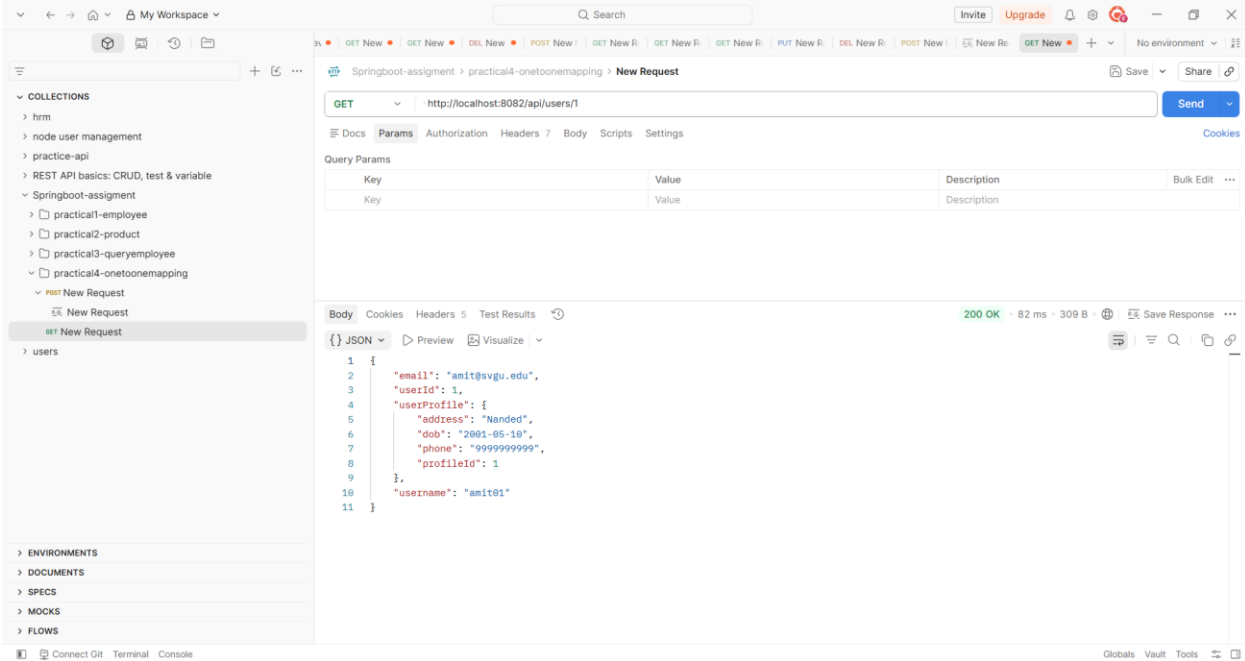
}
```

Post Method – user & userprofile



Get Method

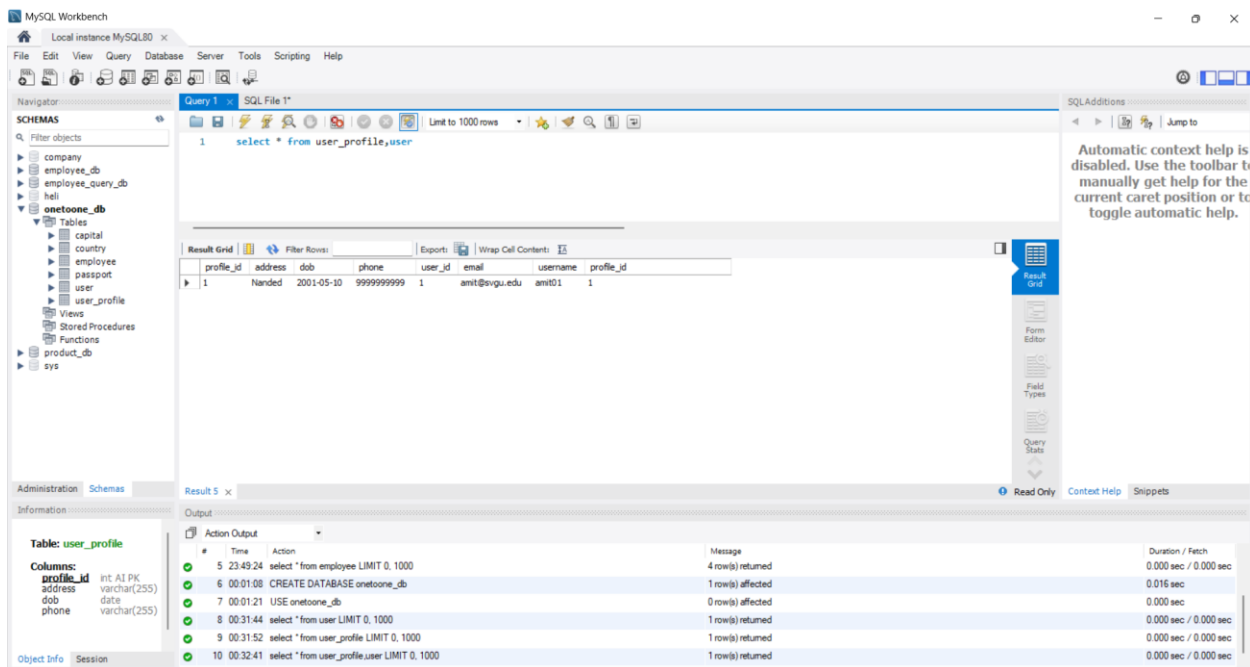
Enroll – 25CI2110039 | Name – Kalariya Heli



The screenshot shows a REST client interface with a GET request to `http://localhost:8082/api/users/1`. The response is a JSON object with the following structure:

```
1 {
2   "email": "amit@svgu.edu",
3   "userId": 1,
4   "userProfile": {
5     "address": "Nanded",
6     "dob": "2001-05-10",
7     "phone": "9999999999",
8     "profileId": 1
9   },
10  "username": "amit01"
11 }
```

Database table of user, user_profile



The screenshot shows MySQL Workbench with the `user_profile` table selected. The table structure is as follows:

Column	Type	PK
profile_id	int	AI PK
address	varchar(255)	
dob	date	
phone	varchar(255)	

The query `select * from user_profile, user` is executed, and the result is shown in the Result Grid:

profile_id	address	dob	phone	user_id	email	username	profile_id
1	Nanded	2001-05-10	9999999999	1	amit@svgu.edu	amit01	1

The Action Output shows the following actions:

#	Time	Action	Message	Duration / Fetch
5	23:49:24	select * from employee LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
6	00:01:08	CREATE DATABASE onetone_db	1 row(s) affected	0.016 sec
7	00:01:21	USE onetone_db	0 row(s) affected	0.000 sec
8	00:31:44	select * from user LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
9	00:31:52	select * from user_profile LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
10	00:32:41	select * from user_profile, user LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec

Put Method

Enroll – 25CI2110039 | Name – Kalariya Heli

Springboot-assignment > practical4-one-to-one-mapping > New Request

PUT http://localhost:8082/api/users/1

Body

```
1 {
2   "email": "amit.new@svgu.edu",
3   "userProfile": {
4     "address": "Pune",
5     "phone": "8080808080",
6     "dob": "2001-05-10"
7   }
8 }
```

Body

```
1 {
2   "email": "amit.new@svgu.edu",
3   "userId": 1,
4   "userProfile": {
5     "address": "Pune",
6     "dob": "2001-05-10",
7     "phone": "8080808080",
8     "profileId": 1
9   },
10  "username": "amit01"
11 }
```

200 OK · 91 ms · 311 B

Delete Method

Springboot-assignment > practical4-one-to-one-mapping > New Request

DELETE http://localhost:8082/api/users/1

Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body

204 No Content · 733 ms · 112 B

Post Method

Enroll – 25CI2110039 | Name – Kalariya Heli

Springboot-assignment > practical4-one-to-one-mapping > New Request

POST http://localhost:8082/api/employees-passport

Body

```
2 {
3   "empName": "Rakesh",
4   "department": "HR",
5   "passport": {
6     "passportNumber": "N1234567",
7     "issuingCountry": "India",
8     "expiryDate": "2030-01-01"
9   }
}
```

201 Created · 67 ms · 324 B

Body

```
1 {
2   "department": "HR",
3   "empId": 1,
4   "empName": "Rakesh",
5   "passport": {
6     "expiryDate": "2030-01-01",
7     "issuingCountry": "India",
8     "passportId": 1,
9     "passportNumber": "N1234567"
10  }
11 }
```

Get Method

Springboot-assignment > practical4-one-to-one-mapping > New Request

GET http://localhost:8082/api/employees-passport/1

Params

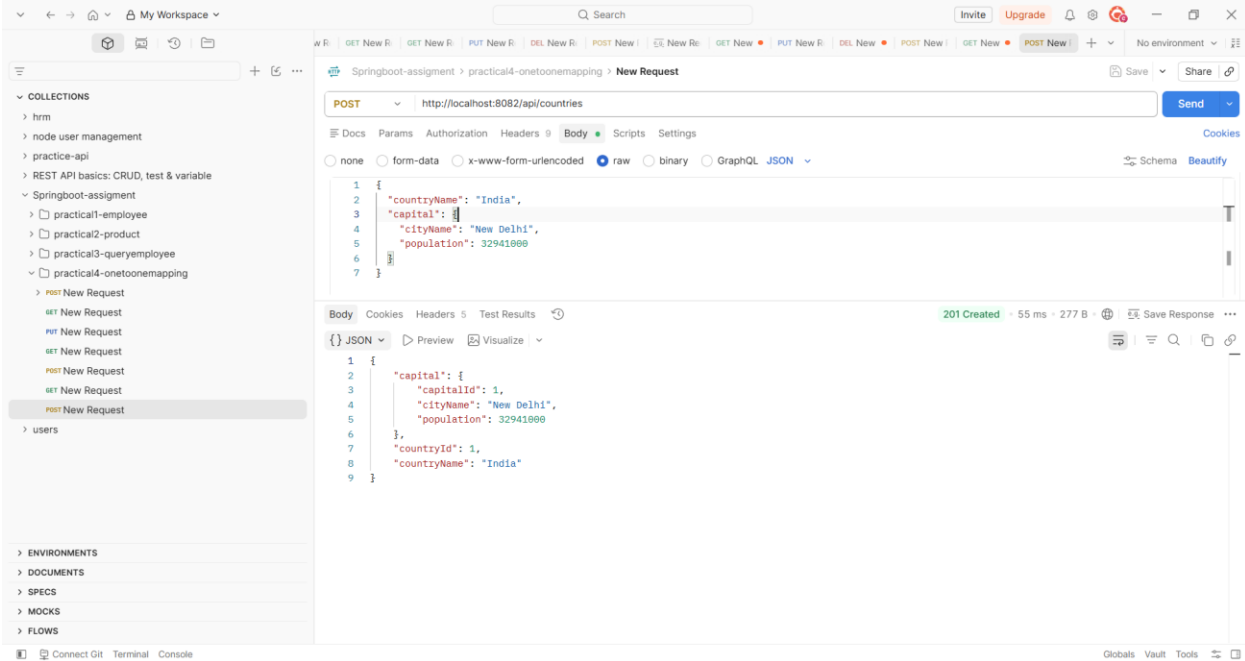
Key	Value	Description	Bulk Edit
Key	Value	Description	

200 OK · 24 ms · 319 B

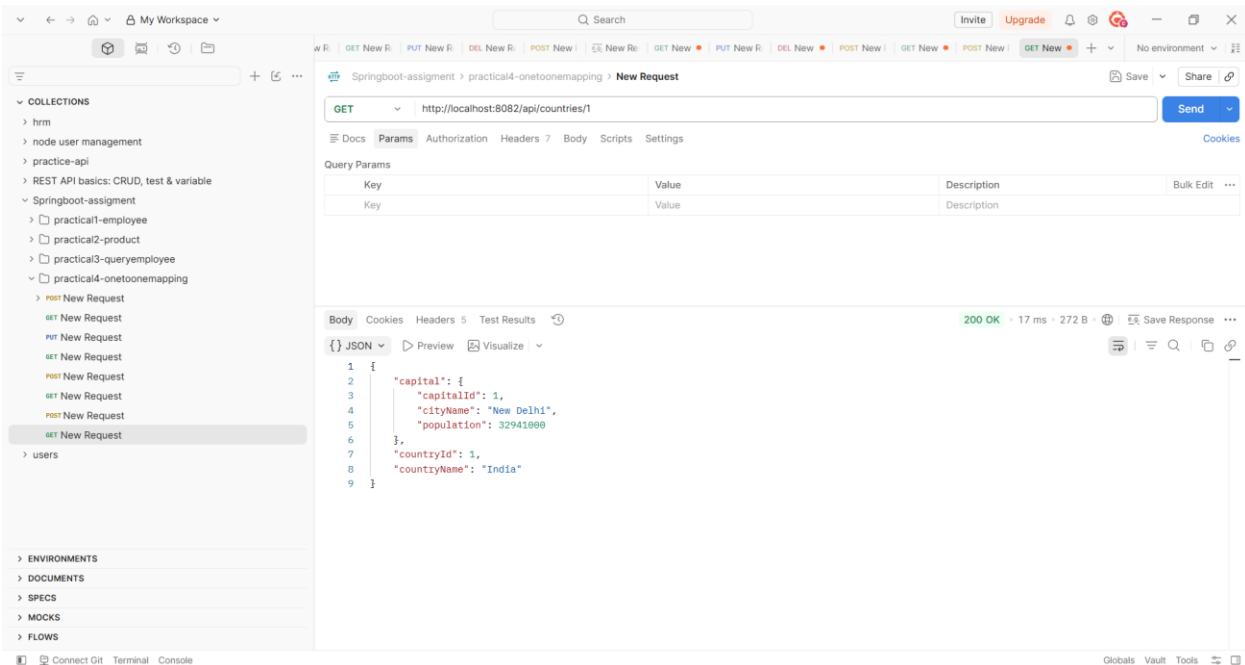
Body

```
1 {
2   "department": "HR",
3   "empId": 1,
4   "empName": "Rakesh",
5   "passport": {
6     "expiryDate": "2030-01-01",
7     "issuingCountry": "India",
8     "passportId": 1,
9     "passportNumber": "N1234567"
10  }
11 }
```

Post Method



Get Method



Method	Endpoint (Postman)	Request Body / Params	Expected Response
--------	--------------------	-----------------------	-------------------

POST	http://localhost:8082/api/users	<code>{"username":"amit01","email":"amit@svgu.edu","userProfile":{"address":"Nanded","phone":"9999999999","dob":"2001-05-10"}}</code>	201 Created — User and nested User Profile saved through cascade
GET	http://localhost:8082/api/users/1	Path Variable: id = 1	200 OK — Displays User with embedded User Profile
PUT	http://localhost:8082/api/users/1	<code>{"email":"amit.new@svgu.edu","userProfile":{"address":"Pune","phone":"8888888888","dob":"2001-05-10"}}</code>	200 OK — User and nested

			User Profile updated
DELETE	http://localhost:8082/api/users/1	Path Variable: id = 1	204 No Content — User and associated User Profile deleted through cascade
POST	http://localhost:8082/api/employees-passport	{"empName":"Rakesh","department":"HR","passport":{"passportNumber":"N1234567","issuingCountry":"India","expiryDate":"2030-01-01"}}	201 Created — Employee and nested Passport saved

GET	http://localhost:8082/api/employees-passport/1	Path Variable: id = 1	200 OK — Displays Employee with associated Passport
POST	http://localhost:8082/api/countries	{"countryName":"India","capital":{"cityName":"New Delhi"},"population":32941000}}	201 Created — Country and nested Capital saved
GET	http://localhost:8082/api/countries/1	Path Variable: id = 1	200 OK — Displays Country with associated

			Capital
--	--	--	---------

Practical No. 5: One-to-Many / Many-to-One Mapping – Department and Employee

Definition / Objective

Employee

Textbook Reference: Chapter 5 — Working with Spring Data JPA

Objective:

To implement a bidirectional One-to-Many (Department → Employees) and Many-to-One (Employee → Department) relationship using mappedBy on the parent side and @JoinColumn on the child side, and to persist a

parent with a list of children in one request.

Class Name(s) with Attributes:

Department (@Entity — parent, One side)

- deptId : int (@Id @GeneratedValue)

- deptName : String

- employees : List<Employee> (@OneToMany(mappedBy="department", cascade=ALL))

Employee (@Entity — child, Many side)

- empId : int (@Id @GeneratedValue)

- empName : String

- salary : double

- department : Department (@ManyToOne @JoinColumn(name="dept_id"))

Step-by-Step Task Instructions:

1. Modify (or recreate) the Employee entity to add a @ManyToOne field department with @JoinColumn(name = "dept_id").

2. Create the Department entity with a @OneToMany(mappedBy = "department", cascade = CascadeType.ALL) list of employees.

3. Create DepartmentRepository extending JpaRepository<Department, Integer>.

4. Create DepartmentRestController exposing POST /api/departments that accepts a department name plus a list

of employee objects, and saves the whole graph via cascading (remember to set employee.setDepartment(department) on each child before saving).

5. Add GET /api/departments/{id} to fetch a department with its employees, and GET /api/departments/{id}/employees to fetch only the employee list.

Enroll – 25CI2110039 | Name – Kalariya Heli

6. Add PUT `/api/departments/{deptId}/employees/{empId}` to reassign or update an existing employee's details

within a department (Many-to-One navigation from Employee back to Department), and PUT `/api/departments/{id}` to rename a department.

7. Add DELETE `/api/departments/{deptId}/employees/{empId}` to remove a single employee from a department

(Many-to-One side), and DELETE `/api/departments/{id}` to remove a department along with all its employees

via cascade (One-to-Many side).

8. Test the parent-with-children creation, both read endpoints, both update endpoints, and both delete endpoints

in Postman.

Code

DepartmentRestController.java

```
package com.example.departmentapi.controller;

import java.util.List;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.example.departmentapi.entity.Department;
import com.example.departmentapi.entity.Employee;
import com.example.departmentapi.repository.DepartmentRepository;

@RestController
@RequestMapping("/api/departments")
public class DepartmentRestController {

    private final DepartmentRepository departmentRepository;

    public DepartmentRestController(DepartmentRepository departmentRepository) {
        this.departmentRepository = departmentRepository;
    }

    @PostMapping
    public ResponseEntity<Department> createDepartment(@RequestBody Department department) {

        for (Employee employee : department.getEmployees()) {
            employee.setDepartment(department);
        }

        Department saved = departmentRepository.save(department);

        return ResponseEntity.status(HttpStatus.CREATED).body(saved);
    }

    @GetMapping("/{id}")
    public ResponseEntity<Department> getDepartmentById(@PathVariable int id) {
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
        return departmentRepository.findById(id).map(ResponseEntity::ok)
            .orElseGet(() -> ResponseEntity.notFound().build());
    }

    @GetMapping("/{id}/employees")
    public ResponseEntity<List<Employee>> getEmployees(
        @PathVariable int id) {

        Department department =
            departmentRepository.findById(id).orElse(null);

        if (department == null) {
            return ResponseEntity.notFound().build();
        }

        return ResponseEntity.ok(department.getEmployees());
    }

    @PutMapping("/{id}")
    public ResponseEntity<Department> updateDepartment(@PathVariable int id, @RequestBody Department
    details) {

        return departmentRepository.findById(id).map(department -> {

            department.setDeptName(details.getDeptName());

            return ResponseEntity.ok(departmentRepository.save(department));
        }).orElseGet(() -> ResponseEntity.notFound().build());
    }

    @PutMapping("/{deptId}/employees/{empId}")
    public ResponseEntity<Employee> updateEmployee(
        @PathVariable int deptId,
        @PathVariable int empId,
        @RequestBody Employee details) {

        Department department =
            departmentRepository.findById(deptId).orElse(null);

        if (department == null) {
            return ResponseEntity.notFound().build();
        }

        Employee employee = department.getEmployees()
            .stream()
            .filter(e -> e.getEmpId() == empId)
            .findFirst()
            .orElse(null);

        if (employee == null) {
            return ResponseEntity.notFound().build();
        }

        if (details.getEmpName() != null) {
            employee.setEmpName(details.getEmpName());
        }

        if (details.getSalary() > 0) {
            employee.setSalary(details.getSalary());
        }

        departmentRepository.save(department);

        return ResponseEntity.ok(employee);
    }

    @DeleteMapping("/{deptId}/employees/{empId}")
    public ResponseEntity<Void> deleteEmployee(
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
        @PathVariable int deptId,
        @PathVariable int empId) {

    Department department =
        departmentRepository.findById(deptId).orElse(null);

    if (department == null) {
        return ResponseEntity.notFound().build();
    }

    Employee employee = department.getEmployees()
        .stream()
        .filter(e -> e.getEmpId() == empId)
        .findFirst()
        .orElse(null);

    if (employee == null) {
        return ResponseEntity.notFound().build();
    }

    department.getEmployees().remove(employee);

    departmentRepository.save(department);

    return ResponseEntity.noContent().build();
}
@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteDepartment(@PathVariable int id) {

    if (!departmentRepository.existsById(id)) {
        return ResponseEntity.notFound().build();
    }

    departmentRepository.deleteById(id);

    return ResponseEntity.noContent().build();
}
}
```

Department.java

```
package com.example.departmentapi.entity;

import java.util.ArrayList;
import java.util.List;

import com.fasterxml.jackson.annotation.JsonManagedReference;

import jakarta.persistence.CascadeType;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.OneToMany;

@Entity
public class Department {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int deptId;

    private String deptName;

    @OneToMany(
        mappedBy = "department",
        cascade = CascadeType.ALL,
        orphanRemoval = true
    )
}
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
@JsonManagedReference
private List<Employee> employees = new ArrayList<>();

public Department() {
}

public int getDeptId() {
    return deptId;
}

public void setDeptId(int deptId) {
    this.deptId = deptId;
}

public String getDeptName() {
    return deptName;
}

public void setDeptName(String deptName) {
    this.deptName = deptName;
}

public List<Employee> getEmployees() {
    return employees;
}

public void setEmployees(List<Employee> employees) {
    this.employees = employees;
}
}
```

Employee.java

```
package com.example.departmentapi.entity;

import com.fasterxml.jackson.annotation.JsonBackReference;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;

@Entity
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int empId;

    private String empName;

    private double salary;

    @ManyToOne
    @JoinColumn(name = "dept_id")
    @JsonBackReference
    private Department department;

    public Employee() {
        super();
        // TODO Auto-generated constructor stub
    }

    public int getEmpId() {
        return empId;
    }
}
```

```
    public void setEmpId(int empId) {
        this.empId = empId;
    }

    public String getEmpName() {
        return empName;
    }

    public void setEmpName(String empName) {
        this.empName = empName;
    }

    public double getSalary() {
        return salary;
    }

    public void setSalary(double salary) {
        this.salary = salary;
    }

    public Department getDepartment() {
        return department;
    }

    public void setDepartment(Department department) {
        this.department = department;
    }
}
```

P5Application.java

```
package com.example.departmentapi;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class P5Application {

    public static void main(String[] args) {
        SpringApplication.run(P5Application.class, args);
    }

}
```

DepartmentRepository.java

```
package com.example.departmentapi.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.example.departmentapi.entity.Department;

public interface DepartmentRepository
    extends JpaRepository<Department, Integer> {

}
```

P5ApplicationTests.java

```
package com.example.departmentapi;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class P5ApplicationTests {
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
@Test
void contextLoads() {
}

}
```

Post Method

Springboot-assignment > practical5-department > New Request

POST http://localhost:8082/api/departments

Body

```
1 {
2   "deptName": "Computer Science",
3   "employees": [
4     {
5       "empName": "Riya",
6       "salary": 42000
7     },
8   ]
9 }
```

201 Created - 796 ms - 318 B

JSON

```
1 {
2   "deptId": 1,
3   "deptName": "Computer Science",
4   "employees": [
5     {
6       "empId": 1,
7       "empName": "Riya",
8       "salary": 42000.0
9     },
10    {
11      "empId": 2,
12      "empName": "Karan",
13      "salary": 39000.0
14    }
15  ]
16 }
```

Get Method - Department

Springboot-assignment > practical5-department > New Request

GET http://localhost:8082/api/departments/1

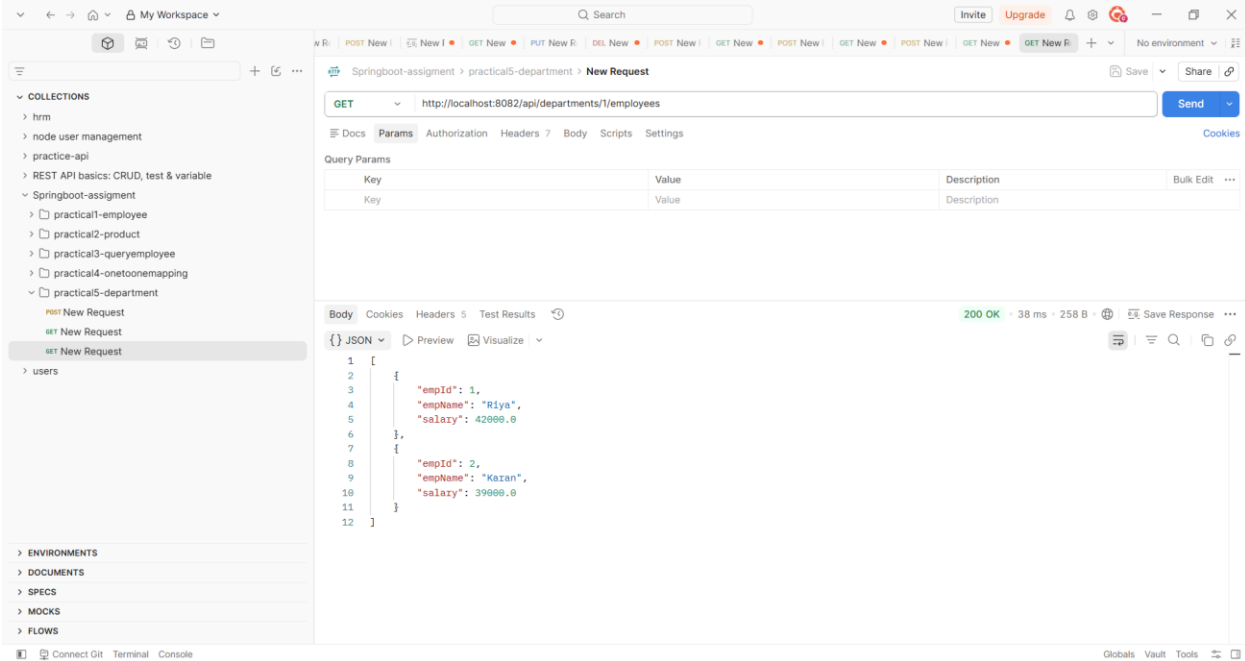
Body

```
1 {
2   "deptId": 1,
3   "deptName": "Computer Science",
4   "employees": [
5     {
6       "empId": 1,
7       "empName": "Riya",
8       "salary": 42000.0
9     },
10    {
11      "empId": 2,
12      "empName": "Karan",
13      "salary": 39000.0
14    }
15  ]
16 }
```

200 OK - 179 ms - 313 B

Get Method - Employees

Enroll – 25CI2110039 | Name – Kalariya Heli



Springboot-assignment > practical5-department > New Request

GET http://localhost:8082/api/departments/1/employees

Query Params

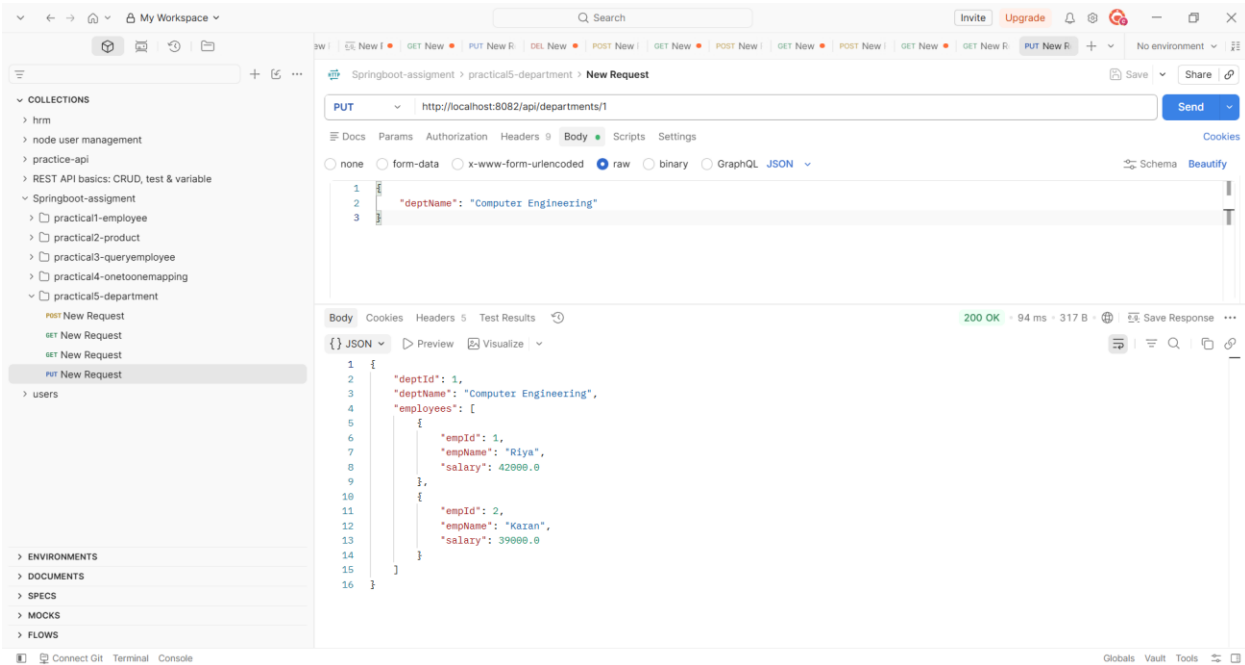
Key	Value	Description	Bulk Edit
Key	Value	Description	

Body

```
{
  "empId": 1,
  "empName": "Riya",
  "salary": 42000.0
},
{
  "empId": 2,
  "empName": "Karan",
  "salary": 39000.0
}
```

200 OK · 38 ms · 258 B

Put Method



Springboot-assignment > practical5-department > New Request

PUT http://localhost:8082/api/departments/1

Body

```
{
  "deptName": "Computer Engineering"
}
```

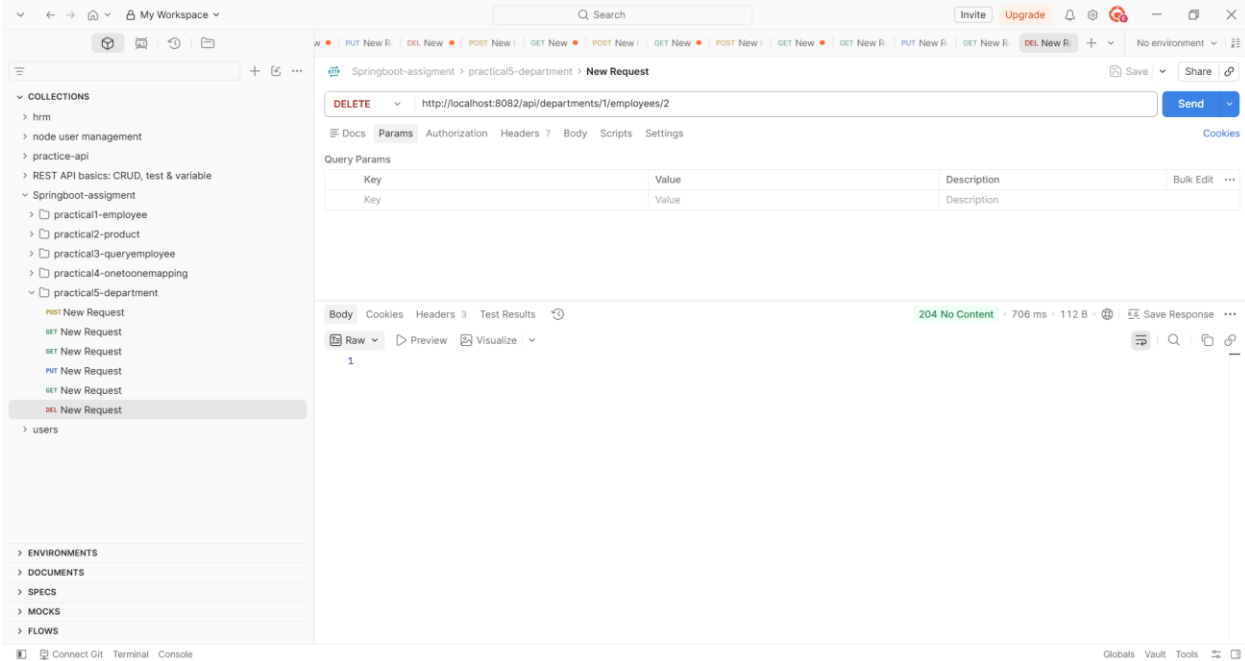
Body

```
{
  "deptId": 1,
  "deptName": "Computer Engineering",
  "employees": [
    {
      "empId": 1,
      "empName": "Riya",
      "salary": 42000.0
    },
    {
      "empId": 2,
      "empName": "Karan",
      "salary": 39000.0
    }
  ]
}
```

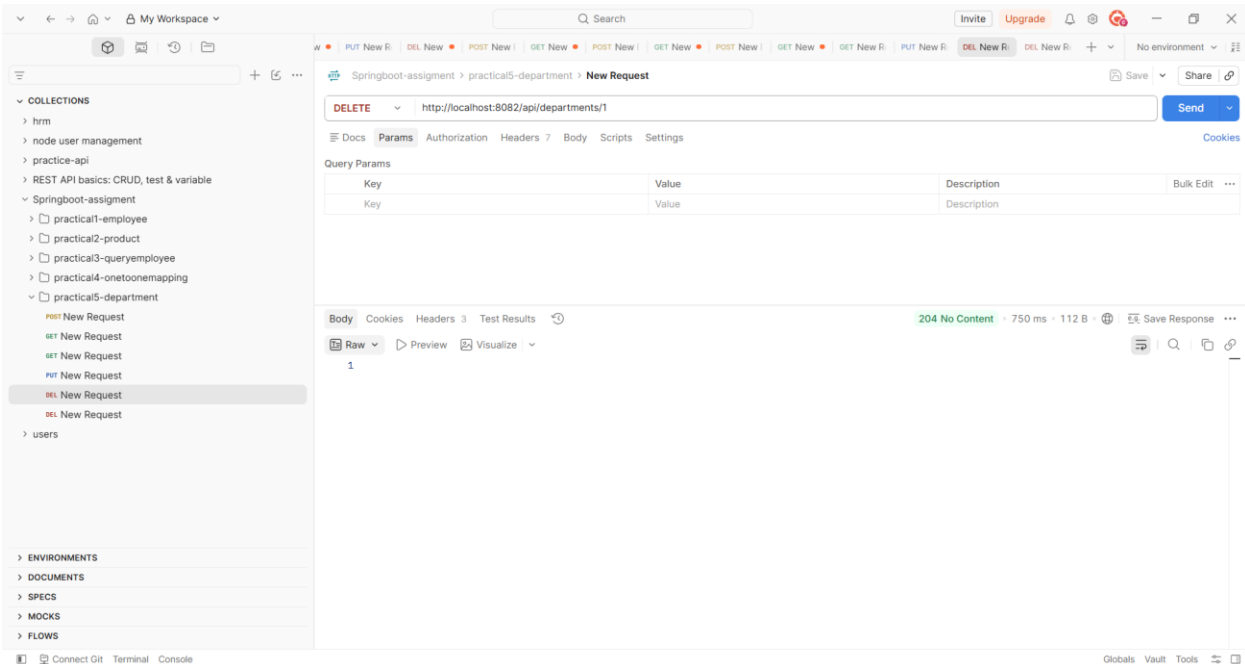
200 OK · 94 ms · 317 B

Delete Method - Employee

Enroll – 25CI2110039 | Name – Kalariya Heli



Delete Method - Department



Method	Endpoint (Postman)	Request Body / Params	Expected
--------	--------------------	-----------------------	----------

			Response
POST	http://localhost:8082/api/departments	<pre>{"deptName":"Computer Science","employees":[{"empName":"Riya","salary":42000},{"empName":"Karan","salary":39000}]}</pre>	201 Created — Department created with employees and dept_id foreign key
GET	http://localhost:8082/api/departments/1	Path Variable: id = 1	200 OK — Displays Department with nested employees list
GET	http://localhost:8082/api/departments/1/employees	Path Variable: id = 1	200 OK — Displays employees

			belonging to Department 1
PUT	http://localhost:8082/api/departments/1	{"deptName":"Computer Engineering"}	200 OK — Department name updated successfully
PUT	http://localhost:8082/api/departments/1/employees/2	{"empName":"Karan Patil","salary":41000}	200 OK — Employee details updated within Department 1
DELETE	http://localhost:8082/api/departments/1/employees/2	Path Variables: deptId = 1, empId = 2	204 No Content — Employee removed

			ved while Depa rtme nt remai ns
DEL ETE	http://localhost:8082/api/departments/1	Path Variable: id = 1	204 No Conte nt — Depa rtme nt and all remai ning empl oyees remo ved throu gh casca de

Definition / Objective

Textbook Reference: Chapter 5 — Working with Spring Data JPA

Objective:

To implement a Many-to-Many relationship between Student and Course using `@ManyToMany` and a join table

via `@JoinTable`, and to avoid infinite JSON recursion using `@JsonManagedReference` / `@JsonBackReference`.

Class Name(s) with Attributes:

Student (`@Entity`)

- `studentId` : `int` (`@Id @GeneratedValue`)
- `studentName` : `String`
- `courses` : `Set<Course>` (`@ManyToMany, @JoinTable(name="student_course")`)

Course (`@Entity`)

- `courseId` : `int` (`@Id @GeneratedValue`)
- `courseName` : `String`
- `students` : `Set<Student>` (`@ManyToMany(mappedBy="courses")`)

Step-by-Step Task Instructions:

1. Create the Course entity with the inverse mapping students, using `mappedBy = "courses"`.
2. Create the Student entity as the owning side with `@ManyToMany` and `@JoinTable(name = "student_course",
joinColumns = @JoinColumn(name="student_id"), inverseJoinColumns =
@JoinColumn(name="course_id"))`.
3. Annotate the owning collection with `@JsonManagedReference` and the inverse collection with `@JsonBackReference` to prevent infinite serialization loops.
4. Create `StudentRepository` and `CourseRepository` extending `JpaRepository`.
5. Create `StudentRestController` with POST `/api/students` accepting a student along with a set of existing course
ids/objects, and a separate endpoint POST `/api/students/{id}/enroll/{courseId}` to enrol an existing
student
into an additional course.
6. Add GET `/api/students/{id}` to view a student with their enrolled courses, PUT `/api/courses/{id}` to
update a
course's details, and PUT `/api/students/{id}` to replace a student's entire course set.
7. Add DELETE `/api/students/{studentId}/unenroll/{courseId}` to remove a single student-course link
from the
join table (without deleting either entity), and DELETE `/api/courses/{id}` to delete a course entirely
(removing its join-table rows first).

8. Test creation with multiple courses and the enrol-into-additional-course endpoint, then confirm from the

Course side which students are enrolled, and finally test the update and unenroll/delete endpoints.

Code

StudentCourseController.java

```
package com.example.studentcourseapi.controller;

import java.util.HashSet;
import java.util.Set;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.example.studentcourseapi.entity.Course;
import com.example.studentcourseapi.entity.Student;
import com.example.studentcourseapi.repository.CourseRepository;
import com.example.studentcourseapi.repository.StudentRepository;

@RestController
@RequestMapping("/api")
public class StudentCourseController {

    private final StudentRepository studentRepository;
    private final CourseRepository courseRepository;

    public StudentCourseController(StudentRepository studentRepository, CourseRepository courseRepository)
    {

        this.studentRepository = studentRepository;
        this.courseRepository = courseRepository;
    }

    @PostMapping("/courses")
    public ResponseEntity<Course> createCourse(@RequestBody Course course) {

        return ResponseEntity.status(HttpStatus.CREATED).body(courseRepository.save(course));
    }

    @PostMapping("/students")
    public ResponseEntity<Student> createStudent(@RequestBody Student student) {

        Set<Course> validCourses = new HashSet<>();

        for (Course course : student.getCourses()) {

            Course existingCourse = courseRepository.findById(course.getCourseId()).orElse(null);

            if (existingCourse != null) {
                validCourses.add(existingCourse);
            }
        }

        student.setCourses(validCourses);

        return ResponseEntity.status(HttpStatus.CREATED).body(studentRepository.save(student));
    }
}
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
@PostMapping("/students/{id}/enroll/{courseId}")
public ResponseEntity<Student> enrollStudent(@PathVariable int id, @PathVariable int courseId) {

    Student student = studentRepository.findById(id).orElse(null);

    Course course = courseRepository.findById(courseId).orElse(null);

    if (student == null || course == null) {
        return ResponseEntity.notFound().build();
    }

    student.getCourses().add(course);

    return ResponseEntity.ok(studentRepository.save(student));
}

@GetMapping("/students/{id}")
public ResponseEntity<Student> getStudent(@PathVariable int id) {

    Student student = studentRepository.findById(id).orElse(null);

    if (student == null) {
        return ResponseEntity.notFound().build();
    }

    return ResponseEntity.ok(student);
}

@GetMapping("/courses/{id}/students")
public ResponseEntity<Set<Student>> getStudentsByCourse(@PathVariable int id) {

    Course course = courseRepository.findById(id).orElse(null);

    if (course == null) {
        return ResponseEntity.notFound().build();
    }

    return ResponseEntity.ok(course.getStudents());
}

@PutMapping("/courses/{id}")
public ResponseEntity<Course> updateCourse(@PathVariable int id, @RequestBody Course details) {

    Course course = courseRepository.findById(id).orElse(null);

    if (course == null) {
        return ResponseEntity.notFound().build();
    }

    if (details.getCourseName() != null) {
        course.setCourseName(details.getCourseName());
    }

    return ResponseEntity.ok(courseRepository.save(course));
}

@PutMapping("/students/{id}")
public ResponseEntity<Student> updateStudentCourses(@PathVariable int id, @RequestBody Student
details) {

    Student student = studentRepository.findById(id).orElse(null);

    if (student == null) {
        return ResponseEntity.notFound().build();
    }

    if (details.getStudentName() != null) {
        student.setStudentName(details.getStudentName());
    }
}
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
    }

    Set<Course> newCourses = new HashSet<>();

    for (Course course : details.getCourses()) {

        Course existingCourse = courseRepository.findById(course.getCourseId()).orElse(null);

        if (existingCourse != null) {
            newCourses.add(existingCourse);
        }

    }

    student.setCourses(newCourses);

    return ResponseEntity.ok(studentRepository.save(student));
}

@DeleteMapping("/students/{studentId}/unenroll/{courseId}")
public ResponseEntity<Void> unenrollStudent(@PathVariable int studentId, @PathVariable int courseId) {

    Student student = studentRepository.findById(studentId).orElse(null);

    Course course = courseRepository.findById(courseId).orElse(null);

    if (student == null || course == null) {
        return ResponseEntity.notFound().build();
    }

    student.getCourses().remove(course);

    studentRepository.save(student);

    return ResponseEntity.noContent().build();
}

@DeleteMapping("/courses/{id}")
public ResponseEntity<Void> deleteCourse(@PathVariable int id) {

    Course course = courseRepository.findById(id).orElse(null);

    if (course == null) {
        return ResponseEntity.notFound().build();
    }

    for (Student student : course.getStudents()) {
        student.getCourses().remove(course);
        studentRepository.save(student);
    }

    courseRepository.delete(course);

    return ResponseEntity.noContent().build();
}
}
```

Course.java

```
package com.example.studentcourseapi.entity;

import java.util.HashSet;
import java.util.Set;

import com.fasterxml.jackson.annotation.JsonIgnore;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
import jakarta.persistence.ManyToMany;

@Entity
public class Course {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int courseId;

    private String courseName;

    @ManyToMany(mappedBy = "courses")
    @JsonIgnore
    private Set<Student> students = new HashSet<>();

    public Course() {
    }

    public int getCourseId() {
        return courseId;
    }

    public void setCourseId(int courseId) {
        this.courseId = courseId;
    }

    public String getCourseName() {
        return courseName;
    }

    public void setCourseName(String courseName) {
        this.courseName = courseName;
    }

    public Set<Student> getStudents() {
        return students;
    }

    public void setStudents(Set<Student> students) {
        this.students = students;
    }
}
```

Student.java

```
package com.example.studentcourseapi.entity;

import java.util.HashSet;
import java.util.Set;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.JoinTable;
import jakarta.persistence.ManyToMany;

@Entity
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int studentId;

    private String studentName;

    @ManyToMany
    @JoinTable(
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
        name = "student_course",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id")
    )
    private Set<Course> courses = new HashSet<>();

    public Student() {
    }

    public int getStudentId() {
        return studentId;
    }

    public void setStudentId(int studentId) {
        this.studentId = studentId;
    }

    public String getStudentName() {
        return studentName;
    }

    public void setStudentName(String studentName) {
        this.studentName = studentName;
    }

    public Set<Course> getCourses() {
        return courses;
    }

    public void setCourses(Set<Course> courses) {
        this.courses = courses;
    }
}
```

P6Application.java

```
package com.example.studentcourseapi;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class P6Application {

    public static void main(String[] args) {
        SpringApplication.run(P6Application.class, args);
    }

}
```

CourseRepository.java

```
package com.example.studentcourseapi.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.example.studentcourseapi.entity.Course;

public interface CourseRepository
    extends JpaRepository<Course, Integer> {

}
```

StudentRepository.java

```
package com.example.studentcourseapi.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.example.studentcourseapi.entity.Student;
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
public interface StudentRepository
    extends JpaRepository<Student, Integer> {

}
```

P6ApplicationTests.java

```
package com.example.studentcourseapi;

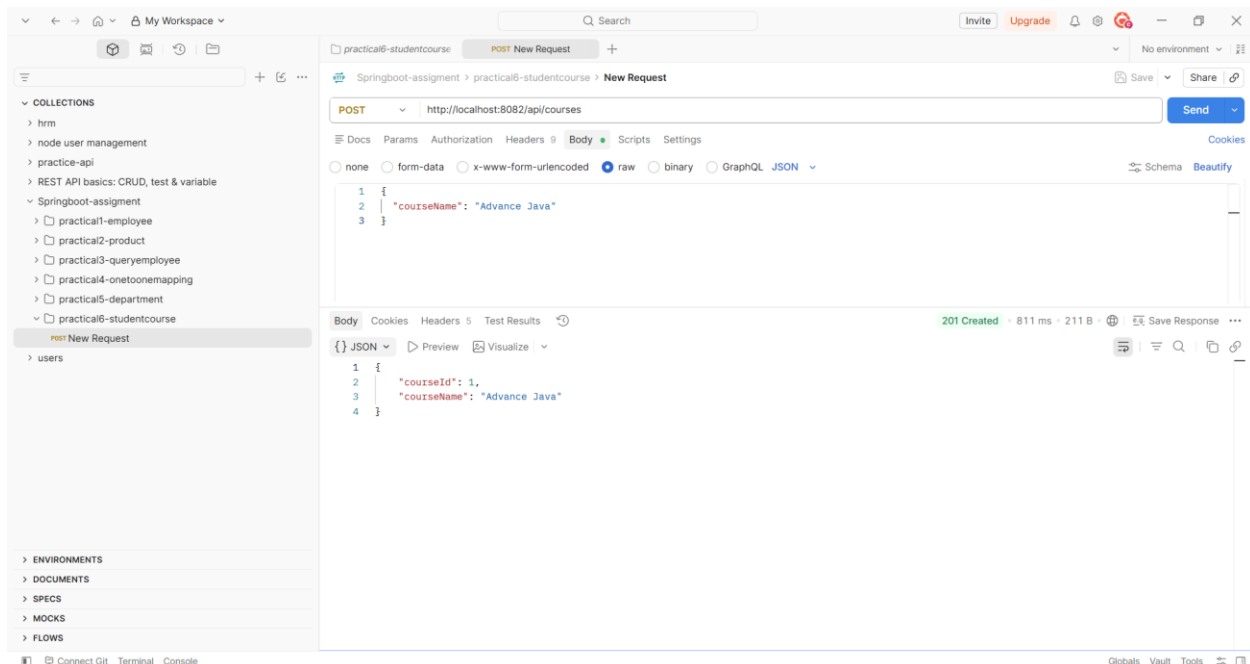
import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class P6ApplicationTests {

    @Test
    void contextLoads() {
    }

}
```

Post Method



Post Method - Students

Enroll – 25CI2110039 | Name – Kalariya Heli

The screenshot shows the Postman interface with a workspace named "My Workspace". On the left, the "COLLECTIONS" list includes "practical6-studentcourse". The main panel displays a "New Request" for a POST method to the URL "http://localhost:8082/api/students". The request body is a JSON object:

```
1 {
2   "studentName": "Priya",
3   "courses": [
4     {
5       "courseId": 1
6     }
7   ]
8 }
```

The response is shown in the "Body" tab, indicating a "201 Created" status with a response time of 141 ms and a size of 261 B. The response body is a JSON object:

```
1 {
2   "courses": [
3     {
4       "courseId": 1,
5       "courseName": "Advance Java"
6     }
7   ],
8   "studentId": 1,
9   "studentName": "Priya"
10 }
```

Post Method – to enroll student in both course

The screenshot shows the Postman interface with a workspace named "My Workspace". On the left, the "COLLECTIONS" list includes "practical6-studentcourse". The main panel displays a "New Request" for a POST method to the URL "http://localhost:8082/api/students/1/enroll/2". The request body is a JSON object:

```
1 {
2   "courses": [
3     {
4       "courseId": 1,
5       "courseName": "Advance Java"
6     },
7     {
8       "courseId": 2,
9       "courseName": "Spring Boot"
10    }
11  ],
12  "studentId": 1,
13  "studentName": "Priya"
14 }
```

The response is shown in the "Body" tab, indicating a "200 OK" status with a response time of 100 ms and a size of 298 B. The response body is a JSON object:

```
1 {
2   "courses": [
3     {
4       "courseId": 1,
5       "courseName": "Advance Java"
6     },
7     {
8       "courseId": 2,
9       "courseName": "Spring Boot"
10    }
11  ],
12  "studentId": 1,
13  "studentName": "Priya"
14 }
```

Get Method

Enroll – 25CI2110039 | Name – Kalariya Heli

The screenshot shows the Postman interface with a GET request to `http://localhost:8082/api/students/1`. The response is a JSON object with the following structure:

```
1 {
2   "courses": [
3     {
4       "courseId": 2,
5       "courseName": "Spring Boot"
6     },
7     {
8       "courseId": 1,
9       "courseName": "Advance Java"
10    }
11  ],
12  "studentId": 1,
13  "studentName": "Priya"
14 }
```

Put Method

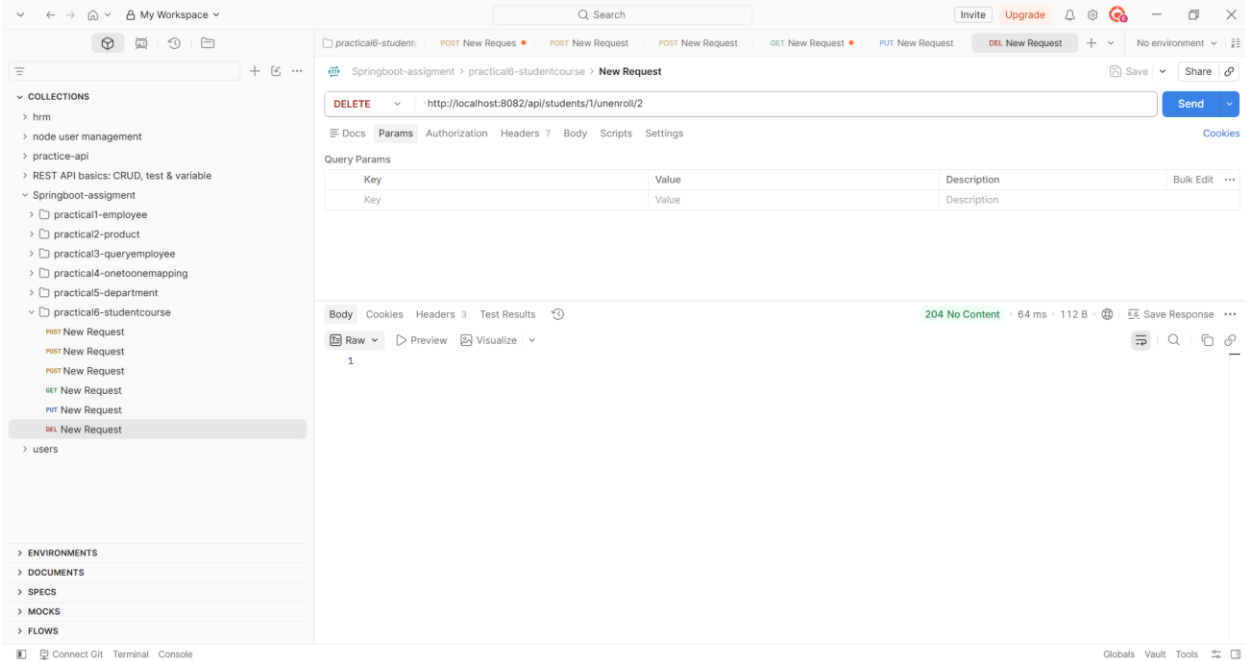
The screenshot shows the Postman interface with a PUT request to `http://localhost:8082/api/courses/1`. The request body is a JSON object with the following structure:

```
1 {
2   "courseName": "Advance Java (Spring Boot)"
3 }
```

The response is a JSON object with the following structure:

```
1 {
2   "courseId": 1,
3   "courseName": "Advance Java (Spring Boot)"
4 }
```

Delete Method



Meth od	Endpoint (Postman)	Request Body / Params	Expec ted Respon se
POST	http://localhost:8082/api/courses	{"courseName":"Advance Java"}	201 Created — Course created successf ully
POST	http://localhost:8082/api/students	{"studentName":"Priya","courses":[{"courseId":1}]}	201 Created — Student linked to Course through

			join table
POST	http://localhost:8082/api/students/1/enroll/2	Path Variables: studentId = 1, courseId = 2	200 OK — Student enrolled into an additional Course
GET	http://localhost:8082/api/students/1	Path Variable: id = 1	200 OK — Displays Student with enrolled Courses
GET	http://localhost:8082/api/courses/1/students	Path Variable: id = 1	200 OK — Displays all Students enrolled in Course 1
PUT	http://localhost:8082/api/courses/1	{"courseName": "Advance Java (Spring Boot)"}	200 OK — Course updated successfully
PUT	http://localhost:8082/api/students/1	{"studentName": "Priya Sharma", "courses": [{"courseId": 1}]}	200 OK — Student details

			and entire Course set updated
DELETE	http://localhost:8082/api/students/1/unenroll/2	Path Variables: studentId = 1, courseId = 2	204 No Content — Student-Course relationship removed; Student and Course remain
DELETE	http://localhost:8082/api/courses/1	Path Variable: id = 1	204 No Content — Course deleted and related join-table entries removed

Practical No. 7: Transaction Management with @Transactional – Bank Fund Transfer

Definition / Objective

Transfer

Textbook Reference: Chapter 5 — Working with Spring Data JPA

Objective:

To understand declarative transaction management in Spring Data JPA, and to ensure atomicity across multiple repository operations (debit + credit) using `@Transactional`, including automatic rollback when a runtime exception occurs.

Class Name(s) with Attributes:

Account (`@Entity`)

- `accountId` : int (`@Id @GeneratedValue`)

- `accountHolder` : String

- `balance` : double

InsufficientBalanceException (custom RuntimeException)

AccountService / AccountServiceImpl (`transferFunds` is `@Transactional`)

- `accountRepository` : AccountRepository

Step-by-Step Task Instructions:

1. Create the Account entity and AccountRepository extending `JpaRepository<Account, Integer>`.

2. Create a custom unchecked exception InsufficientBalanceException.

3. In AccountServiceImpl, write method `transferFunds(fromId, toId, amount)` annotated with `@Transactional`

that: fetches the source account, checks balance, debits it, fetches the destination account, credits it, and saves

both.

4. Deliberately throw InsufficientBalanceException after the debit step (or simulate by transferring more than the

available balance) to prove that `@Transactional` rolls back the debit too.

5. Expose POST `/api/accounts/transfer` in a REST controller accepting `fromId`, `toId`, and `amount`.

6. Add GET `/api/accounts/{id}` to view a single account's balance, PUT `/api/accounts/{id}` to update non-transactional account details such as `accountHolder` name, and DELETE `/api/accounts/{id}` to close an account (only permitted when its balance is zero — throw a custom exception otherwise).

7. Test a successful transfer, then test a failing transfer, and re-check both balances via GET to confirm rollback

left them unchanged.

Code

AccountRestController.java

```
package com.example.bankapi.controller;
```

```
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.example.bankapi.entity.Account;
import com.example.bankapi.entity.TransferRequest;
import com.example.bankapi.exception.InsufficientBalanceException;
import com.example.bankapi.service.AccountService;

@RestController
@RequestMapping("/api/accounts")
public class AccountRestController {

    private final AccountService accountService;

    public AccountRestController(
        AccountService accountService) {

        this.accountService = accountService;
    }

    @PostMapping
    public ResponseEntity<Account> createAccount(
        @RequestBody Account account) {

        return ResponseEntity
            .status(HttpStatus.CREATED)
            .body(accountService.createAccount(account));
    }

    @PostMapping("/transfer")
    public ResponseEntity<String> transferFunds(
        @RequestBody TransferRequest request) {

        try {

            accountService.transferFunds(
                request.getFromId(),
                request.getToId(),
                request.getAmount());

            return ResponseEntity.ok(
                "Transfer successful");
        } catch (InsufficientBalanceException e) {

            return ResponseEntity
                .badRequest()
                .body(e.getMessage());
        } catch (RuntimeException e) {

            return ResponseEntity
                .badRequest()
                .body(e.getMessage());
        }
    }

    @GetMapping("/{id}")
    public ResponseEntity<Account> getAccount(
        @PathVariable int id) {
```

```
        Account account =
            accountService.getAccount(id);

        if (account == null) {
            return ResponseEntity.notFound().build();
        }

        return ResponseEntity.ok(account);
    }

    @PutMapping("/{id}")
    public ResponseEntity<Account> updateAccount(
        @PathVariable int id,
        @RequestBody Account details) {

        Account updated =
            accountService.updateAccount(id, details);

        if (updated == null) {
            return ResponseEntity.notFound().build();
        }

        return ResponseEntity.ok(updated);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<?> deleteAccount(
        @PathVariable int id) {

        try {

            boolean deleted =
                accountService.deleteAccount(id);

            if (!deleted) {
                return ResponseEntity.notFound().build();
            }

            return ResponseEntity.noContent().build();

        } catch (RuntimeException e) {

            return ResponseEntity
                .badRequest()
                .body(e.getMessage());

        }
    }
}
```

Account.java

```
package com.example.bankapi.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Account {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int accountId;

    private String accountHolder;

    private double balance;
```

```
public Account() {
    super();
    // TODO Auto-generated constructor stub
}

public int getAccountId() {
    return accountId;
}

public void setAccountId(int accountId) {
    this.accountId = accountId;
}

public String getAccountHolder() {
    return accountHolder;
}

public void setAccountHolder(String accountHolder) {
    this.accountHolder = accountHolder;
}

public double getBalance() {
    return balance;
}

public void setBalance(double balance) {
    this.balance = balance;
}

}
```

TransferRequest.java

```
package com.example.bankapi.entity;

public class TransferRequest {

    private int fromId;
    private int toId;
    private double amount;

    public int getFromId() {
        return fromId;
    }

    public void setFromId(int fromId) {
        this.fromId = fromId;
    }

    public int getToId() {
        return toId;
    }

    public void setToId(int toId) {
        this.toId = toId;
    }

    public double getAmount() {
        return amount;
    }

    public void setAmount(double amount) {
        this.amount = amount;
    }

}
```

InsufficientBalanceException.java

Enroll – 25CI2110039 | Name – Kalariya Heli

```
package com.example.bankapi.exception;

public class InsufficientBalanceException
    extends RuntimeException {

    public InsufficientBalanceException(String message) {
        super(message);
    }
}
```

P7Application.java

```
package com.example.bankapi;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class P7Application {

    public static void main(String[] args) {
        SpringApplication.run(P7Application.class, args);
    }
}
```

AccountRepository.java

```
package com.example.bankapi.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.example.bankapi.entity.Account;

public interface AccountRepository
    extends JpaRepository<Account, Integer> {

}
```

AccountService.java

```
package com.example.bankapi.service;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import com.example.bankapi.entity.Account;
import com.example.bankapi.exception.InsufficientBalanceException;
import com.example.bankapi.repository.AccountRepository;

@Service
public class AccountService {

    private final AccountRepository accountRepository;

    public AccountService(
        AccountRepository accountRepository) {

        this.accountRepository = accountRepository;
    }

    public Account createAccount(Account account) {

        return accountRepository.save(account);
    }

    public Account getAccount(int id) {

        return accountRepository.findById(id)
            .orElse(null);
    }
}
```

```
}

@Transactional
public void transferFunds(
    int fromId,
    int toId,
    double amount) {

    Account fromAccount =
        accountRepository.findById(fromId)
            .orElseThrow(() ->
                new RuntimeException(
                    "Source account not found"));

    Account toAccount =
        accountRepository.findById(toId)
            .orElseThrow(() ->
                new RuntimeException(
                    "Destination account not found"));

    if (fromAccount.getBalance() < amount) {

        throw new InsufficientBalanceException(
            "Insufficient balance");
    }

    fromAccount.setBalance(
        fromAccount.getBalance() - amount);

    toAccount.setBalance(
        toAccount.getBalance() + amount);

    accountRepository.save(fromAccount);
    accountRepository.save(toAccount);
}

public Account updateAccount(
    int id,
    Account details) {

    Account account =
        accountRepository.findById(id)
            .orElse(null);

    if (account == null) {
        return null;
    }

    if (details.getAccountHolder() != null) {
        account.setAccountHolder(
            details.getAccountHolder());
    }

    return accountRepository.save(account);
}

public boolean deleteAccount(int id) {

    Account account =
        accountRepository.findById(id)
            .orElse(null);

    if (account == null) {
        return false;
    }

    if (account.getBalance() != 0) {
        throw new RuntimeException(
            "Account balance must be zero before deletion");
    }
}
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
    }

    accountRepository.deleteById(id);

    return true;
}
}
```

P7ApplicationTests.java

```
package com.example.bankapi;

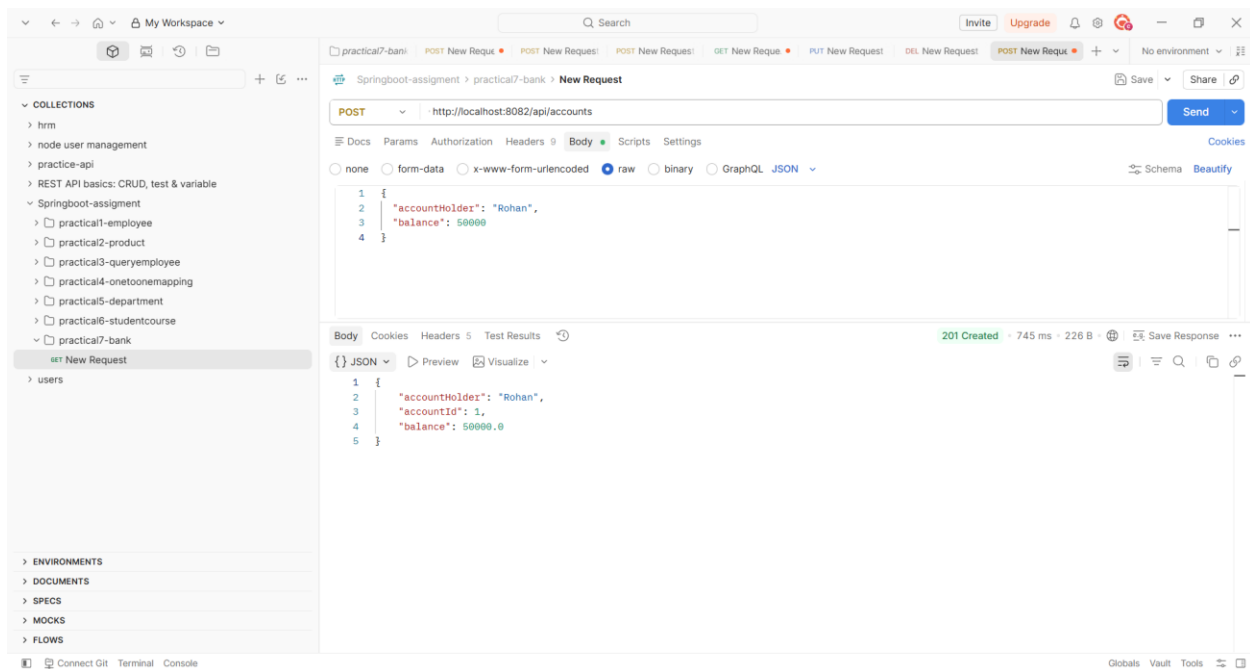
import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class P7ApplicationTests {

    @Test
    void contextLoads() {
    }

}
```

Post Method



Post Method - transfer money

The screenshot shows the Postman interface with a workspace named "My Workspace". On the left, a sidebar lists collections, including "Springboot-assignment" and "practical7-bank". The main panel displays a "New Request" for a POST method to the URL "http://localhost:8082/api/accounts/transfer". The request body is set to "raw" and contains a JSON object:

```
{  "fromId": 1,  "toId": 2,  "amount": 5000}
```

. The response is a "200 OK" status with a response time of 126 ms and a body of "Transfer successful".

Get Method

The screenshot shows the Postman interface with a workspace named "My Workspace". On the left, a sidebar lists collections, including "Springboot-assignment" and "practical7-bank". The main panel displays a "New Request" for a GET method to the URL "http://localhost:8082/api/accounts/1". The response is a "200 OK" status with a response time of 40 ms and a body of

```
{  "accountHolder": "Rohan",  "accountId": 1,  "balance": 45000.0}
```

.

Post Method – Insufficient balance

Enroll – 25CI2110039 | Name – Kalariya Heli

The screenshot shows the Postman interface with a workspace named 'My Workspace'. On the left, a collection named 'Springboot-assignment' is expanded, showing a sub-collection 'practical7-bank'. The 'practical7-bank' collection contains several requests, including 'POST New Request', 'POST New Request', 'GET New Request', and 'DELETE New Request'. The 'POST New Request' is selected, and its details are shown in the main pane. The request is a POST to 'http://localhost:8082/api/accounts/transfer' with a JSON body:

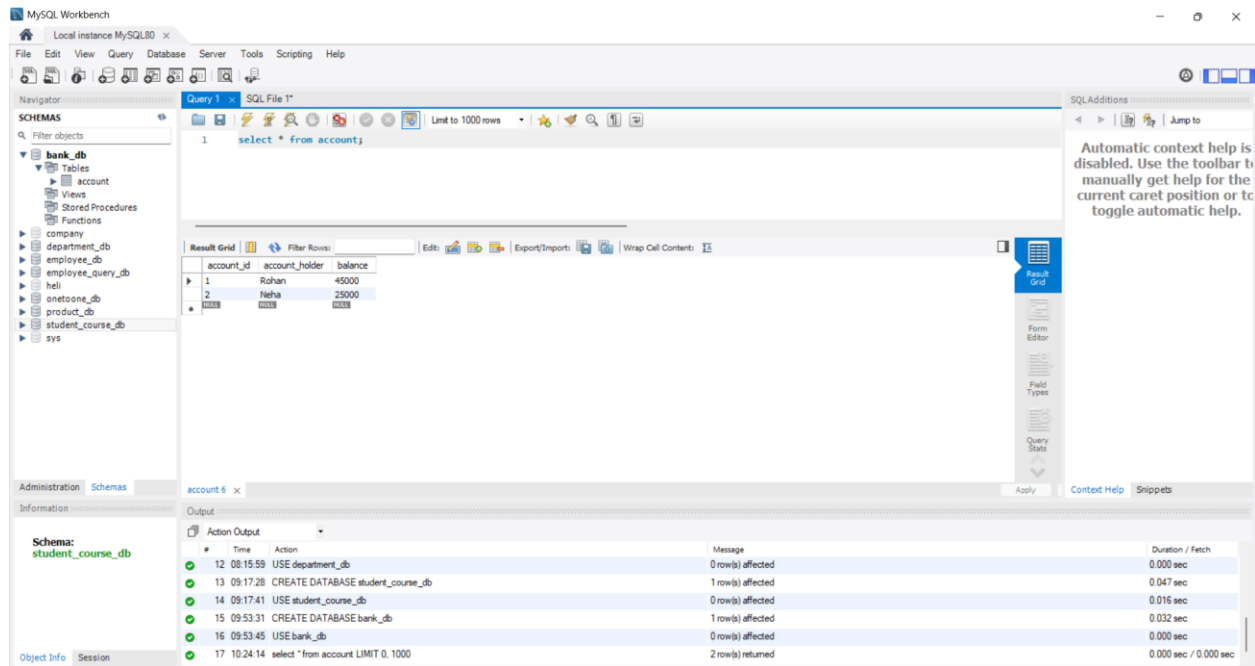
```
{  "fromId": 1,  "toId": 2,  "amount": 999999}
```

. The response is a '400 Bad Request' with a status of 42 ms and 164 B. The response body is 'Insufficient balance'.

Delete Method - balance must be zero

The screenshot shows the Postman interface with a workspace named 'My Workspace'. On the left, a collection named 'Springboot-assignment' is expanded, showing a sub-collection 'practical7-bank'. The 'practical7-bank' collection contains several requests, including 'POST New Request', 'POST New Request', 'GET New Request', and 'DELETE New Request'. The 'DELETE New Request' is selected, and its details are shown in the main pane. The request is a DELETE to 'http://localhost:8082/api/accounts/2'. The response is a '400 Bad Request' with a status of 33 ms and 188 B. The response body is 'Account balance must be zero before deletion'.

Database of account table



Meth od	Endpoint (Postman)	Request Body / Params	Expece d Respons e
POST	http://localhost:8082/api/accounts	{"accountHolder":"Rohan","balance":50000}	201 Created — Source account created
POST	http://localhost:8082/api/accounts	{"accountHolder":"Neha","balance":20000}	201 Created — Destinati on account created

POST	http://localhost:8082/api/accounts/transfer	{"fromId":1,"toId":2,"amount":5000}	200 OK — Transfer successful and both balances updated
POST	http://localhost:8082/api/accounts/transfer	{"fromId":1,"toId":2,"amount":999999}	400 Bad Request — Insufficient balance; transaction rolled back and balances unchanged
GET	http://localhost:8082/api/accounts/1	Path Variable: id = 1	200 OK — Displays current balance of Account 1
GET	http://localhost:8082/api/accounts/2	Path Variable: id = 2	200 OK — Displays current balance of Account 2

PUT	http://localhost:8082/api/accounts/1	{"accountHolder":"Rohan Mehta"}	200 OK — Account holder updated without changing balance
DELETE	http://localhost:8082/api/accounts/2	Path Variable: id = 2	400 Bad Request if balance is not zero
DELETE	http://localhost:8082/api/accounts/3	Path Variable: id = 3	204 No Content if balance is zero

Practical No. 8: Pagination and Sorting with Spring Data JPA

Definition / Objective

Textbook Reference: Chapter 5 — Working with Spring Data JPA

Objective:

To implement server-side pagination and sorting for a large result set using Spring Data's Pageable/PageRequest

abstraction (PagingAndSortingRepository capability), exposed through query parameters on a REST endpoint.

Class Name(s) with Attributes:

Product (reuse from Practical 2)

- productId, productName, category, price, quantity

ProductRestController (extended)

- Method: getProducts(page,size,sortBy) : Page<Product>

Step-by-Step Task Instructions:

1. Insert at least 15–20 sample Product records for meaningful pagination.
2. Ensure ProductRepository extends JpaRepository (which supports findAll(Pageable)).
3. In ProductRestController, add GET /api/products/page accepting @RequestParam page, size, and sortBy (with defaults).
4. Inside the method, build Pageable pageable = PageRequest.of(page, size, Sort.by(sortBy)) and call productRepository.findAll(pageable).
5. Return the resulting Page<Product> directly (Spring serializes content, totalPages, totalElements, etc. automatically).
6. Add POST /api/products (create one more record and observe it appear on the correct page), PUT /api/products/{id} (update a record's price/category and re-sort), and DELETE /api/products/{id} (remove a record and confirm totalElements decreases).
7. Test with different page/size/sortBy combinations in Postman and verify totalPages and totalElements are correct after each create/update/delete.

Code

ProductRestController.java

```
package com.example.paginationapi.controller;

import java.util.List;

import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Sort;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import com.example.paginationapi.entity.Product;
import com.example.paginationapi.repository.ProductRepository;

@RestController
@RequestMapping("/api/products")
public class ProductRestController {

    private final ProductRepository productRepository;

    public ProductRestController(
        ProductRepository productRepository) {

        this.productRepository = productRepository;
    }
}
```

```
@PostMapping
public ResponseEntity<Product> createProduct(
    @RequestBody Product product) {

    Product saved =
        productRepository.save(product);

    return ResponseEntity
        .status(HttpStatus.CREATED)
        .body(saved);
}

@GetMapping
public ResponseEntity<List<Product>> getAllProducts() {

    return ResponseEntity.ok(
        productRepository.findAll());
}

@GetMapping("/page")
public Page<Product> getProducts(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "5") int size,
    @RequestParam(defaultValue = "productId")
    String sortBy) {

    Pageable pageable =
        PageRequest.of(
            page,
            size,
            Sort.by(sortBy));

    return productRepository.findAll(pageable);
}

@GetMapping("/{id}")
public ResponseEntity<Product> getProductById(
    @PathVariable int id) {

    Product product =
        productRepository.findById(id)
            .orElse(null);

    if (product == null) {
        return ResponseEntity.notFound().build();
    }

    return ResponseEntity.ok(product);
}

@PutMapping("/{id}")
public ResponseEntity<Product> updateProduct(
    @PathVariable int id,
    @RequestBody Product details) {

    Product product =
        productRepository.findById(id)
            .orElse(null);

    if (product == null) {
        return ResponseEntity.notFound().build();
    }

    if (details.getProductName() != null) {
        product.setProductName(
            details.getProductName());
    }
}
```

```
        if (details.getCategory() != null) {
            product.setCategory(
                details.getCategory());
        }

        if (details.getPrice() > 0) {
            product.setPrice(
                details.getPrice());
        }

        if (details.getQuantity() > 0) {
            product.setQuantity(
                details.getQuantity());
        }

        return ResponseEntity.ok(
            productRepository.save(product));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteProduct(
        @PathVariable int id) {

        if (!productRepository.existsById(id)) {
            return ResponseEntity.notFound().build();
        }

        productRepository.deleteById(id);

        return ResponseEntity.noContent().build();
    }
}
```

Product.java

```
package com.example.paginationapi.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int productId;

    private String productName;
    private String category;
    private double price;
    private int quantity;
    public Product() {
        super();
        // TODO Auto-generated constructor stub
    }
    public int getProductId() {
        return productId;
    }
    public void setProductId(int productId) {
        this.productId = productId;
    }
    public String getProductName() {
        return productName;
    }
    public void setProductName(String productName) {
        this.productName = productName;
    }
}
```

```
        public String getCategory() {
            return category;
        }
        public void setCategory(String category) {
            this.category = category;
        }
        public double getPrice() {
            return price;
        }
        public void setPrice(double price) {
            this.price = price;
        }
        public int getQuantity() {
            return quantity;
        }
        public void setQuantity(int quantity) {
            this.quantity = quantity;
        }
    }
}
```

P8Application.java

```
package com.example.paginationapi;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class P8Application {

    public static void main(String[] args) {
        SpringApplication.run(P8Application.class, args);
    }

}
```

ProductRepository.java

```
package com.example.paginationapi.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.example.paginationapi.entity.Product;

public interface ProductRepository
    extends JpaRepository<Product, Integer> {

}

}
```

P8ApplicationTests.java

```
package com.example.paginationapi;

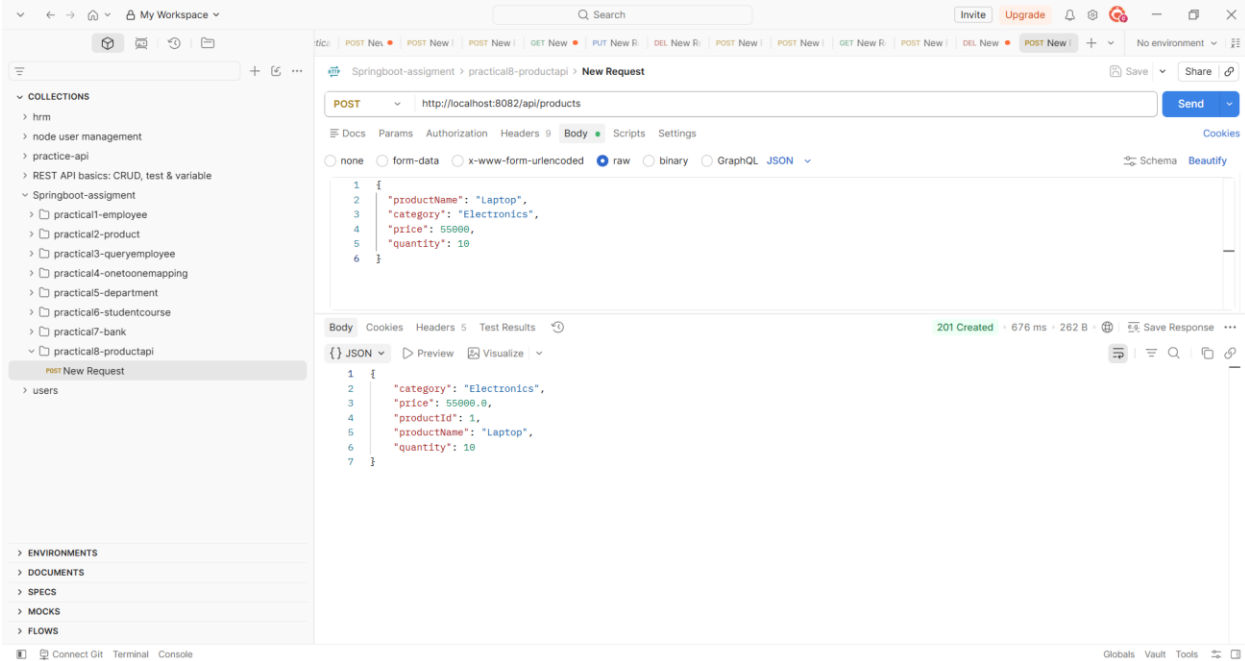
import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class P8ApplicationTests {

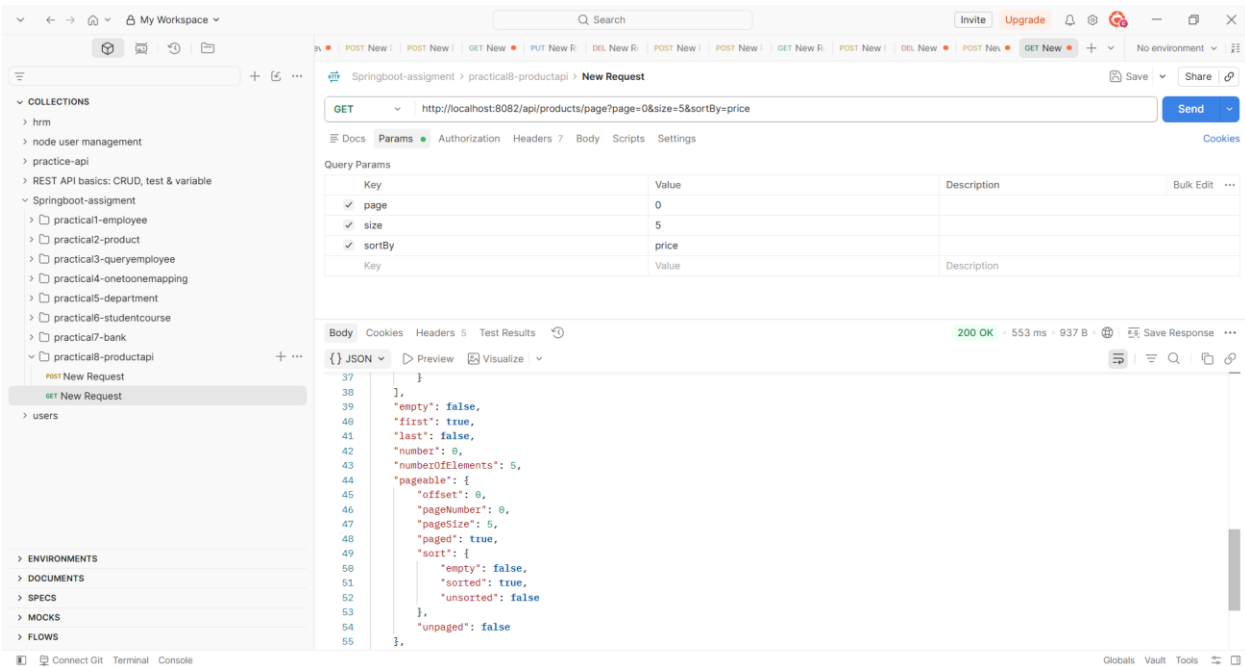
    @Test
    void contextLoads() {
    }

}
```

Post Method



Get Method - with pagination



Get Method – pagination and sortBy productName

Enroll – 25CI2110039 | Name – Kalariya Heli

The screenshot shows the Postman interface with a GET request configured. The URL is `http://localhost:8082/api/products?page=1&size=5&sortBy=productName`. The query parameters are set as follows:

Key	Value	Description
page	1	
size	5	
sortBy	productName	

The response is a JSON object with the following structure:

```
32 {
33   "category": "Furniture",
34   "price": 6500.0,
35   "productId": 6,
36   "productName": "Office Chair",
37   "quantity": 12
38 }
39
40 "empty": false,
41 "first": false,
42 "last": false,
43 "number": 1,
44 "numberOfElements": 5,
45 "pageable": {
46   "offset": 5,
47   "pageNumber": 1,
48   "pageSize": 5,
49   "paged": true,
50   "sort": {
```

Put Method

The screenshot shows the Postman interface with a PUT request configured. The URL is `http://localhost:8082/api/products/4`. The request body is a JSON object with the following structure:

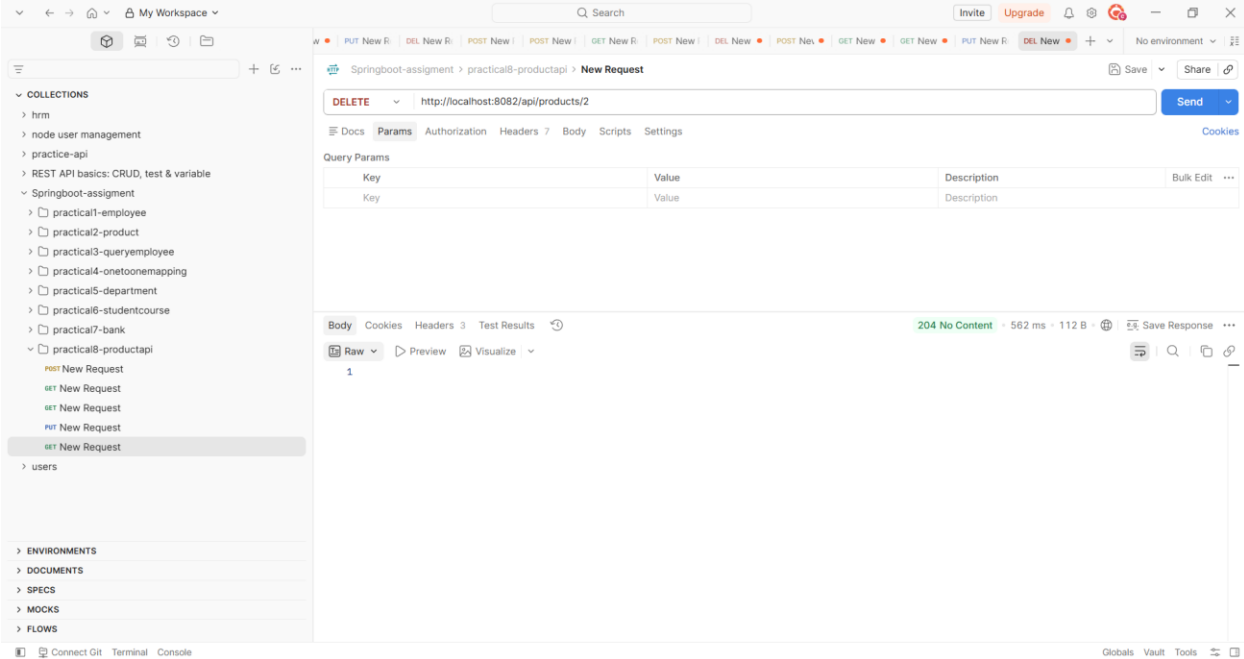
```
1 {
2   "category": "Electronics",
3   "price": 800.0,
4   "productId": 2,
5   "productName": "Mouse",
6   "quantity": 20
7 }
```

The response is a JSON object with the following structure:

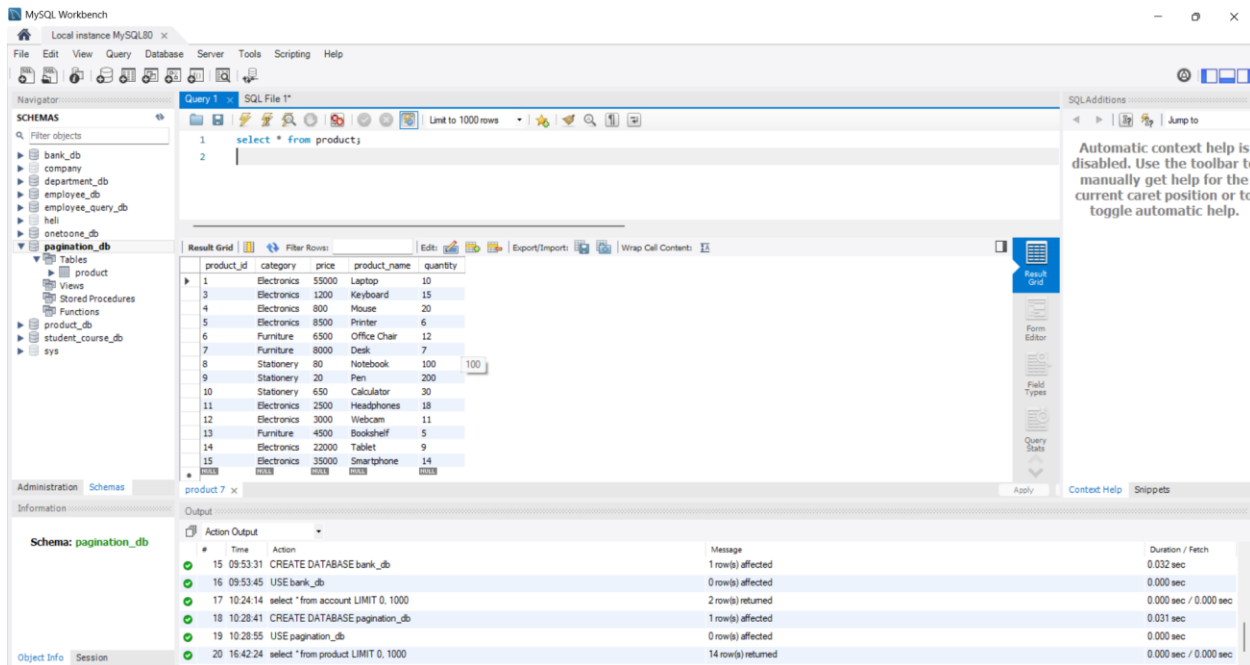
```
1 {
2   "category": "Electronics",
3   "price": 800.0,
4   "productId": 4,
5   "productName": "Mouse",
6   "quantity": 20
7 }
```

Delete Method

Enroll – 25CI2110039 | Name – Kalariya Heli



Database of product table



Method	Endpoint (Postman)	Request Body / Params	Expected Response
GET	http://localhost:8082/api/products/page?page=0&size=5&sortBy=price	Request Params: page = 0, size = 5, sortBy = price	200 OK — JSON contains content[], totalPages, totalElements, number and size
GET	http://localhost:8082/api/products/page?page=1&size=5&sortBy=productName	Request Params: page = 1, size = 5, sortBy = productName	200 OK — Second page of products sorted alphabetically by productName
GET	http://localhost:8082/api/products/page?page=0&size=3&sortBy=price	Request Params: page = 0, size = 3, sortBy = price	200 OK — First page

			containing 3 products sorted by price
POST	http://localhost:8082/api/products	{"productName":"Monitor","category":"Electronics","price":9500,"quantity":8}	201 Created — Product created and totalElements increases by 1
PUT	http://localhost:8082/api/products/4	{"price":8999}	200 OK — Product price updated and reflected in the re-sorted page
DELETE	http://localhost:8082/api/products/2	Path Variable: id = 2	204 No Content — Product

			ct delete d and totalEl ement s decrea ses by 1
--	--	--	---

Practical No. 9: Building a RESTful Microservice using @RestController and ResponseEntity

Definition / Objective

ResponseEntity

Textbook Reference: Chapter 6 — Building RESTful Microservices

Objective:

To design and build a RESTful microservice endpoint using @RestController, mapping HTTP verbs to CRUD

operations, and using ResponseEntity to control the HTTP status code and headers returned to the client — the

core REST-building technique introduced in this chapter.

Class Name(s) with Attributes:

Book (POJO)

- id : int

- title : String

Enroll – 25CI2110039 | Name – Kalariya Heli

- author : String

- price : double

BookRestController (@RestController @RequestMapping("/api/books"))

- Methods return ResponseEntity<Book> / ResponseEntity<List<Book>>

Step-by-Step Task Instructions:

1. Create the Book POJO with the listed attributes and an in-memory List<Book> store inside the controller/service for this practical.

2. Implement POST /api/books returning

ResponseEntity.status(HttpStatus.CREATED).body(savedBook).

3. Implement GET /api/books returning ResponseEntity.ok(bookList).

4. Implement GET /api/books/{id} returning ResponseEntity.ok(book) when found, or

ResponseEntity.notFound().build() when absent.

5. Implement PUT /api/books/{id} and DELETE /api/books/{id}, each returning an appropriate ResponseEntity

and HTTP status.

6. Test every endpoint in Postman and specifically observe the HTTP status code returned for each case (201,

200, 404, 204).

Code

BookRestController.java

```
package com.example.bookapi.controller;

import java.util.ArrayList;
import java.util.List;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.example.bookapi.model.Book;

@RestController
@RequestMapping("/api/books")
public class BookRestController {

    private final List<Book> books = new ArrayList<>();

    private int nextId = 1;

    @PostMapping
    public ResponseEntity<Book> createBook(
        @RequestBody Book book) {

        book.setId(nextId++);
```

```
        books.add(book);

        return ResponseEntity
            .status(HttpStatus.CREATED)
            .body(book);
    }

    @GetMapping
    public ResponseEntity<List<Book>> getAllBooks() {

        return ResponseEntity.ok(books);
    }

    @GetMapping("/{id}")
    public ResponseEntity<Book> getBookById(
        @PathVariable int id) {

        for (Book book : books) {

            if (book.getId() == id) {
                return ResponseEntity.ok(book);
            }
        }

        return ResponseEntity.notFound().build();
    }

    @PutMapping("/{id}")
    public ResponseEntity<Book> updateBook(
        @PathVariable int id,
        @RequestBody Book details) {

        for (Book book : books) {

            if (book.getId() == id) {

                if (details.getTitle() != null) {
                    book.setTitle(
                        details.getTitle());
                }

                if (details.getAuthor() != null) {
                    book.setAuthor(
                        details.getAuthor());
                }

                if (details.getPrice() > 0) {
                    book.setPrice(
                        details.getPrice());
                }

                return ResponseEntity.ok(book);
            }
        }

        return ResponseEntity.notFound().build();
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteBook(
        @PathVariable int id) {

        Book bookToDelete = null;

        for (Book book : books) {

            if (book.getId() == id) {
                bookToDelete = book;
            }
        }
    }
```

```
        break;
    }
}

if (bookToDelete == null) {
    return ResponseEntity.notFound().build();
}

books.remove(bookToDelete);

return ResponseEntity.noContent().build();
}
}
```

Book.java

```
package com.example.bookapi.model;

public class Book {

    private Integer id;
    private String title;
    private String author;
    private double price;

    public Book() {
    }

    public Book(Integer id, String title, String author, double price) {
        this.id = id;
        this.title = title;
        this.author = author;
        this.price = price;
    }

    public Integer getId() {
        return id;
    }

    public void setId(Integer id) {
        this.id = id;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getAuthor() {
        return author;
    }

    public void setAuthor(String author) {
        this.author = author;
    }

    public double getPrice() {
        return price;
    }

    public void setPrice(double price) {
        this.price = price;
    }
}
```

P9Application.java

Enroll – 25CI2110039 | Name – Kalariya Heli

```
package com.example.bookapi;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class P9Application {

    public static void main(String[] args) {
        SpringApplication.run(P9Application.class, args);
    }

}
```

P9ApplicationTests.java

```
package com.example.bookapi;

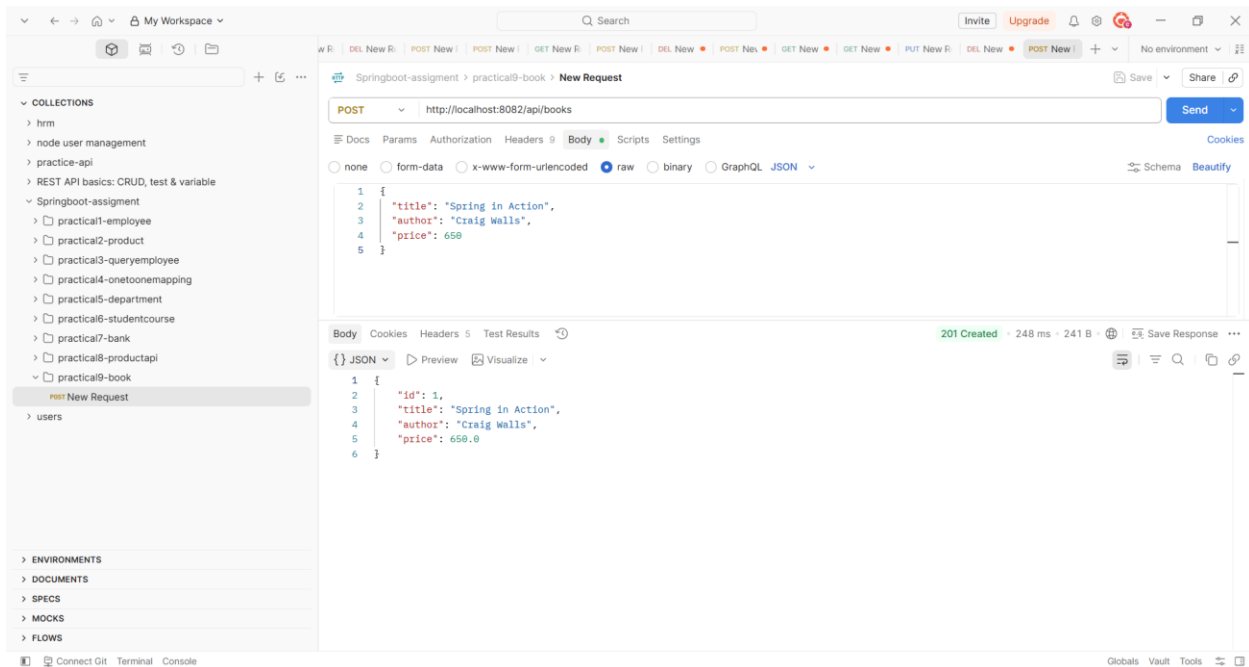
import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class P9ApplicationTests {

    @Test
    void contextLoads() {
    }

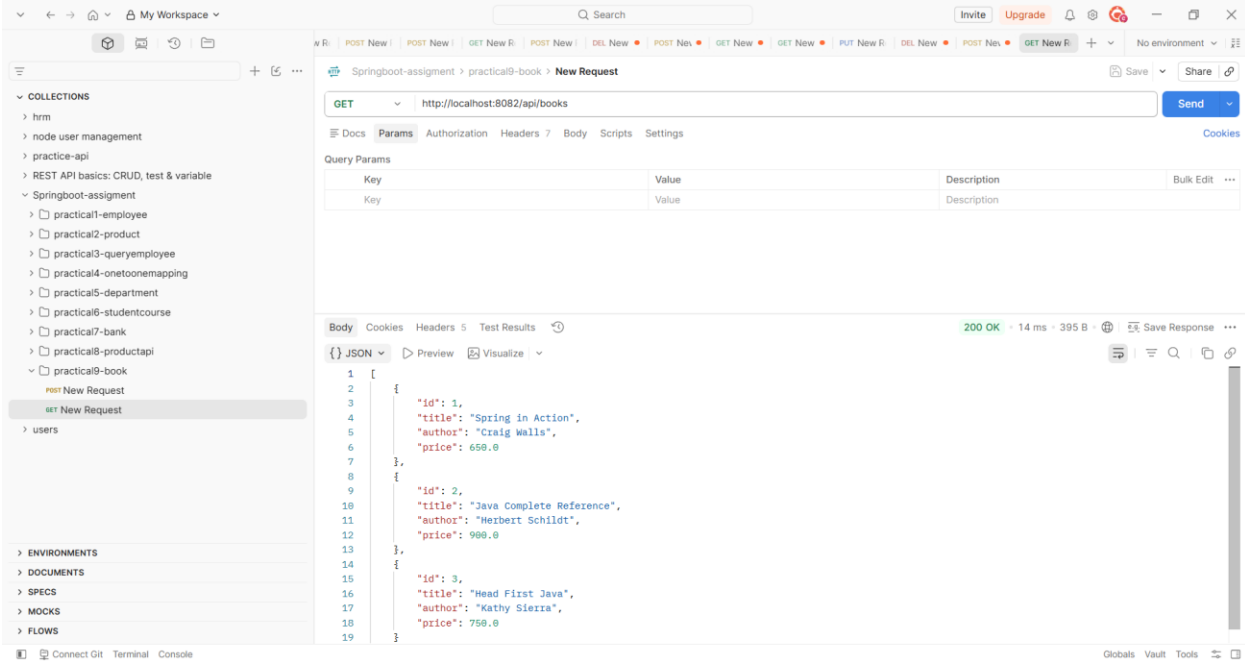
}
```

Post Method

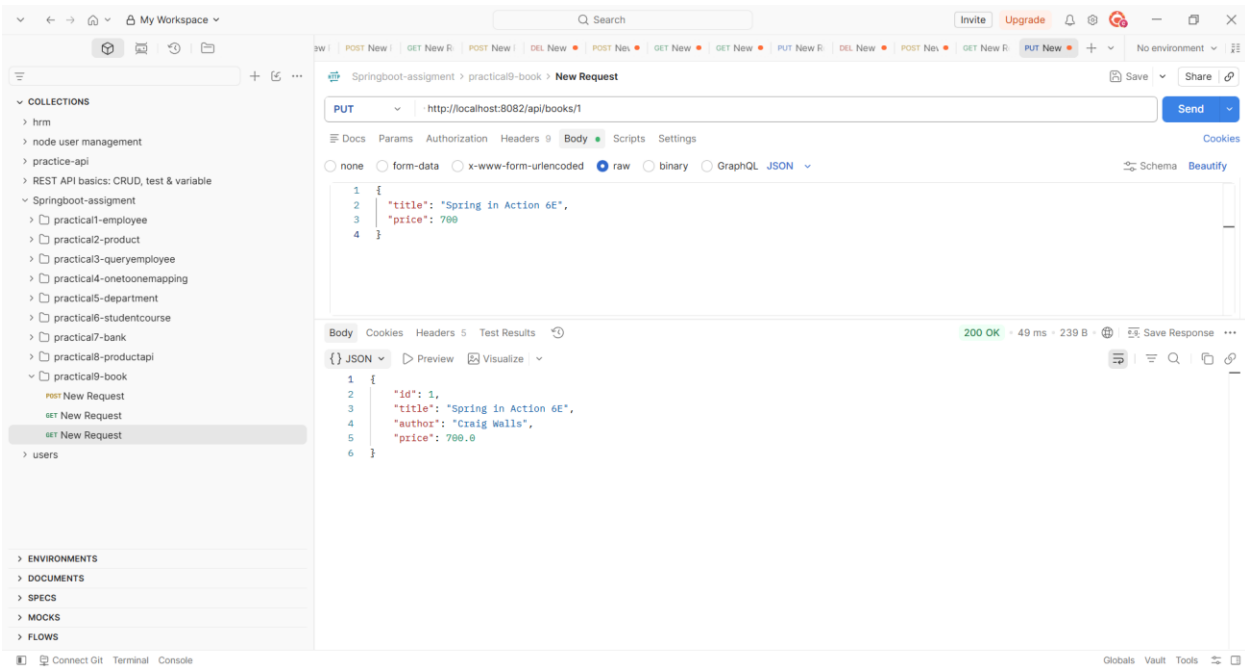


Get Method

Enroll – 25CI2110039 | Name – Kalariya Heli

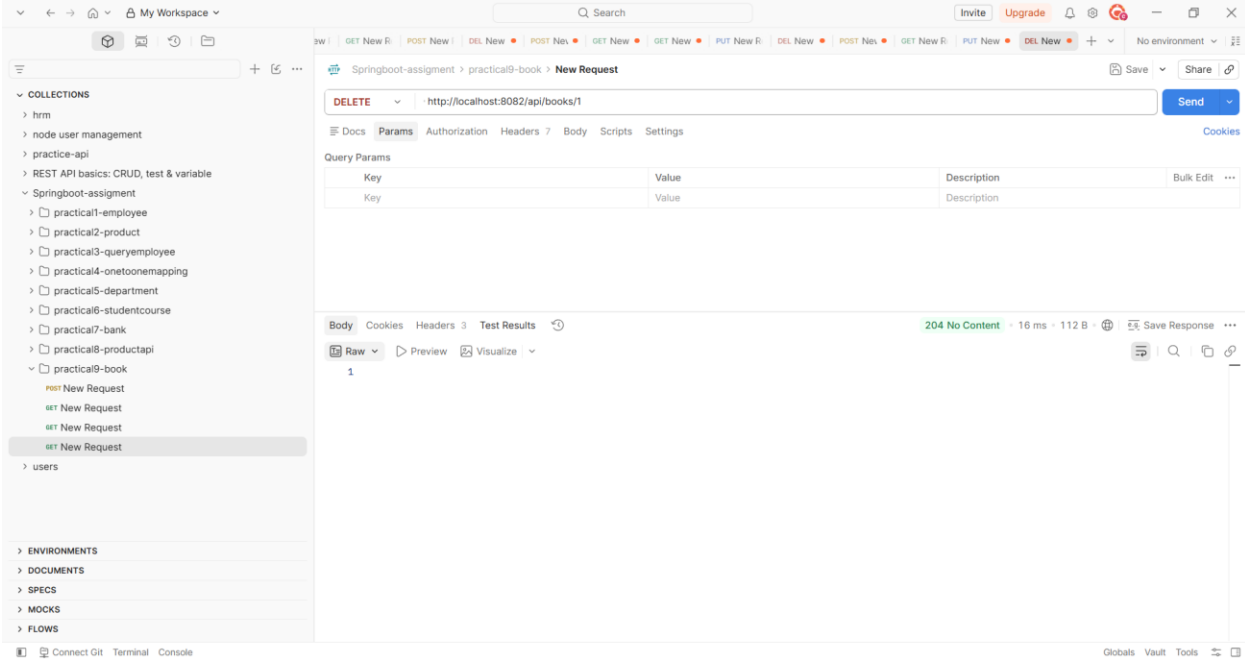


Put Method



Delete Method

Enroll – 25CI2110039 | Name – Kalariya Heli



Method	Endpoint (Postman)	Request Body / Params	Expected Response
POST	http://localhost:8082/api/books	{"title":"Spring in Action","author":"Craig Walls","price":650}	201 Created — Book added successfully and generated ID returned
GET	http://localhost:8082/api/books	None	200 OK — Displays all Books stored in the in-memory list
GET	http://localhost:8082/api/books/1	Path Variable: id = 1	200 OK — Displays Book with ID 1
GET	http://localhost:8082/api/books/999	Path Variable: id = 999	404 Not Found — Book does not exist

PUT	http://localhost:8082/api/books/1	{"title":"Spring in Action 6E","price":700}	200 OK — Existing Book updated successfully
DELETE	http://localhost:8082/api/books/1	Path Variable: id = 1	204 No Content — Book deleted successfully

Practical No. 10: Layered Microservice Architecture – RestController + Service + JPA Repository

Definition / Objective

Service + JPA Repository

Textbook Reference: Chapter 6 — Building RESTful Microservices

Objective:

To structure a RESTful microservice into the standard Controller–Service–Repository layers recommended in this

chapter, keeping business logic out of the controller and persistence access out of the service.

Class Name(s) with Attributes:

Product (@Entity, reused)

- productId, productName, category, price, quantity

ProductService (interface)

- CRUD method signatures

ProductServiceImpl (@Service)

- productRepository : ProductRepository

ProductRestController (@RestController)

- productService : ProductService

Step-by-Step Task Instructions:

1. Create ProductService (interface) declaring create, getAll, getById, update, delete methods.
2. Create ProductServiceImpl annotated @Service, injecting ProductRepository and implementing the interface.
3. Create ProductRestController injecting ProductService only (never the repository directly), exposing the full

CRUD REST API.

4. For update, fetch the existing entity by id in the service layer, apply changes, and call save() again.
5. Confirm via code review that the controller has no direct repository reference (proper layering).
6. Test the complete CRUD lifecycle in Postman, saving requests into a Postman Collection for reuse.

Code

ProductRestController.java

```
package com.example.layeredapi.controller;

import java.util.List;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.example.layeredapi.entity.Product;
import com.example.layeredapi.service.ProductService;

@RestController
@RequestMapping("/api/products")
public class ProductRestController {

    private final ProductService productService;

    public ProductRestController(
        ProductService productService) {

        this.productService = productService;
    }

    @PostMapping
    public ResponseEntity<Product> createProduct(
        @RequestBody Product product) {

        Product saved =
            productService.createProduct(product);

        return ResponseEntity
            .status(HttpStatus.CREATED)
            .body(saved);
    }

    @GetMapping
    public ResponseEntity<List<Product>> getAllProducts() {

        return ResponseEntity.ok(
            productService.getAllProducts());
    }

    @GetMapping("/{id}")
    public ResponseEntity<Product> getProductById(
        @PathVariable int id) {

        Product product =
            productService.getProductById(id);

        if (product == null) {
```

```
        return ResponseEntity.notFound().build();
    }

    return ResponseEntity.ok(product);
}

@PutMapping("/{id}")
public ResponseEntity<Product> updateProduct(
    @PathVariable int id,
    @RequestBody Product details) {

    Product updated =
        productService.updateProduct(id, details);

    if (updated == null) {
        return ResponseEntity.notFound().build();
    }

    return ResponseEntity.ok(updated);
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteProduct(
    @PathVariable int id) {

    boolean deleted =
        productService.deleteProduct(id);

    if (!deleted) {
        return ResponseEntity.notFound().build();
    }

    return ResponseEntity.noContent().build();
}
}
```

Product.java

```
package com.example.layeredapi.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int productId;

    private String productName;
    private String category;
    private double price;
    private int quantity;

    public Product() {
        super();
        // TODO Auto-generated constructor stub
    }

    public int getProductId() {
        return productId;
    }

    public void setProductId(int productId) {
        this.productId = productId;
    }

    public String getProductName() {
        return productName;
    }
}
```

```
    }  
    public void setProductName(String productName) {  
        this.productName = productName;  
    }  
    public String getCategory() {  
        return category;  
    }  
    public void setCategory(String category) {  
        this.category = category;  
    }  
    public double getPrice() {  
        return price;  
    }  
    public void setPrice(double price) {  
        this.price = price;  
    }  
    public int getQuantity() {  
        return quantity;  
    }  
    public void setQuantity(int quantity) {  
        this.quantity = quantity;  
    }  
}
```

P10Application.java

```
package com.example.layeredapi;  
  
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
  
@SpringBootApplication  
public class P10Application {  
  
    public static void main(String[] args) {  
        SpringApplication.run(P10Application.class, args);  
    }  
}
```

ProductRepository.java

```
package com.example.layeredapi.repository;  
  
import org.springframework.data.jpa.repository.JpaRepository;  
  
import com.example.layeredapi.entity.Product;  
  
public interface ProductRepository  
    extends JpaRepository<Product, Integer> {  
  
}
```

ProductService.java

```
package com.example.layeredapi.service;  
  
import java.util.List;  
  
import com.example.layeredapi.entity.Product;  
  
public interface ProductService {  
  
    Product createProduct(Product product);  
  
    List<Product> getAllProducts();  
  
    Product getProductById(int id);  
}
```

```
        Product updateProduct(int id, Product product);

        boolean deleteProduct(int id);
    }
}
```

ProductServiceImpl.java

```
package com.example.layeredapi.serviceimpl;

import java.util.List;

import org.springframework.stereotype.Service;

import com.example.layeredapi.entity.Product;
import com.example.layeredapi.repository.ProductRepository;
import com.example.layeredapi.service.ProductService;

@Service
public class ProductServiceImpl
    implements ProductService {

    private final ProductRepository productRepository;

    public ProductServiceImpl(
        ProductRepository productRepository) {

        this.productRepository = productRepository;
    }

    @Override
    public Product createProduct(Product product) {

        return productRepository.save(product);
    }

    @Override
    public List<Product> getAllProducts() {

        return productRepository.findAll();
    }

    @Override
    public Product getProductById(int id) {

        return productRepository.findById(id)
            .orElse(null);
    }

    @Override
    public Product updateProduct(
        int id,
        Product details) {

        Product existingProduct =
            productRepository.findById(id)
                .orElse(null);

        if (existingProduct == null) {
            return null;
        }

        if (details.getProductName() != null) {
            existingProduct.setProductName(
                details.getProductName());
        }

        if (details.getCategory() != null) {
            existingProduct.setCategory(
```

```
        details.getCategory());
    }

    if (details.getPrice() > 0) {
        existingProduct.setPrice(
            details.getPrice());
    }

    if (details.getQuantity() > 0) {
        existingProduct.setQuantity(
            details.getQuantity());
    }

    return productRepository.save(existingProduct);
}

@Override
public boolean deleteProduct(int id) {

    if (!productRepository.existsById(id)) {
        return false;
    }

    productRepository.deleteById(id);

    return true;
}
}
```

P10ApplicationTests.java

```
package com.example.layeredapi;

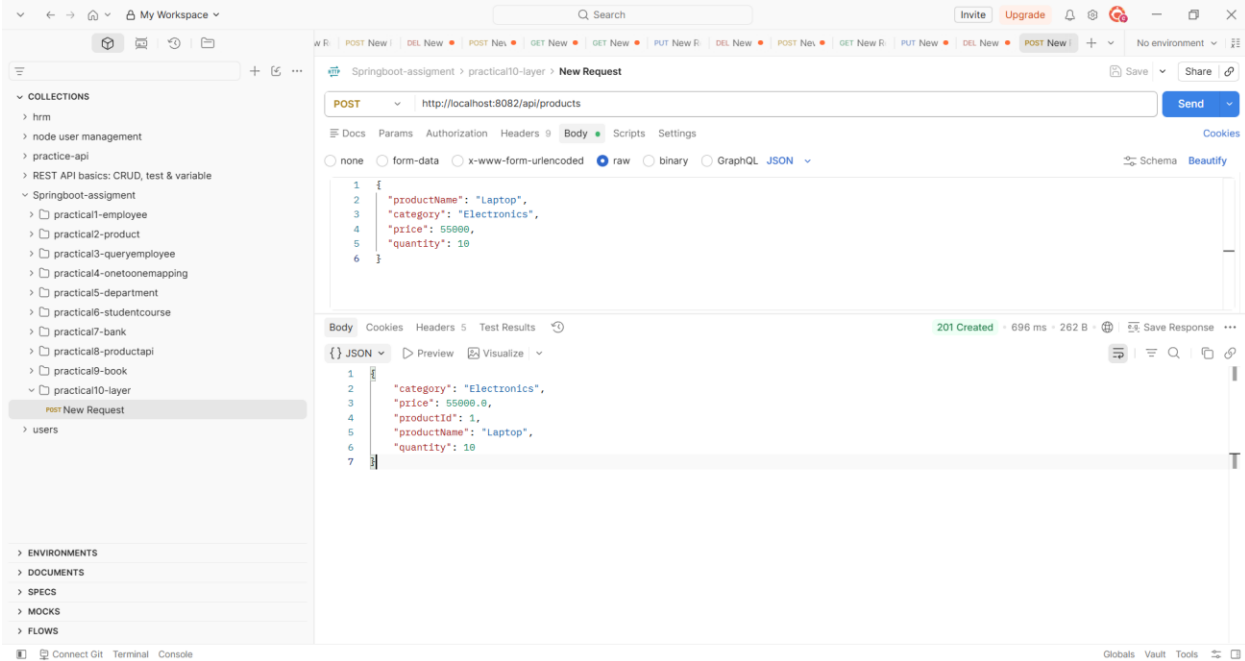
import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class P10ApplicationTests {

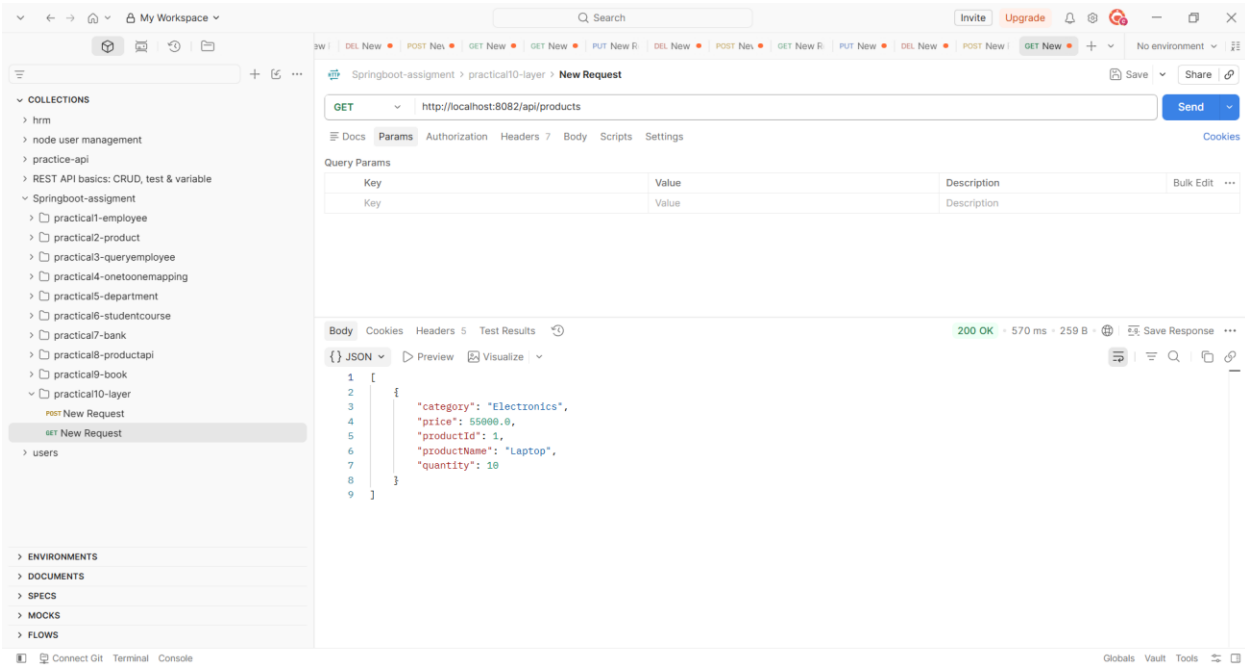
    @Test
    void contextLoads() {
    }

}
```

Post Method

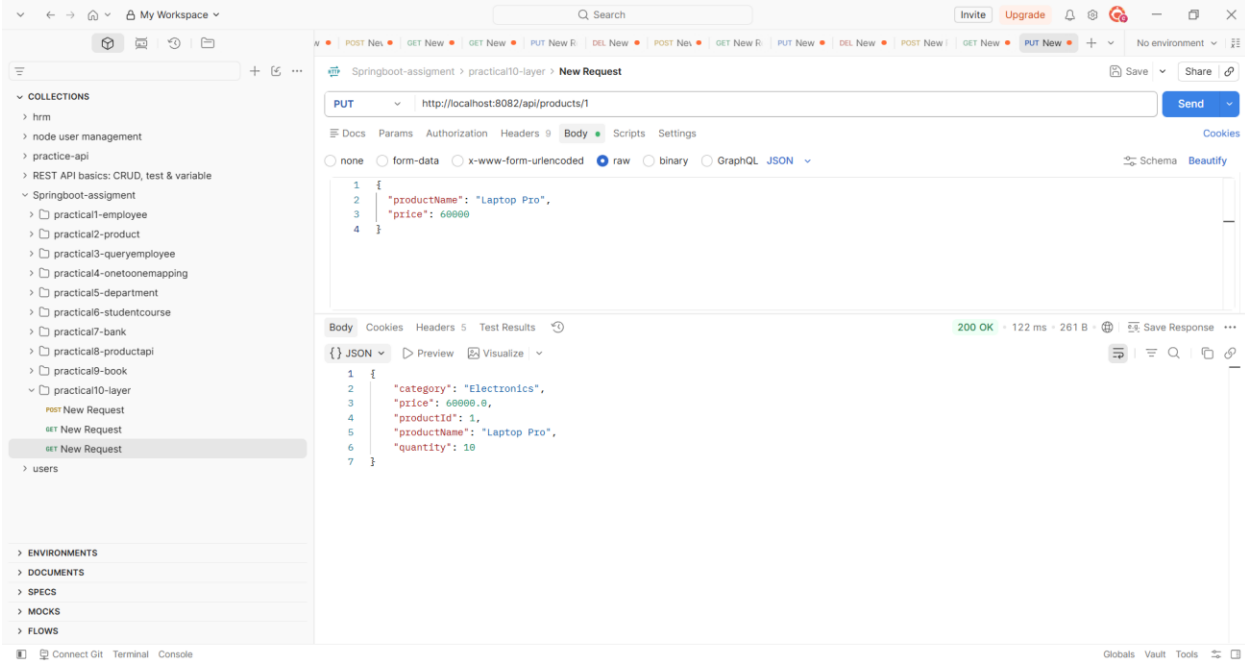


Get Method

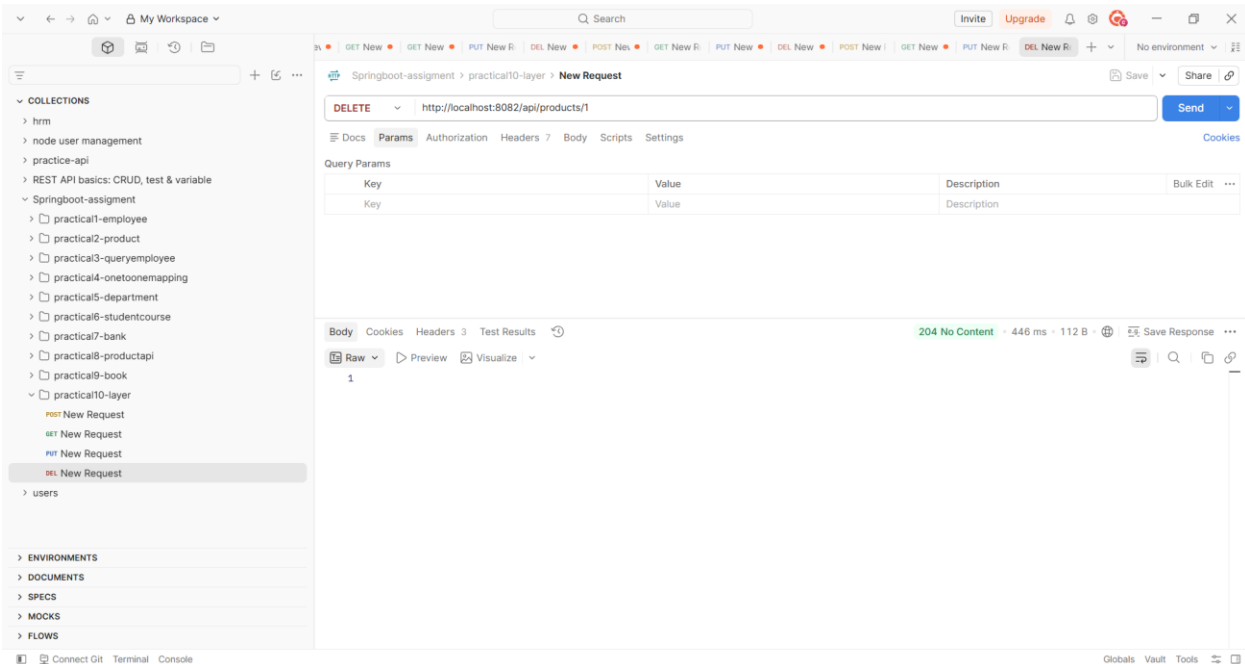


Put Method

Enroll – 25CI2110039 | Name – Kalariya Heli



Delete Method



Met hod	Endpoint (Postman)	Request Body / Params	Expect ed
------------	--------------------	-----------------------	--------------

			Response
POST	http://localhost:8082/api/products	{"productName":"Laptop","category":"Electronics","price":55000,"quantity":10}	201 Created — Product created through Controller → Service → Repository layers
GET	http://localhost:8082/api/products	None	200 OK — Displays list of all products
GET	http://localhost:8082/api/products/1	Path Variable: id = 1	200 OK — Displays Product with ID 1
PUT	http://localhost:8082/api/products/1	{"productName":"Laptop Pro","price":60000}	200 OK — Existing Product

			t fetche d and update d in Service layer
DELE TE	http://localhost:8082/a pi/products/1	Path Variable: id = 1	204 No Conten t — Produc t delete d succes sfully

Practical No. 11: Handling Request Data – @PathVariable, @RequestParam and @RequestBody

Definition / Objective

and @RequestBody

Textbook Reference: Chapter 6 — Building RESTful Microservices

Objective:

To correctly use the three main ways a Spring REST controller receives client input — @PathVariable for

identifying a resource, @RequestParam for optional query filters, and @RequestBody for JSON payloads — as

covered in the request-handling section of this chapter.

Class Name(s) with Attributes:

Customer (@Entity)

- customerId : int (@Id @GeneratedValue)

- name : String

- city : String

Enroll – 25CI2110039 | Name – Kalariya Heli

- age : int

CustomerRestController (@RestController)

- Method: getById(@PathVariable id)

- Method: search(@RequestParam city, @RequestParam(required=false) minAge)

- Method: create(@RequestBody CustomerDTO)

Step-by-Step Task Instructions:

1. Create the Customer entity and CustomerRepository extending JpaRepository.
2. Implement GET /api/customers/{id} using @PathVariable to fetch a single customer.
3. Implement GET /api/customers/search?city=..&minAge=.. using @RequestParam (with minAge optional and defaulting to 0).
4. Implement POST /api/customers accepting the full Customer object via @RequestBody.
5. Implement PUT /api/customers/{id} (combining @PathVariable for the id with @RequestBody for the updated fields) and DELETE /api/customers/{id} (using @PathVariable alone) to complete the CRUD set.
6. Add logging (System.out / SLF4J) inside each handler to print which parameter source was used, for demonstration.
7. Test all three parameter-passing styles from Postman using the URL bar, query params tab, and body tab respectively.

Code

CustomerRestController.java

```
package com.example.customerapi.controller;

import java.util.List;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import com.example.customerapi.entity.Customer;
import com.example.customerapi.repository.CustomerRepository;

@RestController
@RequestMapping("/api/customers")
public class CustomerRestController {

    private final CustomerRepository customerRepository;

    public CustomerRestController(
        CustomerRepository customerRepository) {
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
        this.customerRepository = customerRepository;
    }

    @PostMapping
    public ResponseEntity<Customer> createCustomer(
        @RequestBody Customer customer) {

        System.out.println(
            "POST: Data received using @RequestBody");

        Customer saved =
            customerRepository.save(customer);

        return ResponseEntity
            .status(HttpStatus.CREATED)
            .body(saved);
    }

    @GetMapping
    public ResponseEntity<List<Customer>> getAllCustomers() {

        return ResponseEntity.ok(
            customerRepository.findAll());
    }

    @GetMapping("/{id}")
    public ResponseEntity<Customer> getById(
        @PathVariable int id) {

        System.out.println(
            "GET: id received using @PathVariable = " + id);

        Customer customer =
            customerRepository.findById(id)
                .orElse(null);

        if (customer == null) {
            return ResponseEntity.notFound().build();
        }

        return ResponseEntity.ok(customer);
    }

    @GetMapping("/search")
    public ResponseEntity<List<Customer>> searchCustomers(
        @RequestParam String city,
        @RequestParam(defaultValue = "0") int minAge) {

        System.out.println(
            "GET search: city and minAge received using @RequestParam");

        List<Customer> customers =
            customerRepository
                .findByCityAndAgeGreaterThanOrEqualTo(
                    city,
                    minAge);

        return ResponseEntity.ok(customers);
    }

    @PutMapping("/{id}")
    public ResponseEntity<Customer> updateCustomer(
        @PathVariable int id,
        @RequestBody Customer details) {

        System.out.println(
            "PUT: id from @PathVariable and data from @RequestBody");

        Customer customer =
```

```
        customerRepository.findById(id)
            .orElse(null);

    if (customer == null) {
        return ResponseEntity.notFound().build();
    }

    if (details.getName() != null) {
        customer.setName(details.getName());
    }

    if (details.getCity() != null) {
        customer.setCity(details.getCity());
    }

    if (details.getAge() > 0) {
        customer.setAge(details.getAge());
    }

    Customer updated =
        customerRepository.save(customer);

    return ResponseEntity.ok(updated);
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteCustomer(
    @PathVariable int id) {

    System.out.println(
        "DELETE: id received using @PathVariable = " + id);

    if (!customerRepository.existsById(id)) {
        return ResponseEntity.notFound().build();
    }

    customerRepository.deleteById(id);

    return ResponseEntity.noContent().build();
}
}
```

Customer.java

```
package com.example.customerapi.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Customer {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int customerId;

    private String name;
    private String city;
    private int age;
    public Customer() {
        super();
        // TODO Auto-generated constructor stub
    }
    public int getCustomerId() {
        return customerId;
    }
    public void setCustomerId(int customerId) {
```

```
        this.customerId = customerId;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getCity() {
        return city;
    }
    public void setCity(String city) {
        this.city = city;
    }
    public int getAge() {
        return age;
    }
    public void setAge(int age) {
        this.age = age;
    }
}

}
```

P11Application.java

```
package com.example.customerapi;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class P11Application {

    public static void main(String[] args) {
        SpringApplication.run(P11Application.class, args);
    }

}
```

CustomerRepository.java

```
package com.example.customerapi.repository;

import java.util.List;

import org.springframework.data.jpa.repository.JpaRepository;

import com.example.customerapi.entity.Customer;

public interface CustomerRepository
    extends JpaRepository<Customer, Integer> {

    List<Customer> findByCityAndAgeGreaterThanOrEqual(
        String city,
        int age);
}
```

P11ApplicationTests.java

```
package com.example.customerapi;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

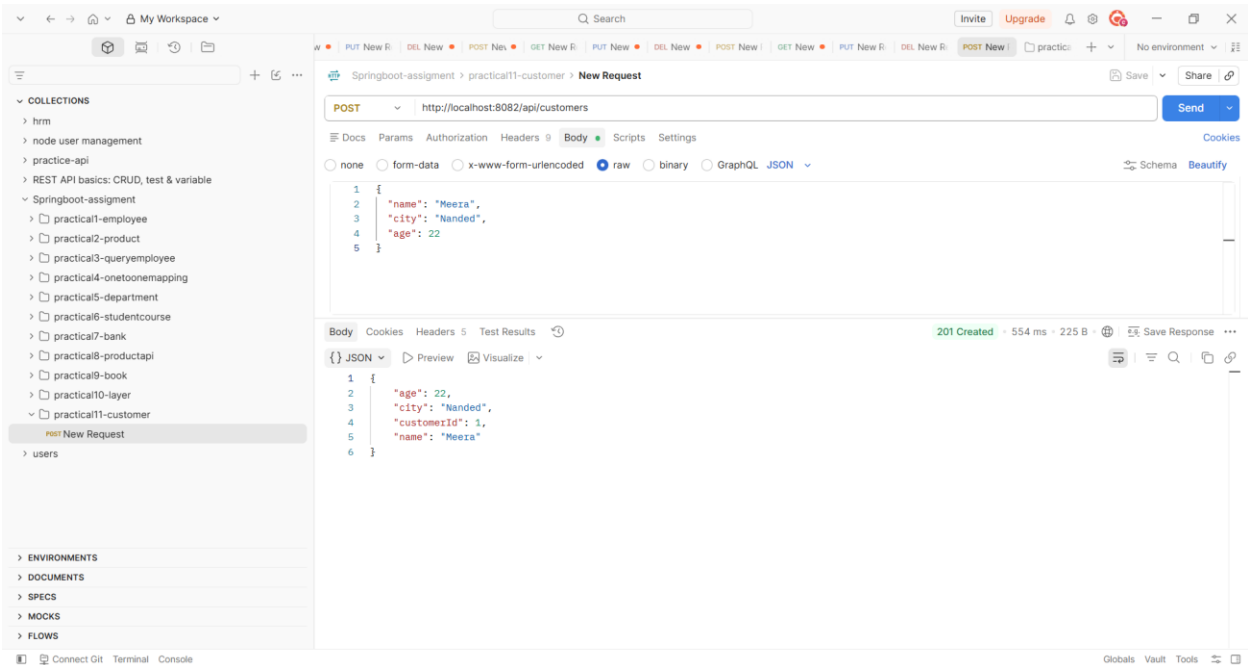
@SpringBootTest
class P11ApplicationTests {

    @Test
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
void contextLoads() {  
}  
  
}
```

Post Method



Get Method

Enroll – 25CI2110039 | Name – Kalariya Heli

The screenshot shows the Postman interface with a workspace named "My Workspace". On the left, a collection tree is visible under "Springboot-assignment" > "practical11-customer", with "New Request" selected. The main panel displays a GET request to "http://localhost:8082/api/customers/1". The "Params" tab is active, showing an empty table. The "Body" tab shows a JSON response:

```
{
  "age": 22,
  "city": "Nanded",
  "customerId": 1,
  "name": "Meeza"
}
```

The status bar indicates a "200 OK" response with a time of 91 ms and a body size of 220 B.

Get Method – by minage

The screenshot shows the Postman interface with a workspace named "My Workspace". On the left, a collection tree is visible under "Springboot-assignment" > "practical11-customer", with "New Request" selected. The main panel displays a GET request to "http://localhost:8082/api/customers/search?city=Pune&minAge=25". The "Params" tab is active, showing a table with two parameters:

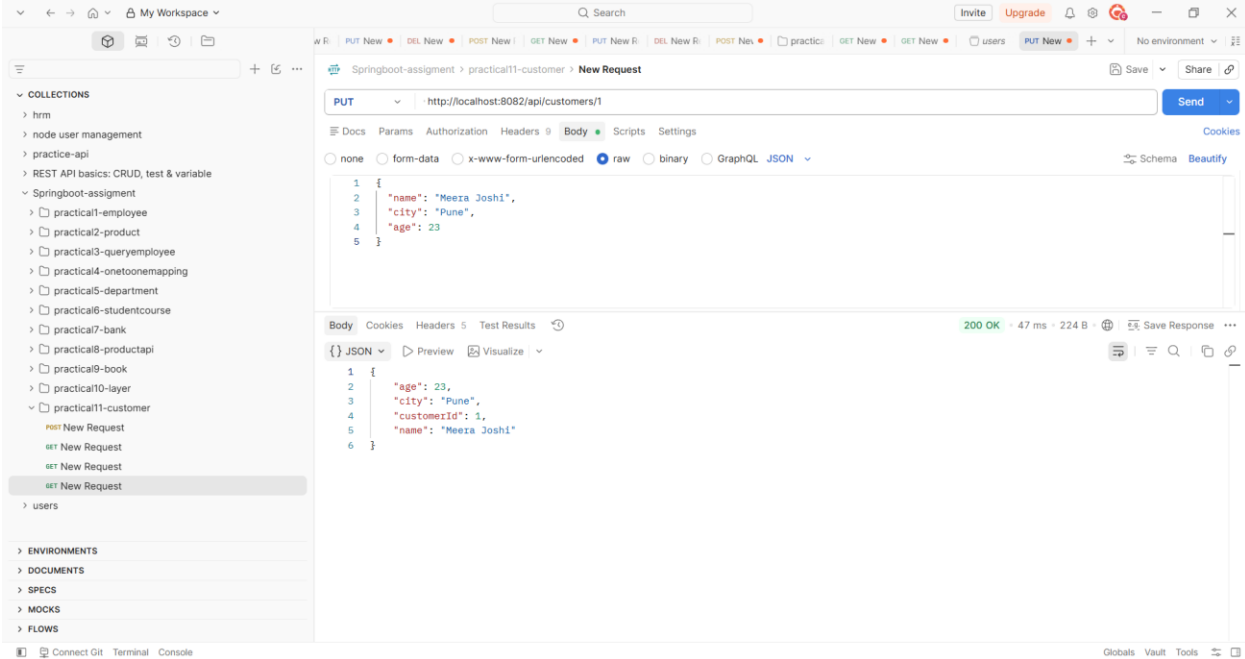
Key	Value	Description	Bulk Edit
city	Pune		
minAge	25		

The "Body" tab shows a JSON response:

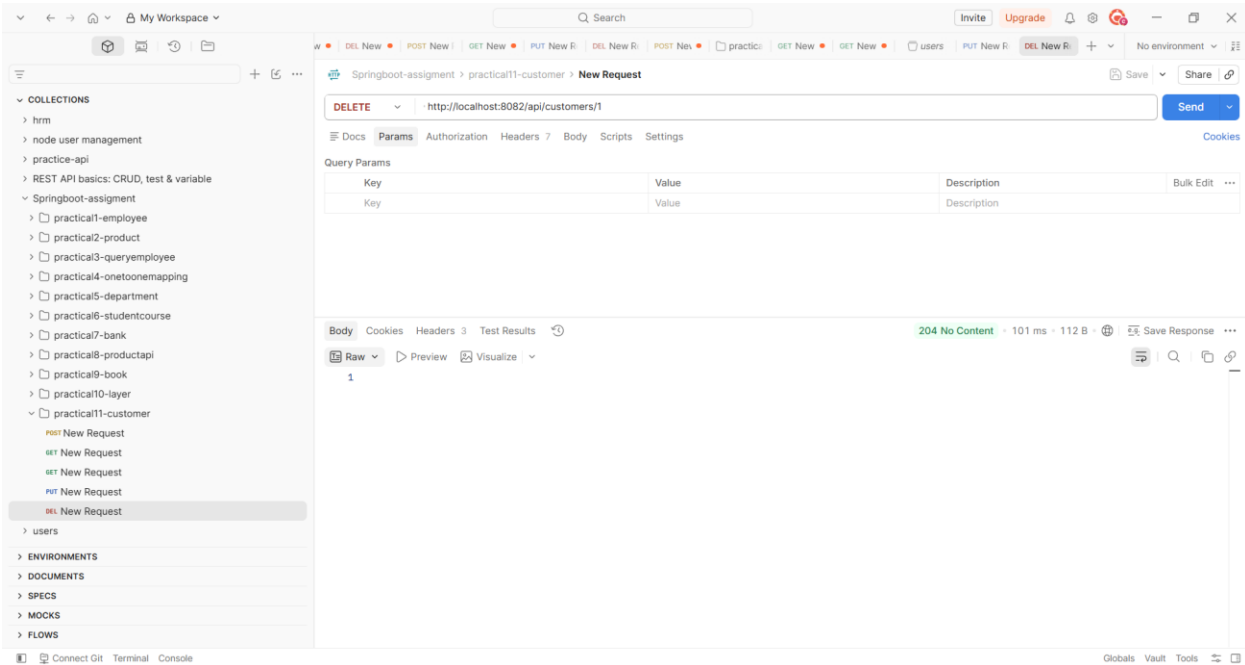
```
[
  {
    "age": 32,
    "city": "Pune",
    "customerId": 3,
    "name": "Amit"
  }
]
```

The status bar indicates a "200 OK" response with a time of 723 ms and a body size of 219 B.

Put Method



Delete Method



Met hod	Endpoint (Postman)	Request Body / Params	Expected Response
POS T	http://localhost:8082/api/customers	<code>{\"name\":\"Meera\",\"city\":\"Nanded\",\"age\":22}</code>	201 Created — Customer

			created using @RequestBody
GET	http://localhost:8082/api/customers/3	Path Variable: id = 3	200 OK — Customer retrieved using @PathVariable
GET	http://localhost:8082/api/customers/search?city=Pune&minAge=25	Request Params: city = Pune, minAge = 25	200 OK — Filtered customers returned using @RequestParam
GET	http://localhost:8082/api/customers/search?city=Pune	Request Param: city = Pune; minAge omitted, defaults to 0	200 OK — All Pune customers returned
PUT	http://localhost:8082/api/customers/3	{"name":"Meera Joshi","city":"Pune","age":23}	200 OK — Customer updated using @PathVariable + @RequestBody
DELETE	http://localhost:8082/api/customers/3	Path Variable: id = 3	204 No Content — Customer deleted using @PathVariable

Practical No. 12: DTO Pattern – Separating the Entity from the API Contract

Definition / Objective

Textbook Reference: Chapter 6 — Building RESTful Microservices

Objective:

To decouple the persistent JPA entity from the data actually exposed by the REST API using the Data Transfer

Object (DTO) pattern discussed in this chapter, hiding sensitive/internal fields and mapping between Entity and

DTO.

Class Name(s) with Attributes:

Employee (@Entity — add sensitive field)

- empId, empName, department, salary
- password : String (sensitive — never exposed)

EmployeeDTO (POJO)

- empId : int
- empName : String
- department : String

EmployeeMapper (utility class)

- static EmployeeDTO toDto(Employee e)
- static Employee toEntity(EmployeeDTO d)

Step-by-Step Task Instructions:

1. Add a password field to the Employee entity to represent sensitive data that should never leave the server.
2. Create the EmployeeDTO class containing only the fields safe to expose.
3. Create EmployeeMapper with static conversion methods toDto() and toEntity().
4. Modify EmployeeRestController's GET endpoints to convert Employee → EmployeeDTO using the mapper before returning the response.
5. Modify the POST endpoint to accept an EmployeeDTO, convert it to an entity, and save.
6. Add PUT /api/employees/{id} that accepts an EmployeeDTO, maps it onto the existing entity (preserving the untouched password field), and saves; add DELETE /api/employees/{id} to remove the record.
7. Test GET /api/employees and confirm the password field is absent from the JSON response.

Code

EmployeeRestController.java

```
package com.example.dtoapi.controller;

import java.util.List;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.example.dtoapi.dto.EmployeeDTO;
import com.example.dtoapi.entity.Employee;
import com.example.dtoapi.mapper.EmployeeMapper;
import com.example.dtoapi.repository.EmployeeRepository;

@RestController
@RequestMapping("/api/employees")
public class EmployeeRestController {

    private final EmployeeRepository employeeRepository;

    public EmployeeRestController(
        EmployeeRepository employeeRepository) {

        this.employeeRepository = employeeRepository;
    }

    @PostMapping
    public ResponseEntity<EmployeeDTO> createEmployee(
        @RequestBody EmployeeDTO dto) {

        Employee employee =
            EmployeeMapper.toEntity(dto);

        employee.setPassword("internal123");

        Employee savedEmployee =
            employeeRepository.save(employee);

        return ResponseEntity
            .status(HttpStatus.CREATED)
            .body(EmployeeMapper.toDto(savedEmployee));
    }

    @GetMapping
    public ResponseEntity<List<EmployeeDTO>> getAllEmployees() {

        List<Employee> employees =
            employeeRepository.findAll();

        List<EmployeeDTO> dtoList =
            employees.stream()
                .map(EmployeeMapper::toDto)
                .toList();

        return ResponseEntity.ok(dtoList);
    }

    @GetMapping("/{id}")
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
public ResponseEntity<EmployeeDTO> getEmployeeById(
    @PathVariable int id) {

    Employee employee =
        employeeRepository.findById(id)
            .orElse(null);

    if (employee == null) {
        return ResponseEntity.notFound().build();
    }

    return ResponseEntity.ok(
        EmployeeMapper.toDto(employee));
}

@PutMapping("/{id}")
public ResponseEntity<EmployeeDTO> updateEmployee(
    @PathVariable int id,
    @RequestBody EmployeeDTO dto) {

    Employee existingEmployee =
        employeeRepository.findById(id)
            .orElse(null);

    if (existingEmployee == null) {
        return ResponseEntity.notFound().build();
    }

    if (dto.getEmpName() != null) {
        existingEmployee.setEmpName(
            dto.getEmpName());
    }

    if (dto.getDepartment() != null) {
        existingEmployee.setDepartment(
            dto.getDepartment());
    }

    Employee updatedEmployee =
        employeeRepository.save(existingEmployee);

    return ResponseEntity.ok(
        EmployeeMapper.toDto(updatedEmployee));
}

>DeleteMapping("/{id}")
public ResponseEntity<Void> deleteEmployee(
    @PathVariable int id) {

    if (!employeeRepository.existsById(id)) {
        return ResponseEntity.notFound().build();
    }

    employeeRepository.deleteById(id);

    return ResponseEntity.noContent().build();
}
}
```

EmployeeDTO.java

```
package com.example.dtoapi.dto;

public class EmployeeDTO {

    private int empId;
    private String empName;
    private String department;
    public EmployeeDTO() {
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
        super();
        // TODO Auto-generated constructor stub
    }
    public int getEmpId() {
        return empId;
    }
    public void setEmpId(int empId) {
        this.empId = empId;
    }
    public String getEmpName() {
        return empName;
    }
    public void setEmpName(String empName) {
        this.empName = empName;
    }
    public String getDepartment() {
        return department;
    }
    public void setDepartment(String department) {
        this.department = department;
    }
}
}
```

Employee.java

```
package com.example.dtoapi.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int empId;

    private String empName;
    private String department;
    private double salary;

    private String password;

    public Employee() {
        super();
        // TODO Auto-generated constructor stub
    }

    public int getEmpId() {
        return empId;
    }

    public void setEmpId(int empId) {
        this.empId = empId;
    }

    public String getEmpName() {
        return empName;
    }

    public void setEmpName(String empName) {
        this.empName = empName;
    }

    public String getDepartment() {
```

```
        return department;
    }

    public void setDepartment(String department) {
        this.department = department;
    }

    public double getSalary() {
        return salary;
    }

    public void setSalary(double salary) {
        this.salary = salary;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }
}
```

EmployeeMapper.java

```
package com.example.dtoapi.mapper;

import com.example.dtoapi.dto.EmployeeDTO;
import com.example.dtoapi.entity.Employee;

public class EmployeeMapper {

    public static EmployeeDTO toDto(Employee employee) {

        EmployeeDTO dto = new EmployeeDTO();

        dto.setEmpId(employee.getEmpId());
        dto.setEmpName(employee.getEmpName());
        dto.setDepartment(employee.getDepartment());

        return dto;
    }

    public static Employee toEntity(EmployeeDTO dto) {

        Employee employee = new Employee();

        employee.setEmpId(dto.getEmpId());
        employee.setEmpName(dto.getEmpName());
        employee.setDepartment(dto.getDepartment());

        return employee;
    }
}
```

P12Application.java

```
package com.example.dtoapi;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class P12Application {

    public static void main(String[] args) {
        SpringApplication.run(P12Application.class, args);
    }
}
```

```
}  
}
```

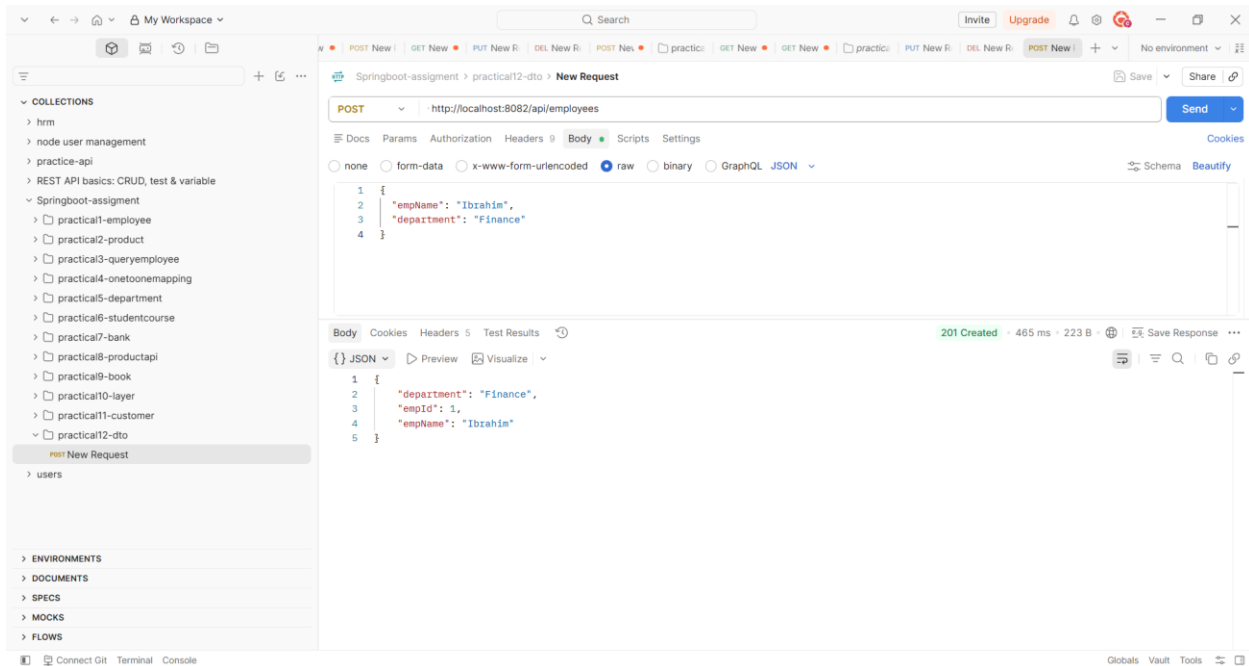
EmployeeRepository.java

```
package com.example.dtoapi.repository;  
  
import org.springframework.data.jpa.repository.JpaRepository;  
  
import com.example.dtoapi.entity.Employee;  
  
public interface EmployeeRepository  
    extends JpaRepository<Employee, Integer> {  
  
}
```

P12ApplicationTests.java

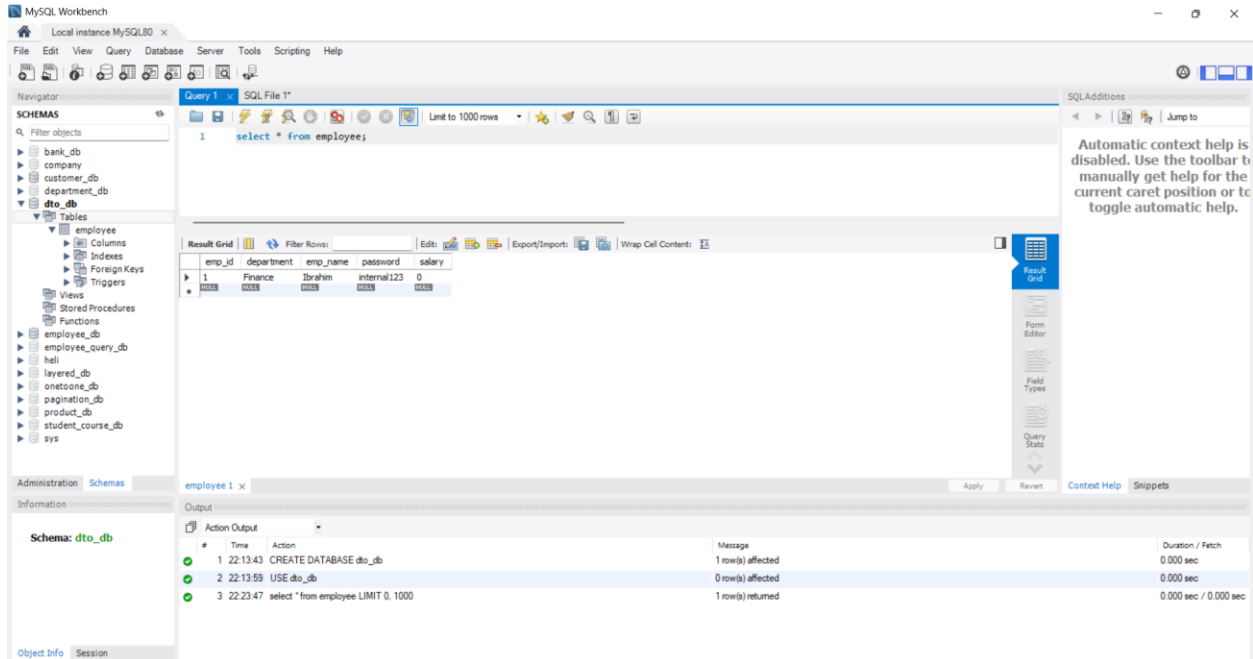
```
package com.example.dtoapi;  
  
import org.junit.jupiter.api.Test;  
import org.springframework.boot.test.context.SpringBootTest;  
  
@SpringBootTest  
class P12ApplicationTests {  
  
    @Test  
    void contextLoads() {  
    }  
  
}
```

Post Method

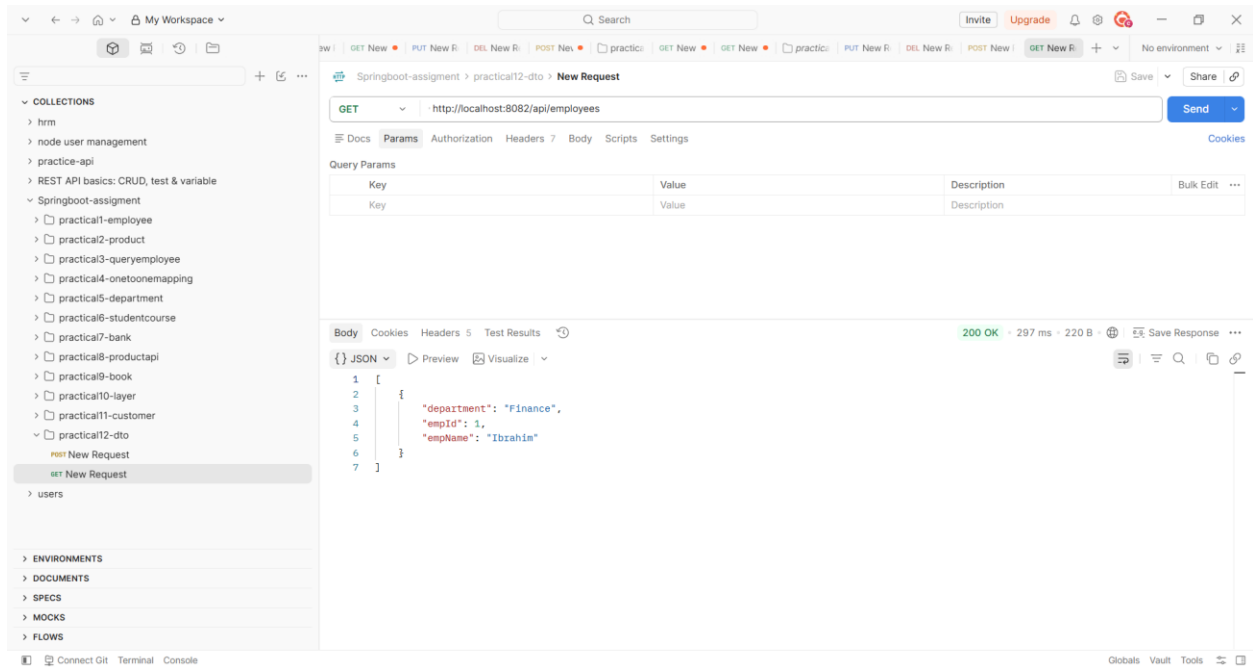


Verify Password in MySQL

Enroll – 25CI2110039 | Name – Kalariya Heli

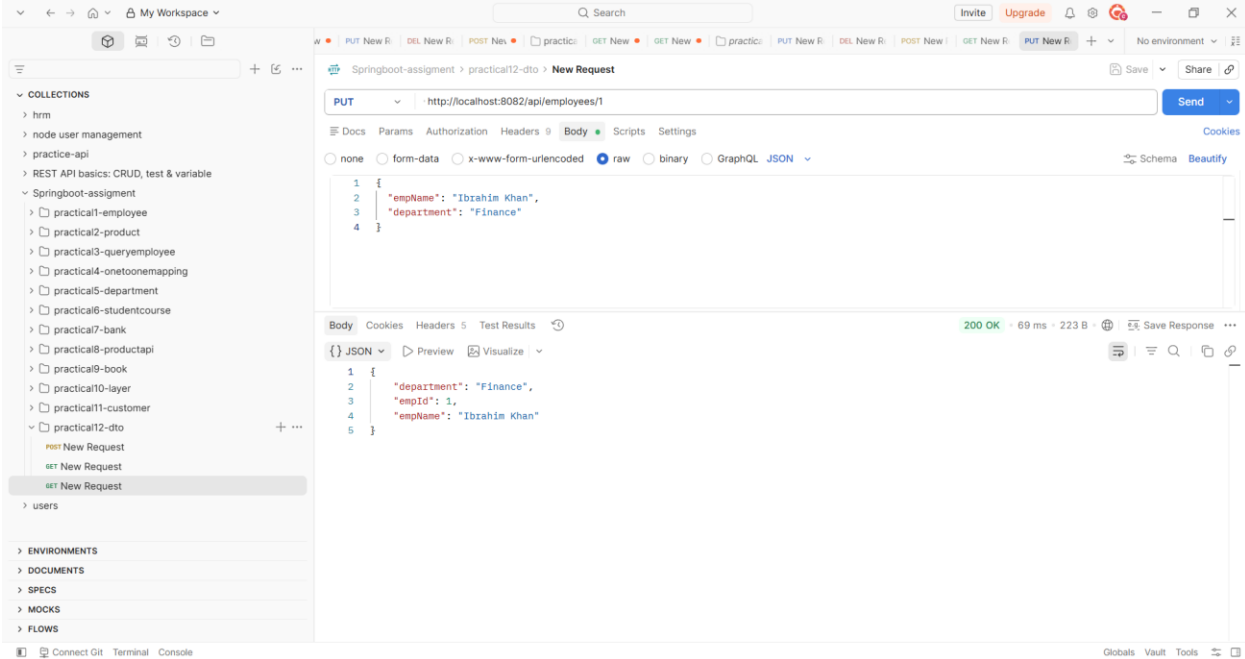


Get Method

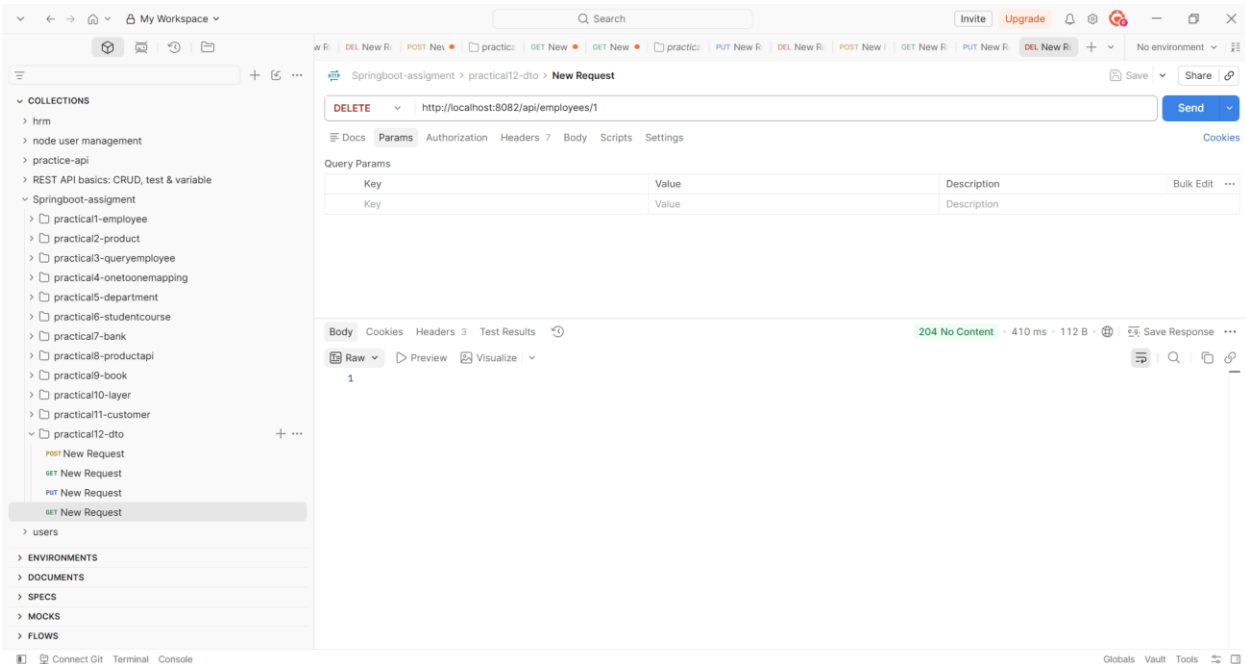


Put Method

Enroll – 25CI2110039 | Name – Kalariya Heli



Delete Method



Meth od	Endpoint (Postman)	Request Body / Params	Expected Response
POST	http://localhost:8082/api/employees	<code>{\"empName\":\"Ibrahim\",\"department\":\"Finance\"}</code>	201 Created —

			Employee DTO returned with empld, empName and department only; password is not exposed
GET	http://localhost:8082/api/employees	None	200 OK — Displays Employee DTO list containing empld, empName and department only; no password field
GET	http://localhost:8082/api/employees/1	Path Variable: id = 1	200 OK — Employee returned as Employee DTO without sensitive fields
PUT	http://localhost:8082/api/employees/1	{"empName":"Ibrahim Khan","department":"Finance"}	200 OK — Employee DTO updated;

			existing password remains unchanged internally
DELETE	http://localhost:8082/api/employees/1	Path Variable: id = 1	204 No Content — Employee deleted successfully

Practical No. 13: Request Validation using Bean Validation (@Valid) with RestController

Definition / Objective

RestController

Textbook Reference: Chapter 6 — Building RESTful Microservices

Objective:

To validate incoming JSON payloads declaratively using Jakarta Bean Validation annotations

(@NotBlank,

@Email, @Min) combined with @Valid on the controller method, as covered in this chapter's discussion of

building robust REST endpoints.

Class Name(s) with Attributes:

Customer (@Entity / DTO, reused)

- customerId : int (@Id @GeneratedValue)

- name : String (@NotBlank)

- email : String (@Email)

- age : int (@Min(18))

Step-by-Step Task Instructions:

1. Add spring-boot-starter-validation dependency.
2. Annotate the Customer class fields with @NotBlank, @Email and @Min(18) as listed.
3. In CustomerRestController, annotate the @RequestBody parameter of the create method with @Valid.
4. Run the application and POST invalid data (blank name, bad email format, age below 18) — observe Spring's default 400 Bad Request response.

5. Note that field-level error detail will be structured properly once `GlobalExceptionHandler` (next practical) is added.

6. Add GET `/api/customers/{id}` to fetch a saved customer, and PUT `/api/customers/{id}` annotated with `@Valid`

on its `@RequestBody` so updates are validated the same way as creation; add DELETE `/api/customers/{id}` to remove a customer.

7. Test again with fully valid data and confirm successful creation (201), then test an invalid PUT update and a valid DELETE.

Code

CustomerRestController.java

```
package com.example.customerapi.controller;

import java.util.List;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.example.customerapi.entity.Customer;
import com.example.customerapi.repository.CustomerRepository;

import jakarta.validation.Valid;

@RestController
@RequestMapping("/api/customers")
public class CustomerRestController {

    private final CustomerRepository customerRepository;

    public CustomerRestController(
        CustomerRepository customerRepository) {

        this.customerRepository = customerRepository;
    }

    @PostMapping
    public ResponseEntity<Customer> createCustomer(
        @Valid @RequestBody Customer customer) {

        Customer savedCustomer =
            customerRepository.save(customer);

        return ResponseEntity
            .status(HttpStatus.CREATED)
            .body(savedCustomer);
    }

    @GetMapping
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
public ResponseEntity<List<Customer>> getAllCustomers() {
    return ResponseEntity.ok(
        customerRepository.findAll());
}

@GetMapping("/{id}")
public ResponseEntity<Customer> getCustomerById(
    @PathVariable int id) {

    Customer customer =
        customerRepository.findById(id)
            .orElse(null);

    if (customer == null) {
        return ResponseEntity.notFound().build();
    }

    return ResponseEntity.ok(customer);
}

@PutMapping("/{id}")
public ResponseEntity<Customer> updateCustomer(
    @PathVariable int id,
    @Valid @RequestBody Customer details) {

    Customer customer =
        customerRepository.findById(id)
            .orElse(null);

    if (customer == null) {
        return ResponseEntity.notFound().build();
    }

    customer.setName(details.getName());
    customer.setEmail(details.getEmail());
    customer.setAge(details.getAge());

    Customer updatedCustomer =
        customerRepository.save(customer);

    return ResponseEntity.ok(updatedCustomer);
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteCustomer(
    @PathVariable int id) {

    if (!customerRepository.existsById(id)) {
        return ResponseEntity.notFound().build();
    }

    customerRepository.deleteById(id);

    return ResponseEntity.noContent().build();
}
}
```

Customer.java

```
package com.example.customerapi.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
@Entity
public class Customer {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int customerId;

    @NotBlank
    private String name;

    @Email
    private String email;

    @Min(18)
    private int age;

    public Customer() {
    }

    public int getCustomerId() {
        return customerId;
    }

    public void setCustomerId(int customerId) {
        this.customerId = customerId;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }
}
```

P13Application.java

```
package com.example.customerapi;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class P13Application {

    public static void main(String[] args) {
        SpringApplication.run(P13Application.class, args);
    }

}
```

CustomerRepository.java

```
package com.example.customerapi.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.example.customerapi.entity.Customer;

public interface CustomerRepository
    extends JpaRepository<Customer, Integer> {

}
```

P13ApplicationTests.java

```
package com.example.customerapi;

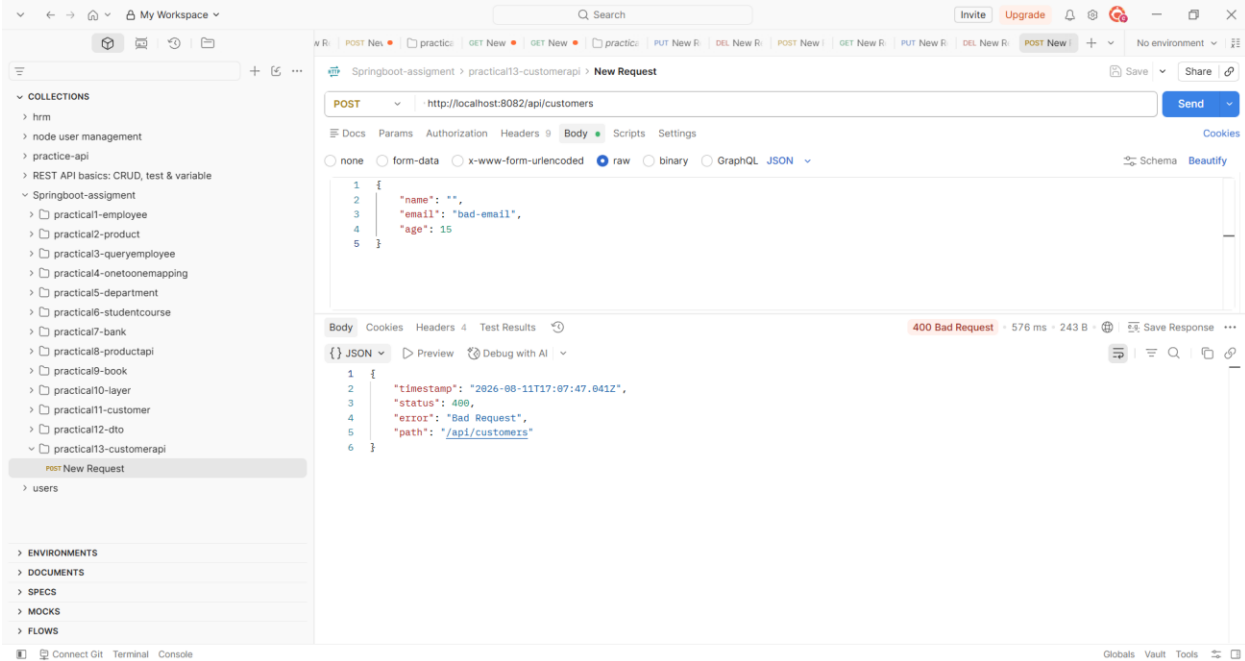
import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class P13ApplicationTests {

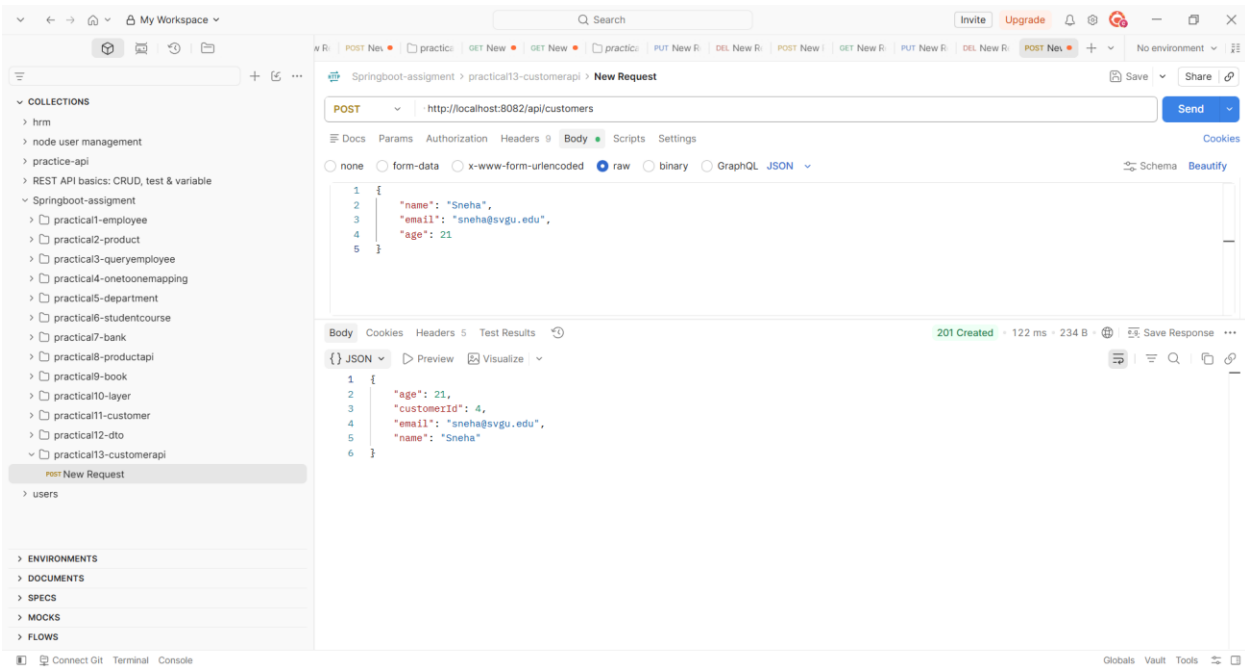
    @Test
    void contextLoads() {
    }

}
```

Post Method – invalid testing data



Post Method – valid data



Get Method

Enroll – 25CI2110039 | Name – Kalariya Heli

The screenshot shows the Postman interface with a workspace named 'My Workspace'. On the left, a 'COLLECTIONS' sidebar lists various API collections, including 'Springboot-assignment' and 'practical13-customerapi'. The main panel displays a 'New Request' for the GET method at the URL 'http://localhost:8082/api/customers/4'. The 'Query Params' section is empty. The 'Body' tab is selected, showing a JSON response:

```
1 {
2   "age": 21,
3   "customerId": 4,
4   "email": "sneha@svgu.edu",
5   "name": "Sneha"
6 }
```

The status bar at the bottom indicates a '200 OK' response with a 16 ms duration and 229 B of data.

Put Method – invalid data testing

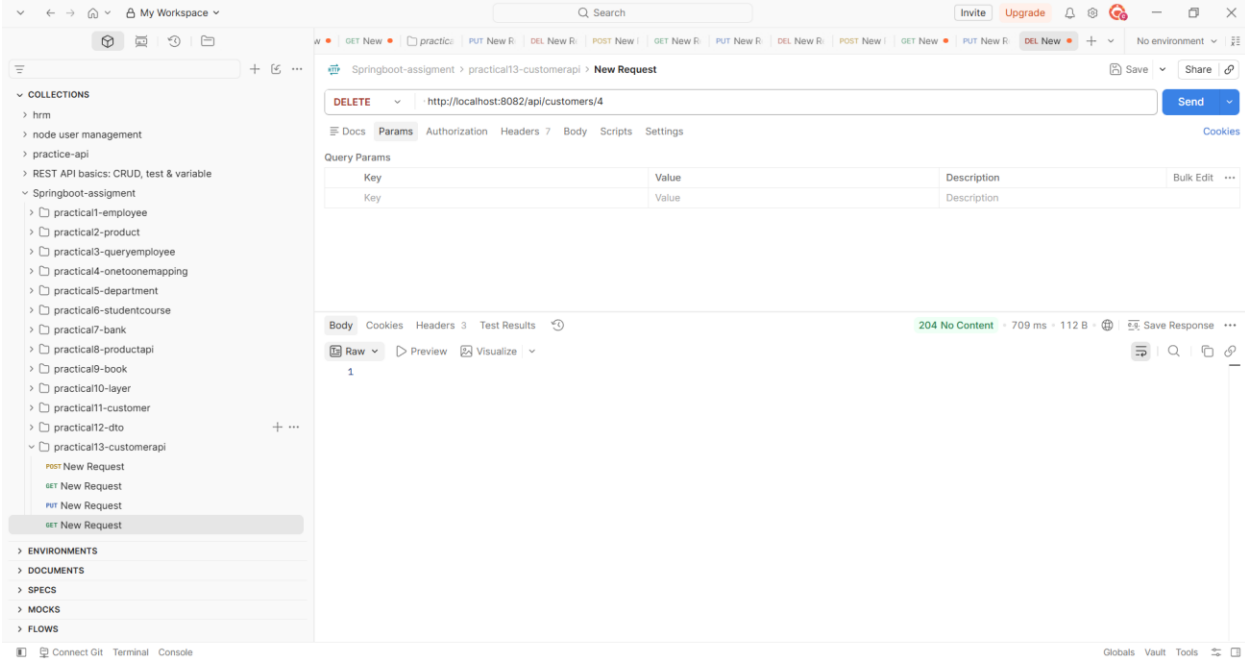
The screenshot shows the Postman interface with a workspace named 'My Workspace'. On the left, a 'COLLECTIONS' sidebar lists various API collections, including 'Springboot-assignment' and 'practical13-customerapi'. The main panel displays a 'New Request' for the PUT method at the URL 'http://localhost:8082/api/customers/4'. The 'Body' tab is selected, showing a JSON request:

```
1 {
2   "name": "Sneha",
3   "email": "not-an-email",
4   "age": 21
5 }
```

The status bar at the bottom indicates a '400 Bad Request' response with a 13 ms duration and 245 B of data. The 'Body' tab shows the error response:

```
1 {
2   "timestamp": "2026-08-11T17:11:37.282Z",
3   "status": 400,
4   "error": "Bad Request",
5   "path": "/api/customers/4"
6 }
```

Delete Method



Method	Endpoint (Postman)	Request Body / Params	Expected Response
POST	http://localhost:8082/api/customers	{"name":"","email":"bad-email","age":15}	400 Bad Request — validation failure because name is blank, email is invalid and age is below 18
POST	http://localhost:8082/api/customers	{"name":"Sneha","email":"sneha@svgu.edu","age":21}	201 Created — valid Customer saved

			successfully
GET	http://localhost:8082/api/customers/1	None	200 OK — saved Customer returned
PUT	http://localhost:8082/api/customers/1	{"name":"Sneha","email":"not-an-email","age":21}	400 Bad Request — @Valid rejects malformed email
DELETE	http://localhost:8082/api/customers/1	None	204 No Content — Customer deleted successfully

Practical No. 14: Global Exception Handling using @ControllerAdvice and @ExceptionHandler

Definition / Objective

@ExceptionHandler

Textbook Reference: Chapter 6 — Building RESTful Microservices

Objective:

To centralize exception handling across all REST controllers using @ControllerAdvice, returning a consistent,

structured error response (timestamp, message, status, path) instead of default Spring stack-trace pages — the

concluding topic of this chapter.

Enroll – 25CI2110039 | Name – Kalariya Heli

Class Name(s) with Attributes:

ResourceNotFoundException (custom RuntimeException)

ErrorResponse (POJO)

- timestamp : LocalDateTime

- message : String

- status : int

- path : String

GlobalExceptionHandler (@ControllerAdvice)

- Method: handleNotFound(ResourceNotFoundException)

- Method: handleValidation(MethodArgumentNotValidException)

- Method: handleGenericException(Exception)

Step-by-Step Task Instructions:

1. Create ResourceNotFoundException extending RuntimeException.
2. Create the ErrorResponse POJO with the listed fields.
3. In the service layer (e.g., EmployeeService or ProductService), throw ResourceNotFoundException when findById() returns empty instead of returning null.
4. Create GlobalExceptionHandler annotated @ControllerAdvice with an @ExceptionHandler(ResourceNotFoundException.class) method returning ResponseEntity<ErrorResponse> with HTTP 404.
5. Add an @ExceptionHandler(MethodArgumentNotValidException.class) method that collects field-wise validation errors from Practical 13 into a structured response with HTTP 400.
6. Add a catch-all @ExceptionHandler(Exception.class) method returning HTTP 500 for any unhandled exception, and re-test the invalid Customer payload and a non-existent Employee id.
7. Make sure PUT /api/employees/{id} and DELETE /api/employees/{id} also route through ResourceNotFoundException (not a raw NullPointerException) when the id does not exist, and re-test all four HTTP verbs against a non-existent id.

Code

CustomerRestController.java

```
package com.example.exceptionapi.controller;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import com.example.exceptionapi.entity.Customer;
import com.example.exceptionapi.repository.CustomerRepository;
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
import jakarta.validation.Valid;

@RestController
@RequestMapping("/api/customers")
public class CustomerRestController {

    private final CustomerRepository customerRepository;

    public CustomerRestController(
        CustomerRepository customerRepository) {

        this.customerRepository = customerRepository;
    }

    @PostMapping
    public ResponseEntity<Customer> createCustomer(
        @Valid @RequestBody Customer customer) {

        return ResponseEntity
            .status(HttpStatus.CREATED)
            .body(customerRepository.save(customer));
    }
}
```

EmployeeRestController.java

```
package com.example.exceptionapi.controller;

import java.util.List;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import com.example.exceptionapi.entity.Employee;
import com.example.exceptionapi.service.EmployeeService;

@RestController
@RequestMapping("/api/employees")
public class EmployeeRestController {

    private final EmployeeService employeeService;

    public EmployeeRestController(
        EmployeeService employeeService) {

        this.employeeService = employeeService;
    }

    @PostMapping
    public ResponseEntity<Employee> createEmployee(
        @RequestBody Employee employee) {

        return ResponseEntity
            .status(HttpStatus.CREATED)
            .body(employeeService.createEmployee(employee));
    }

    @GetMapping
    public ResponseEntity<List<Employee>> getAllEmployees() {

        return ResponseEntity.ok(
            employeeService.getAllEmployees());
    }

    @GetMapping("/{id}")
    public ResponseEntity<Employee> getEmployeeById(
        @PathVariable int id) {
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
        return ResponseEntity.ok(
            employeeService.getEmployeeById(id));
    }

    @PutMapping("/{id}")
    public ResponseEntity<Employee> updateEmployee(
        @PathVariable int id,
        @RequestBody Employee details) {

        return ResponseEntity.ok(
            employeeService.updateEmployee(id, details));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteEmployee(
        @PathVariable int id) {

        employeeService.deleteEmployee(id);

        return ResponseEntity.noContent().build();
    }
}
```

Customer.java

```
package com.example.exceptionapi.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;

@Entity
public class Customer {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int customerId;

    @NotBlank
    private String name;

    @Email
    private String email;

    @Min(18)
    private int age;

    public Customer() {
        super();
        // TODO Auto-generated constructor stub
    }

    public int getCustomerId() {
        return customerId;
    }

    public void setCustomerId(int customerId) {
        this.customerId = customerId;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
```

```
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }
}
```

Employee.java

```
package com.example.exceptionapi.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int empId;

    private String empName;
    private String department;
    private double salary;
    public Employee() {
        super();
        // TODO Auto-generated constructor stub
    }
    public int getEmpId() {
        return empId;
    }
    public void setEmpId(int empId) {
        this.empId = empId;
    }
    public String getEmpName() {
        return empName;
    }
    public void setEmpName(String empName) {
        this.empName = empName;
    }
    public String getDepartment() {
        return department;
    }
    public void setDepartment(String department) {
        this.department = department;
    }
    public double getSalary() {
        return salary;
    }
    public void setSalary(double salary) {
        this.salary = salary;
    }
}
```

```
}
```

ErrorResponse.java

```
package com.example.exceptionapi.exception;

import java.time.LocalDateTime;

public class ErrorResponse {

    private LocalDateTime timestamp;
    private String message;
    private int status;
    private String path;

    public ErrorResponse() {
    }

    public ErrorResponse(
        LocalDateTime timestamp,
        String message,
        int status,
        String path) {

        this.timestamp = timestamp;
        this.message = message;
        this.status = status;
        this.path = path;
    }

    public LocalDateTime getTimestamp() {
        return timestamp;
    }

    public void setTimestamp(LocalDateTime timestamp) {
        this.timestamp = timestamp;
    }

    public String getMessage() {
        return message;
    }

    public void setMessage(String message) {
        this.message = message;
    }

    public int getStatus() {
        return status;
    }

    public void setStatus(int status) {
        this.status = status;
    }

    public String getPath() {
        return path;
    }

    public void setPath(String path) {
        this.path = path;
    }
}
```

GlobalExceptionHandler.java

```
package com.example.exceptionapi.exception;

import java.time.LocalDateTime;
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
import java.util.HashMap;
import java.util.Map;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ExceptionHandler;

import jakarta.servlet.http.HttpServletRequest;

@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(ResourceNotFoundException ex, HttpServletRequest request) {

        ErrorResponse error = new ErrorResponse(LocalDateTime.now(), ex.getMessage(),
        HttpStatus.NOT_FOUND.value(),
            request.getRequestURI());

        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, Object>> handleValidation(MethodArgumentNotValidException ex,
        HttpServletRequest request) {

        Map<String, String> fieldErrors = new HashMap<>();

        ex.getBindingResult().getFieldErrors()
            .forEach(error -> fieldErrors.put(error.getField(), error.getDefaultMessage()));

        Map<String, Object> response = new HashMap<>();

        response.put("timestamp", LocalDateTime.now());

        response.put("status", HttpStatus.BAD_REQUEST.value());

        response.put("message", "Validation failed");

        response.put("path", request.getRequestURI());

        response.put("errors", fieldErrors);

        return ResponseEntity.badRequest().body(response);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGenericException(Exception ex, HttpServletRequest request)
    {

        ErrorResponse error = new ErrorResponse(LocalDateTime.now(), ex.getMessage(),
        HttpStatus.INTERNAL_SERVER_ERROR.value(), request.getRequestURI());

        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(error);
    }
}
```

ResourceNotFoundException.java

```
package com.example.exceptionapi.exception;

public class ResourceNotFoundException
    extends RuntimeException {

    public ResourceNotFoundException(String message) {
        super(message);
    }
}
```

```
}  
}
```

P14Application.java

```
package com.example.exceptionapi;  
  
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
  
@SpringBootApplication  
public class P14Application {  
  
    public static void main(String[] args) {  
        SpringApplication.run(P14Application.class, args);  
    }  
  
}
```

CustomerRepository.java

```
package com.example.exceptionapi.repository;  
  
import org.springframework.data.jpa.repository.JpaRepository;  
  
import com.example.exceptionapi.entity.Customer;  
  
public interface CustomerRepository  
    extends JpaRepository<Customer, Integer> {  
  
}
```

EmployeeRepository.java

```
package com.example.exceptionapi.repository;  
  
import org.springframework.data.jpa.repository.JpaRepository;  
  
import com.example.exceptionapi.entity.Employee;  
  
public interface EmployeeRepository  
    extends JpaRepository<Employee, Integer> {  
  
}
```

EmployeeService.java

```
package com.example.exceptionapi.service;  
  
import java.util.List;  
  
import org.springframework.stereotype.Service;  
  
import com.example.exceptionapi.entity.Employee;  
import com.example.exceptionapi.exception.ResourceNotFoundException;  
import com.example.exceptionapi.repository.EmployeeRepository;  
  
@Service  
public class EmployeeService {  
  
    private final EmployeeRepository employeeRepository;  
  
    public EmployeeService(  
        EmployeeRepository employeeRepository) {  
  
        this.employeeRepository = employeeRepository;  
    }  
  
    public Employee createEmployee(Employee employee) {  
  
        return employeeRepository.save(employee);  
    }  
  
}
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
}

public List<Employee> getAllEmployees() {
    return employeeRepository.findAll();
}

public Employee getEmployeeById(int id) {
    return employeeRepository.findById(id)
        .orElseThrow(() ->
            new ResourceNotFoundException(
                "Employee not found with id: " + id));
}

public Employee updateEmployee(
    int id,
    Employee details) {

    Employee employee =
        employeeRepository.findById(id)
            .orElseThrow(() ->
                new ResourceNotFoundException(
                    "Employee not found with id: " + id));

    if (details.getEmpName() != null) {
        employee.setEmpName(details.getEmpName());
    }

    if (details.getDepartment() != null) {
        employee.setDepartment(details.getDepartment());
    }

    if (details.getSalary() > 0) {
        employee.setSalary(details.getSalary());
    }

    return employeeRepository.save(employee);
}

public void deleteEmployee(int id) {

    Employee employee =
        employeeRepository.findById(id)
            .orElseThrow(() ->
                new ResourceNotFoundException(
                    "Employee not found with id: " + id));

    employeeRepository.delete(employee);
}
}
```

P14ApplicationTests.java

```
package com.example.exceptionapi;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

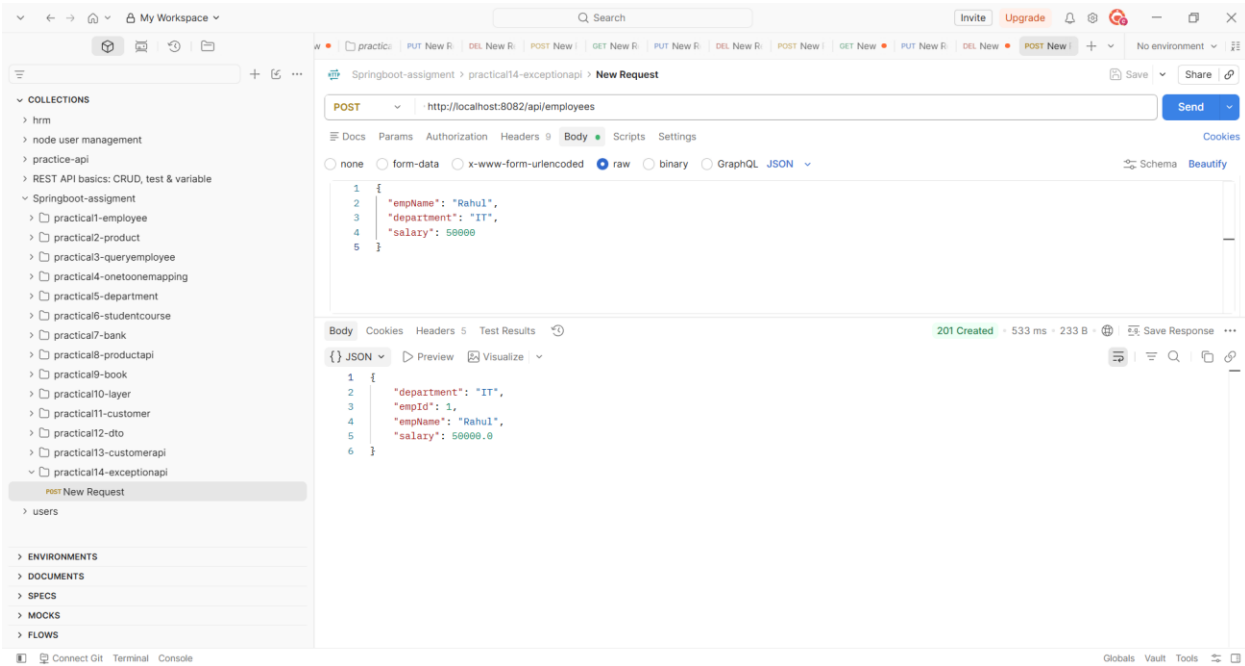
@SpringBootTest
class P14ApplicationTests {

    @Test
    void contextLoads() {
    }

}
```

Enroll – 25CI2110039 | Name – Kalariya Heli

Post Method



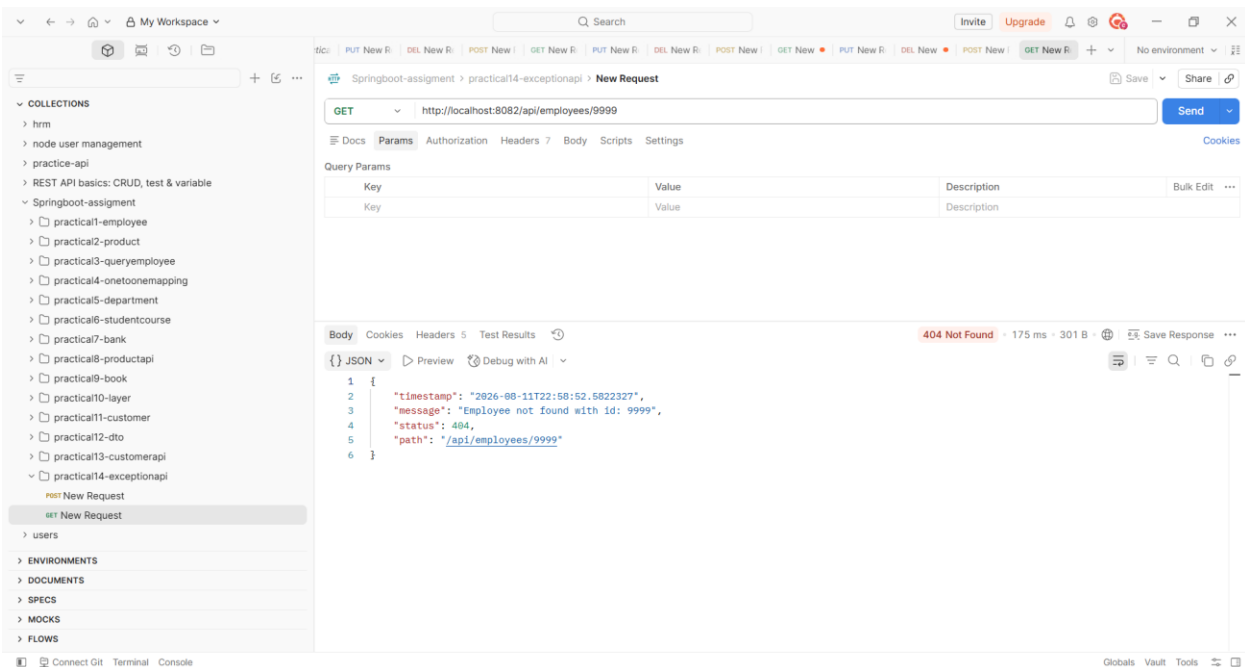
The screenshot shows the Postman interface with a new request for the POST method at the URL `http://localhost:8082/api/employees`. The request body is a JSON object:

```
1 {
2   "empName": "Rahul",
3   "department": "IT",
4   "salary": 50000
5 }
```

The response is a 201 Created status with a JSON object:

```
1 {
2   "department": "IT",
3   "empId": 1,
4   "empName": "Rahul",
5   "salary": 50000.0
6 }
```

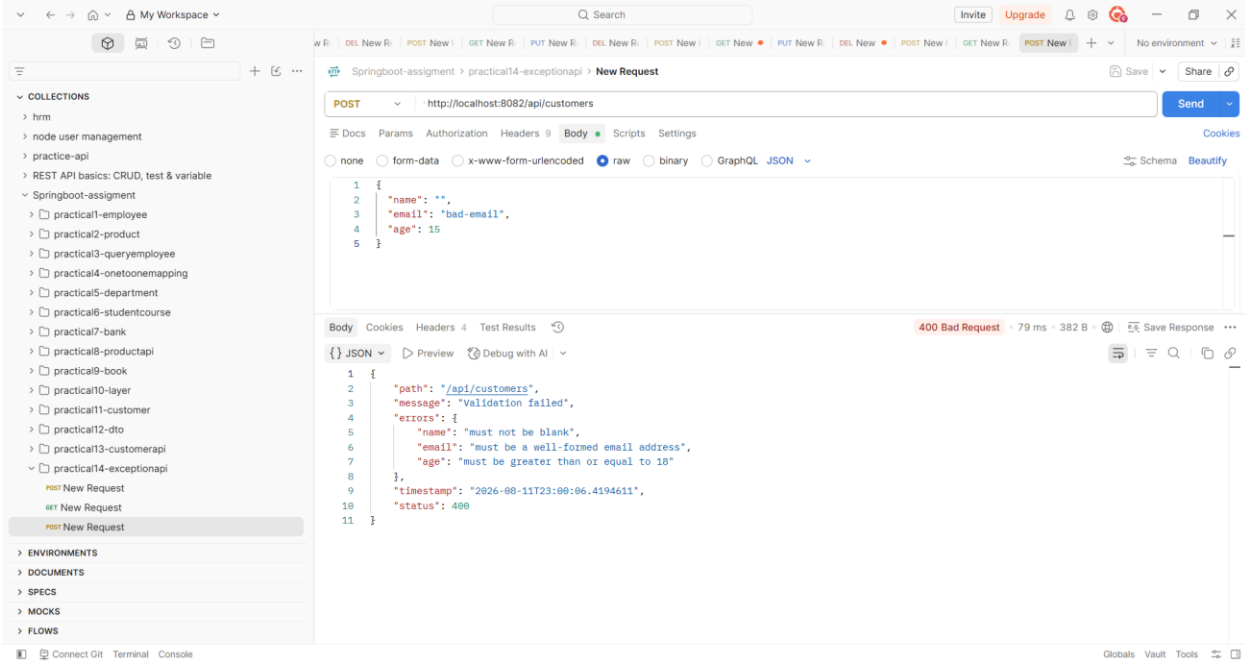
Get Method - Test 404 Global Exception



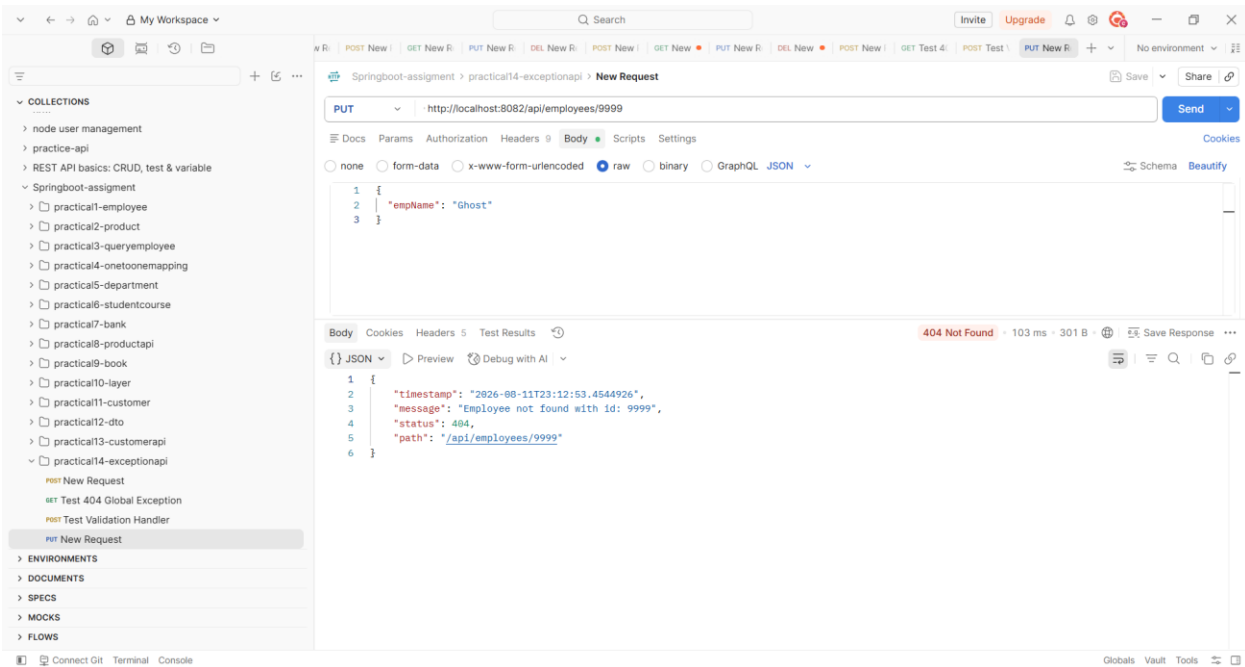
The screenshot shows the Postman interface with a new request for the GET method at the URL `http://localhost:8082/api/employees/9999`. The response is a 404 Not Found status with a JSON object:

```
1 {
2   "timestamp": "2026-08-11T22:58:52.582327",
3   "message": "Employee not found with id: 9999",
4   "status": 404,
5   "path": "/api/employees/9999"
6 }
```

Post Method - Test Validation Handler



Put Method - Test with Non-existent ID



Met hod	Endpoint (Postman)	Request Body / Params	Expected Response
------------	--------------------	-----------------------	----------------------

POST	http://localhost:8082/api/employees	{"empName": "Rahul", "department": "IT", "salary": 50000}	201 Created — Employee created normally
GET	http://localhost:8082/api/employees/9999	None	404 Not Found — Structured JSON containing timestamp, message, status and path through GlobalExceptionHandler
POST	http://localhost:8082/api/customers	{"name": "", "email": "bad-email", "age": 15}	400 Bad Request — Structured field-level validation errors through @ControllerAdvice
PUT	http://localhost:8082/api/employees/9999	{"empName": "Ghost"}	404 Not Found — Structured error returned instead of a stack trace
DELETE	http://localhost:8082/api/employees/9999	None	404 Not Found — Structured error through GlobalExceptionHandler

Practical No. 15: Securing REST API with HTTP Basic Authentication (Spring Security)

Definition / Objective

(Spring Security)

Enroll – 25CI2110039 | Name – Kalariya Heli

Textbook Reference: Chapter 7 — Securing a Web Application

Objective:

To add Spring Security to a Spring Boot REST application, configure in-memory users with encoded passwords,

and secure selected endpoints so that only authenticated requests can access them — the introductory security

mechanism covered in this chapter.

Class Name(s) with Attributes:

SecurityConfig (@Configuration)

- Bean: SecurityFilterChain filterChain()

- Bean: PasswordEncoder passwordEncoder() — BCryptPasswordEncoder

- Bean: InMemoryUserDetailsManager userDetailsService()

Step-by-Step Task Instructions:

1. Add spring-boot-starter-security dependency to pom.xml.
2. Create SecurityConfig with a PasswordEncoder bean using BCryptPasswordEncoder.
3. Define a UserDetailsService bean with at least one in-memory user (username 'user', BCrypt-encoded password, role USER).
4. Define a SecurityFilterChain bean that permits access to /api/public/** without authentication and requires authentication for /api/secure/**, enabling httpBasic().
5. Create two endpoints: GET /api/public/data (unsecured) and GET /api/secure/data (secured).
6. Add POST /api/secure/data, PUT /api/secure/data/{id} and DELETE /api/secure/data/{id} to a simple secured resource (e.g., an in-memory Note list) so that every CRUD verb — not just GET — is protected by the same

SecurityFilterChain rule.

7. Test /api/secure/data without credentials (expect 401) and then with correct Basic Auth credentials configured

in Postman's Authorization tab (expect 200), then repeat for POST/PUT/DELETE.

Code

SecurityConfig.java

```
package com.example.securityapi.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
@Configuration
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {

        return new BCryptPasswordEncoder();
    }

    @Bean
    public UserDetailsService userDetailsService(PasswordEncoder passwordEncoder) {

        UserDetails user =
        User.withUsername("user").password(passwordEncoder.encode("user123")).roles("USER").build();

        return new InMemoryUserDetailsManager(user);
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {

        http.csrf(csrf -> csrf.disable())

            .authorizeHttpRequests(auth -> auth

                .requestMatchers("/api/public/**").permitAll()

                .requestMatchers("/api/secure/**").authenticated()

                .anyRequest().authenticated())

            .httpBasic(basic -> {
            });

        return http.build();
    }
}
```

PublicController.java

```
package com.example.securityapi.controller;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/public")
public class PublicController {

    @GetMapping("/data")
    public ResponseEntity<String> getPublicData() {

        return ResponseEntity.ok(
            "This is public data");
    }
}
```

SecureDataController.java

```
package com.example.securityapi.controller;

import java.util.ArrayList;
import java.util.List;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.example.securityapi.model.Note;

@RestController
@RequestMapping("/api/secure")
public class SecureDataController {

    private final List<Note> notes =
        new ArrayList<>();

    private int nextId = 1;

    @GetMapping("/data")
    public ResponseEntity<List<Note>> getSecureData() {

        return ResponseEntity.ok(notes);
    }

    @PostMapping("/data")
    public ResponseEntity<Note> createNote(
        @RequestBody Note note) {

        note.setId(nextId++);

        notes.add(note);

        return ResponseEntity
            .status(HttpStatus.CREATED)
            .body(note);
    }

    @PutMapping("/data/{id}")
    public ResponseEntity<Note> updateNote(
        @PathVariable int id,
        @RequestBody Note details) {

        for (Note note : notes) {

            if (note.getId() == id) {

                note.setText(details.getText());

                return ResponseEntity.ok(note);
            }
        }

        return ResponseEntity.notFound().build();
    }

    @DeleteMapping("/data/{id}")
    public ResponseEntity<Void> deleteNote(
        @PathVariable int id) {

        Note noteToDelete = null;

        for (Note note : notes) {

            if (note.getId() == id) {

                noteToDelete = note;
                break;
            }
        }
    }
}
```

```
    }

    if (noteToDelete == null) {
        return ResponseEntity.notFound().build();
    }

    notes.remove(noteToDelete);

    return ResponseEntity.noContent().build();
}
}
```

Note.java

```
package com.example.securityapi.model;

public class Note {

    private int id;
    private String text;
    public Note() {
        super();
        // TODO Auto-generated constructor stub
    }
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getText() {
        return text;
    }
    public void setText(String text) {
        this.text = text;
    }
}

}
```

P15Application.java

```
package com.example.securityapi;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class P15Application {

    public static void main(String[] args) {
        SpringApplication.run(P15Application.class, args);
    }
}

}
```

P15ApplicationTests.java

```
package com.example.securityapi;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class P15ApplicationTests {

    @Test
    void contextLoads() {
    }

}
```

Enroll – 25CI2110039 | Name – Kalariya Heli

}

Get Method

The screenshot shows the Postman interface with a new request created. The request is a GET method to the URL `http://localhost:8082/api/public/data`. The response is a 200 OK status with a response time of 135 ms and a body of `1 This is public data`.

Key	Value	Description
Key	Value	Description

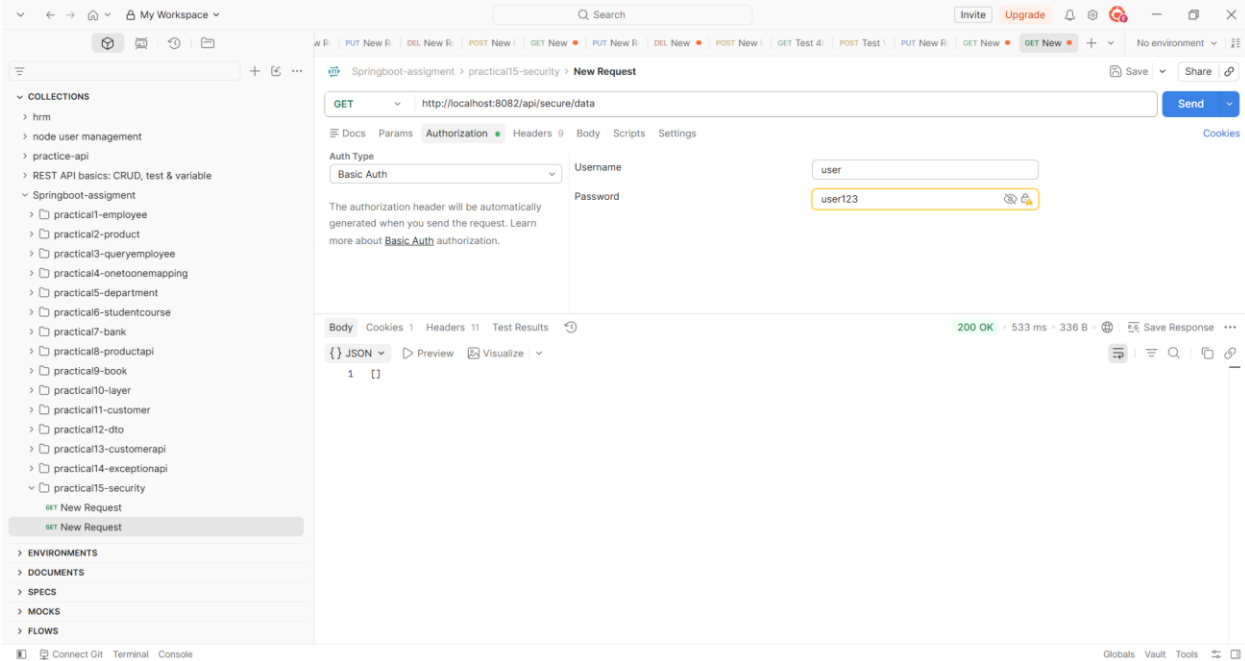
Get Method - Test Secure GET WITHOUT Login

The screenshot shows the Postman interface with a new request created. The request is a GET method to the URL `http://localhost:8082/api/secure/data`. The response is a 401 Unauthorized status with a response time of 141 ms and a body of `1`. The error message indicates that the correct authentication credentials must be passed.

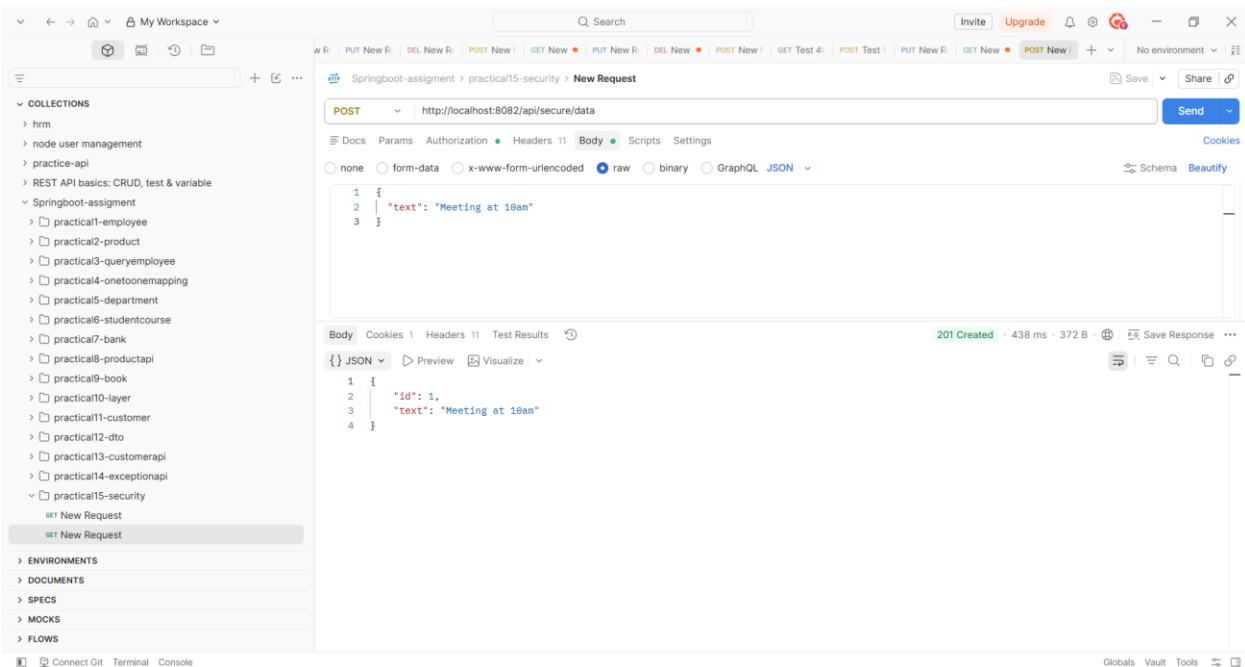
Key	Value	Description
Key	Value	Description

Get Method - Add Basic Authentication

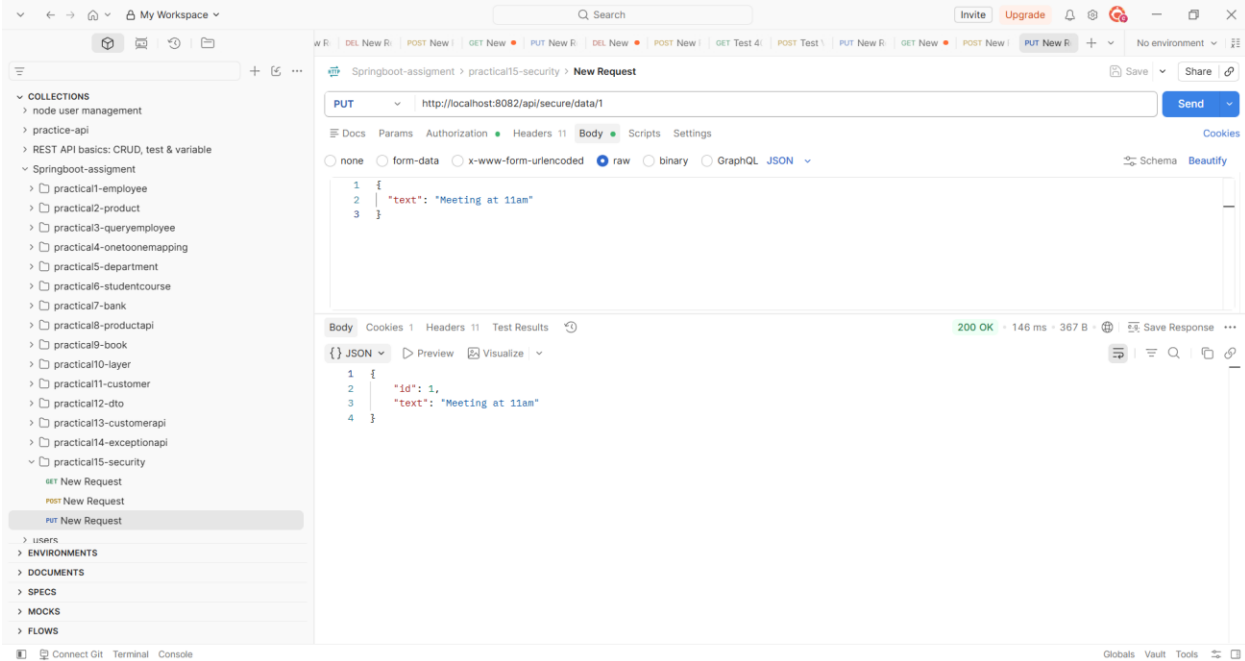
Enroll – 25CI2110039 | Name – Kalariya Heli



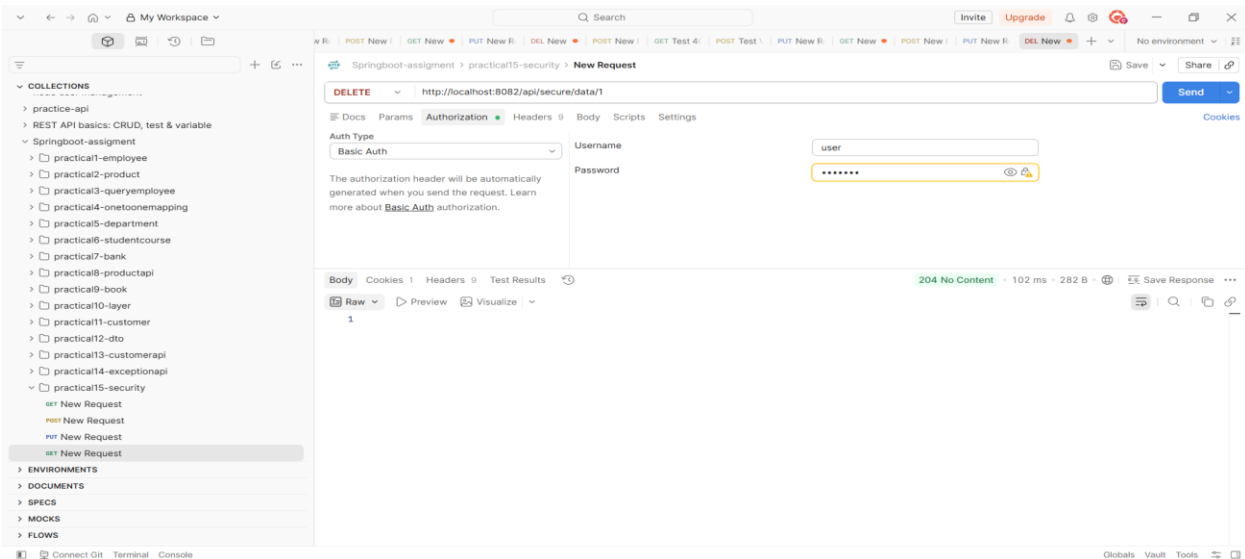
Post Method - POST Secured Note



Put Method – secure endpoint



Delete Method



Method	Endpoint (Postman)	Request Body / Authentication	Expected Response
GET	http://localhost:8082/api/public/data	No Authentication	200 OK — Public data accessible without login
GET	http://localhost:8082/api/secure/data	No Authentication	401 Unauthorized

			— Secured endpoint cannot be accessed without credentials
GET	http://localhost:8082/api/secure/data	Basic Auth: username = user, password = user123	200 OK — Secured data returned
POST	http://localhost:8082/api/secure/data	Basic Auth + {"text": "Meeting at 10am"}	201 Created — Secured Note created only with valid Basic Auth
PUT	http://localhost:8082/api/secure/data/1	Basic Auth + {"text": "Meeting at 11am"}	200 OK — Secured Note updated only with valid authentication
DELETE	http://localhost:8082/api/secure/data/1	Basic Auth: user / user123	204 No Content — Secured Note deleted successfully
GET	http://localhost:8082/api/secure/data	Basic Auth with wrong password	401 Unauthorized — Invalid credentials rejected

Practical No. 16: User Registration and Login with BCrypt Password Encoding

Definition / Objective

Encoding

Textbook Reference: Chapter 7 — Securing a Web Application

Objective:

To implement a database-backed user registration and login mechanism, storing passwords securely using BCryptPasswordEncoder instead of plain text, and authenticating against the database via a custom UserDetailsService — extending the password-encoding concept from this chapter to a persistent user store.

Class Name(s) with Attributes:

AppUser (@Entity)

- userId : int (@Id @GeneratedValue)

- username : String (unique)

- password : String (BCrypt-encoded)

- role : String

AppUserRepository (extends JpaRepository<AppUser, Integer>)

- Optional<AppUser> findByUsername(String username)

CustomUserDetailsService (implements UserDetailsService)

- appUserRepository : AppUserRepository

AuthController (@RestController)

- Method: register(@RequestBody AppUser) — encodes password before saving

Step-by-Step Task Instructions:

1. Create the AppUser entity and AppUserRepository with a findByUsername() derived query method.
2. Create CustomUserDetailsService implementing UserDetailsService, loading a user by username and returning a Spring Security User object built from the stored (encoded) password and role.
3. Update SecurityConfig to use CustomUserDetailsService instead of the in-memory user store from Practical 15, with an AuthenticationProvider wired to the BCryptPasswordEncoder bean.
4. Create AuthController with POST /api/auth/register that encodes the incoming plain-text password using passwordEncoder.encode() before saving the AppUser.
5. Register a new user via Postman, then inspect the database to confirm the stored password is a BCrypt hash, not plain text.
6. Add PUT /api/auth/profile (secured, Basic Auth) to let the authenticated user update their own username/role, and DELETE /api/auth/deregister (secured) to remove the authenticated user's account.
7. Test HTTP Basic Auth login against a secured endpoint using the newly registered username and original (plain-text) password, then test the profile update and deregistration.

Code

SecurityConfig.java

```
package com.example.authapi.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationProvider;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

import com.example.authapi.service.CustomUserDetailsService;

@Configuration
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {

        return new BCryptPasswordEncoder();
    }

    @Bean
    public AuthenticationProvider authenticationProvider(
        CustomUserDetailsService userDetailsService,
        PasswordEncoder passwordEncoder) {

        DaoAuthenticationProvider provider =
            new DaoAuthenticationProvider(userDetailsService);

        provider.setPasswordEncoder(passwordEncoder);

        return provider;
    }

    @Bean
    public SecurityFilterChain filterChain(
        HttpSecurity http,
        AuthenticationProvider authenticationProvider)
        throws Exception {

        http
            .csrf(csrf -> csrf.disable())

            .authenticationProvider(authenticationProvider)

            .authorizeHttpRequests(auth -> auth

                .requestMatchers("/api/auth/register")
                .permitAll()

                .requestMatchers(
                    "/api/auth/profile",
                    "/api/auth/deregister",
                    "/api/secure/**")
                .authenticated()

                .anyRequest()
                .authenticated()
            )

            .httpBasic(basic -> {
            });

        return http.build();
    }
}
```

AuthController.java

```
package com.example.authapi.controller;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.example.authapi.entity.AppUser;
import com.example.authapi.repository.AppUserRepository;

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private final AppUserRepository appUserRepository;
    private final PasswordEncoder passwordEncoder;

    public AuthController(
        AppUserRepository appUserRepository,
        PasswordEncoder passwordEncoder) {

        this.appUserRepository = appUserRepository;
        this.passwordEncoder = passwordEncoder;
    }

    @PostMapping("/register")
    public ResponseEntity<String> register(
        @RequestBody AppUser appUser) {

        if (appUserRepository
            .findByUsername(appUser.getUsername())
            .isPresent()) {

            return ResponseEntity
                .badRequest()
                .body("Username already exists");
        }

        appUser.setPassword(
            passwordEncoder.encode(
                appUser.getPassword()));

        appUserRepository.save(appUser);

        return ResponseEntity
            .status(HttpStatus.CREATED)
            .body("User registered successfully");
    }

    @PutMapping("/profile")
    public ResponseEntity<AppUser> updateProfile(
        Authentication authentication,
        @RequestBody AppUser details) {

        AppUser user =
            appUserRepository
                .findByUsername(authentication.getName())
                .orElse(null);

        if (user == null) {
            return ResponseEntity.notFound().build();
        }
    }
}
```

```
    }

    if (details.getUsername() != null) {
        user.setUsername(details.getUsername());
    }

    if (details.getRole() != null) {
        user.setRole(details.getRole());
    }

    AppUser updated =
        appUserRepository.save(user);

    updated.setPassword(null);

    return ResponseEntity.ok(updated);
}

@DeleteMapping("/deregister")
public ResponseEntity<Void> deregister(
    Authentication authentication) {

    AppUser user =
        appUserRepository
            .findByUsername(authentication.getName())
            .orElse(null);

    if (user == null) {
        return ResponseEntity.notFound().build();
    }

    appUserRepository.delete(user);

    return ResponseEntity.noContent().build();
}
}
```

SecureDataController.java

```
package com.example.authapi.controller;

import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/secure")
public class SecureDataController {

    @GetMapping("/data")
    public ResponseEntity<String> getSecureData(
        Authentication authentication) {

        return ResponseEntity.ok(
            "Welcome " + authentication.getName()
            + ", you are authenticated");
    }
}
```

AppUser.java

```
package com.example.authapi.entity;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
```

```
@Entity
public class AppUser {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int userId;

    @Column(unique = true)
    private String username;

    private String password;

    private String role;

    public AppUser() {
        super();
        // TODO Auto-generated constructor stub
    }

    public int getUserId() {
        return userId;
    }

    public void setUserId(int userId) {
        this.userId = userId;
    }

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public String getRole() {
        return role;
    }

    public void setRole(String role) {
        this.role = role;
    }

}
```

P16Application.java

```
package com.example.authapi;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class P16Application {

    public static void main(String[] args) {
        SpringApplication.run(P16Application.class, args);
    }

}
```

```
}
```

AppUserRepository.java

```
package com.example.authapi.repository;

import java.util.Optional;

import org.springframework.data.jpa.repository.JpaRepository;

import com.example.authapi.entity.AppUser;

public interface AppUserRepository
    extends JpaRepository<AppUser, Integer> {

    Optional<AppUser> findByUsername(String username);
}
```

CustomUserDetailsService.java

```
package com.example.authapi.service;

import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

import com.example.authapi.entity.AppUser;
import com.example.authapi.repository.AppUserRepository;

@Service
public class CustomUserDetailsService
    implements UserDetailsService {

    private final AppUserRepository appUserRepository;

    public CustomUserDetailsService(
        AppUserRepository appUserRepository) {

        this.appUserRepository = appUserRepository;
    }

    @Override
    public UserDetails loadUserByUsername(String username)
        throws UsernameNotFoundException {

        AppUser appUser =
            appUserRepository.findByUsername(username)
                .orElseThrow(() ->
                    new UsernameNotFoundException(
                        "User not found: " + username));

        return User.withUsername(appUser.getUsername())
            .password(appUser.getPassword())
            .roles(appUser.getRole())
            .build();
    }
}
```

P16ApplicationTests.java

```
package com.example.authapi;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

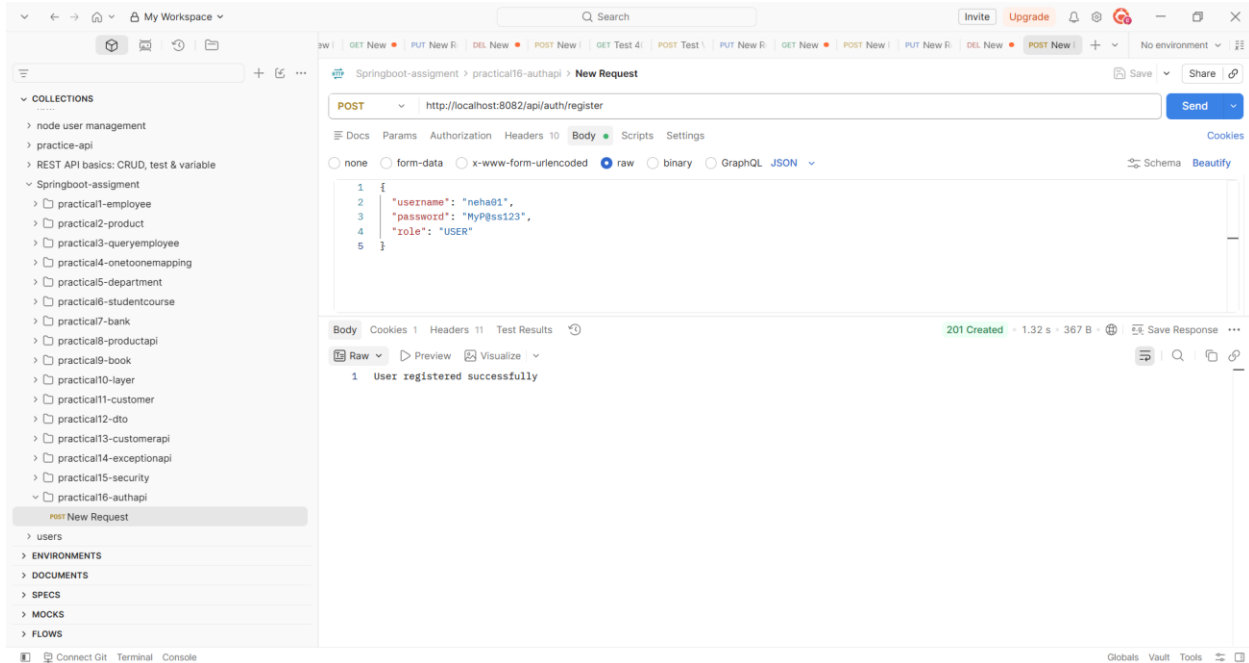
@SpringBootTest
class P16ApplicationTests {
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
@Test
void contextLoads() {
}

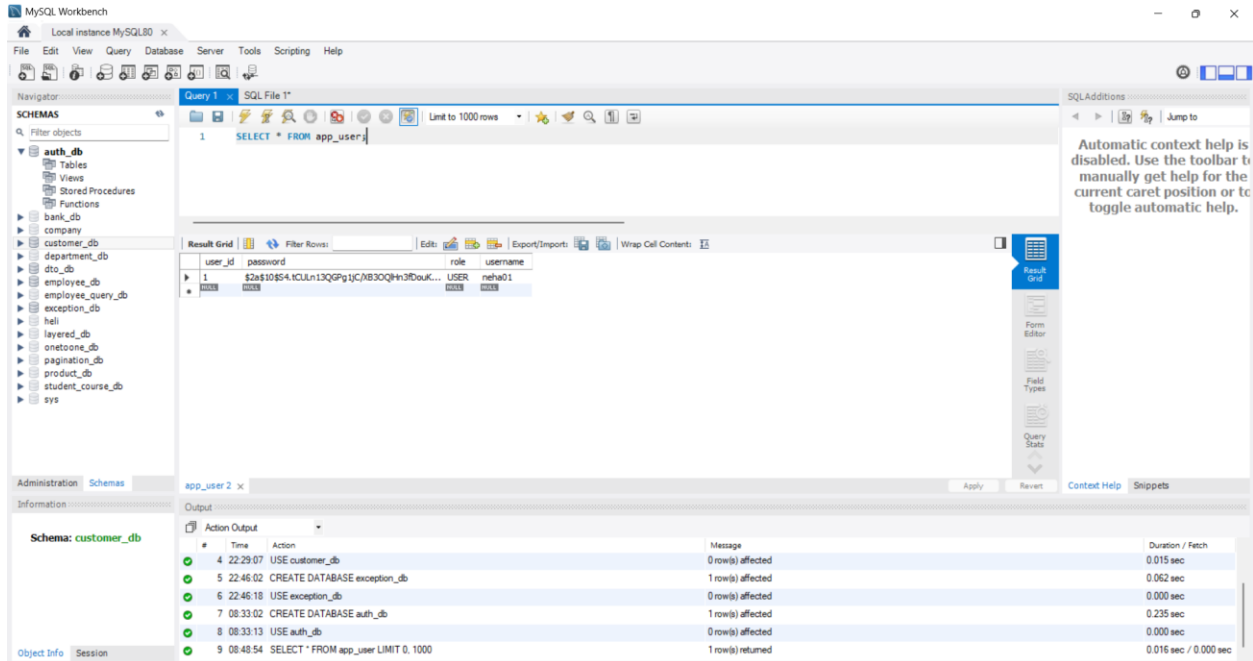
}
```

Post Method

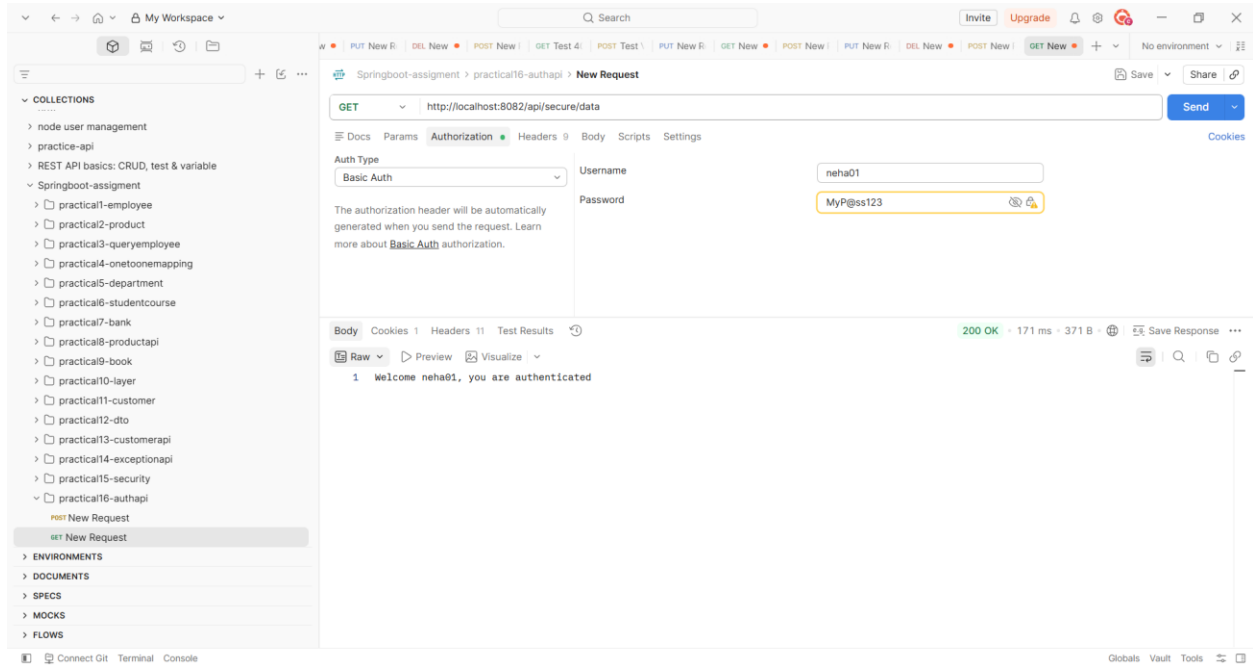


Verify BCrypt Password in MySQL

Enroll – 25CI2110039 | Name – Kalariya Heli

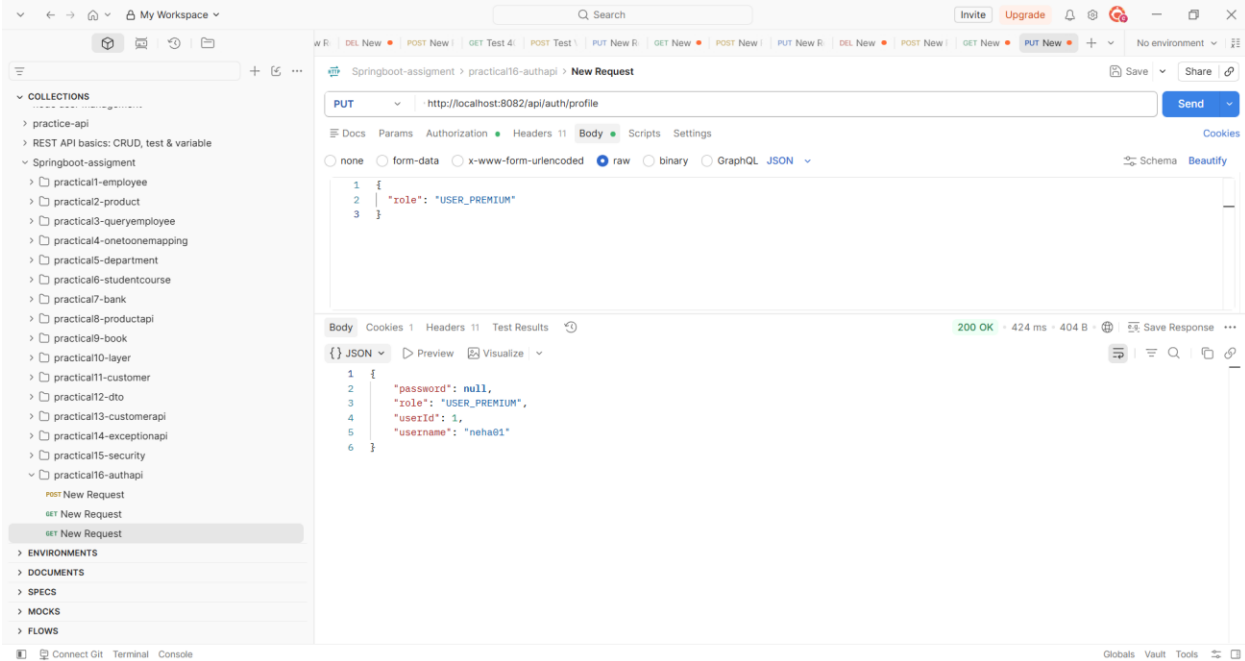


Get Method - Login / Test Secure Endpoint

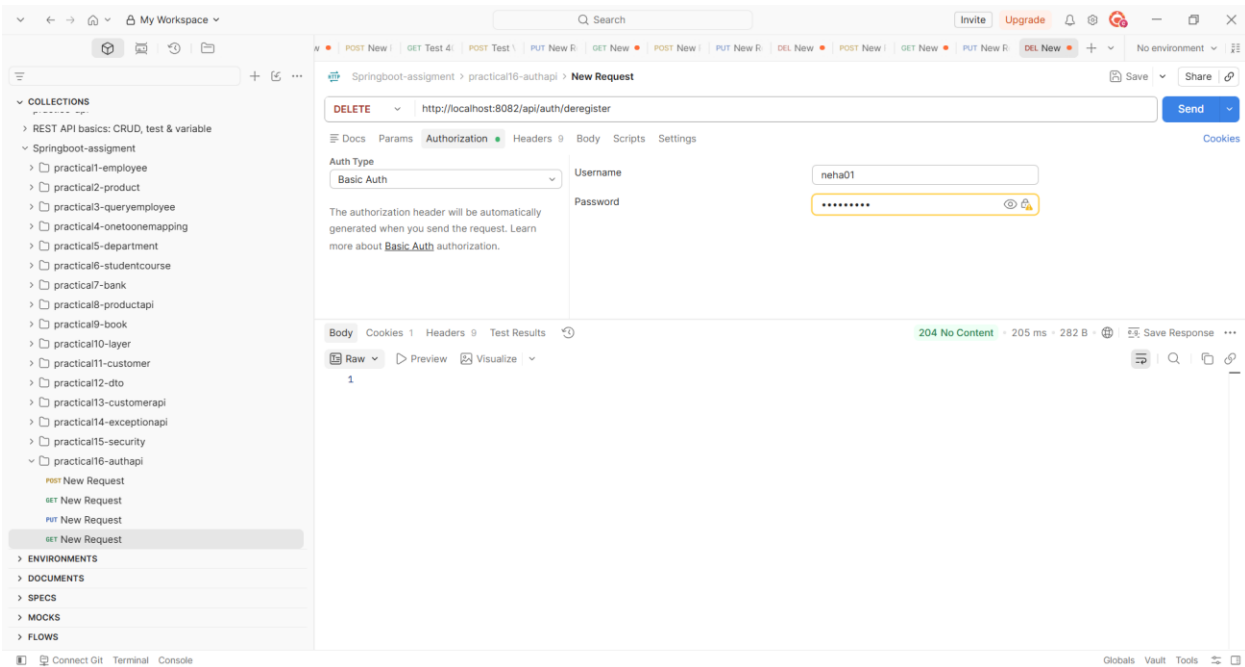


Put Method

Enroll – 25CI2110039 | Name – Kalariya Heli



Delete Method



Method	Endpoint (Postman)	Request Body / Authentication	Expected
--------	--------------------	-------------------------------	----------

			Response
POST	http://localhost:8082/api/auth/register	No Auth + { "username": "neha01", "password": "MyP@ss123", "role": "USER" }	201 Created — User saved in MySQL with BCrypt-hashed password
GET	http://localhost:8082/api/secure/data	Basic Auth: neha01 / MyP@ss123	200 OK — User authenticated against BCrypt-hashed database password
GET	http://localhost:8082/api/secure/data	Basic Auth: neha01 / wrong123	401 Unauthorized — Incorrect password rejected
PUT	http://localhost:8082/api/auth/profile	Basic Auth: neha01 / MyP@ss123 + { "role": "USER_PREMIUM" }	200 OK — Role updated for authentication

			cated user while passwor d remains unchang ed
DELE TE	http://localhost:8082/api/auth/deregister	Basic Auth: neha01 / MyP@ss123	204 No Content — Authent icated user's account remove d from databas e

Practical No. 17: Role-Based Access Control (RBAC) with Spring Security

Objective:

To extend Spring Security configuration for role-based authorization, defining users with different roles (ADMIN, USER) and restricting specific endpoints using `@PreAuthorize` or role-based matchers, as covered in this chapter's authorization section.

Class Name(s) with Attributes: `SecurityConfig` (extended)

- Users carrying `ROLE_ADMIN` / `ROLE_USER`
- `@EnableMethodSecurity` enabled `AdminController` (`@RestController`)
- Method: `getAdminData()` — `@PreAuthorize("hasRole('ADMIN')")`

Step-by-Step Task Instructions:

1. Ensure at least one `AppUser` (or in-memory user) has role `ADMIN` and another has role `USER`.
2. Add `@EnableMethodSecurity` on the `SecurityConfig` class.

3. Create AdminController with GET /api/admin/data annotated @PreAuthorize("hasRole('ADMIN')").
4. Create a second endpoint GET /api/user/data annotated @PreAuthorize("hasAnyRole('ADMIN','USER')") accessible to both roles.
5. Add POST /api/admin/data, PUT /api/admin/data/{id} and DELETE /api/admin/data/{id}, all annotated @PreAuthorize("hasRole('ADMIN')"), managing a simple in-memory admin-only resource.
6. Test /api/admin/data first with USER credentials (expect 403 Forbidden) and then with ADMIN credentials (expect 200), and repeat the same USER-vs-ADMIN comparison for the POST, PUT and DELETE endpoints.
7. Test /api/user/data with both sets of credentials and confirm both succeed.

P17Application.java

```
package com.example.rbaapi;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class P17Application {

    public static void main(String[] args) {
        SpringApplication.run(P17Application.class, args);
    }

}
```

SecurityConfig.java

```
package com.example.rbaapi.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.EnableMethodSecurity;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableMethodSecurity
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

}
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
}

@Bean
public UserDetailsService userDetailsService(
    PasswordEncoder passwordEncoder) {

    UserDetails normalUser =
        User.withUsername("user")
            .password(passwordEncoder.encode("pass"))
            .roles("USER")
            .build();

    UserDetails adminUser =
        User.withUsername("admin")
            .password(passwordEncoder.encode("pass"))
            .roles("ADMIN")
            .build();

    return new InMemoryUserDetailsManager(
        normalUser,
        adminUser);
}

@Bean
public SecurityFilterChain filterChain(
    HttpSecurity http) throws Exception {

    http
        .csrf(csrf -> csrf.disable())

        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/**")
            .authenticated()
            .anyRequest()
            .authenticated()
        )

        .httpBasic(basic -> {
        });

    return http.build();
}
}
```

AdminController.java

```
package com.example.rbaapi.controller;

import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;
import org.springframework.http.*;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
@RestController
@RequestMapping("/api")
public class AdminController {
    private final List<Map<String,Object>> data = new ArrayList<>();
    private final AtomicInteger ids = new AtomicInteger(1);

    @GetMapping("/admin/data")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<List<Map<String,Object>>> getAdminData() {
        return ResponseEntity.ok(data);
    }

    @PostMapping("/admin/data")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<Map<String,Object>> create(@RequestBody Map<String,Object> body) {
        body.put("id", ids.getAndIncrement());
        data.add(body);
        return ResponseEntity.status(HttpStatus.CREATED).body(body);
    }

    @PutMapping("/admin/data/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<Map<String,Object>> update(@PathVariable int id,
        @RequestBody Map<String,Object> body) {
        body.put("id", id);
        return ResponseEntity.ok(body);
    }

    @DeleteMapping("/admin/data/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<Void> delete(@PathVariable int id) {
        data.removeIf(x -> Integer.valueOf(id).equals(x.get("id")));
        return ResponseEntity.noContent().build();
    }

    @GetMapping("/user/data")
    @PreAuthorize("hasAnyRole('ADMIN','USER')")
    public ResponseEntity<String> getUserData() {
        return ResponseEntity.ok("User data accessed successfully");
    }
}
```

Get Method - User

The screenshot shows the Postman interface with a new request configured. The request is a GET method to the URL `http://localhost:8082/api/admin/data`. The Authorization tab is selected, showing Basic Auth with the username `user` and password `****`. The response body is displayed in JSON format, showing a 403 Forbidden status with the following details:

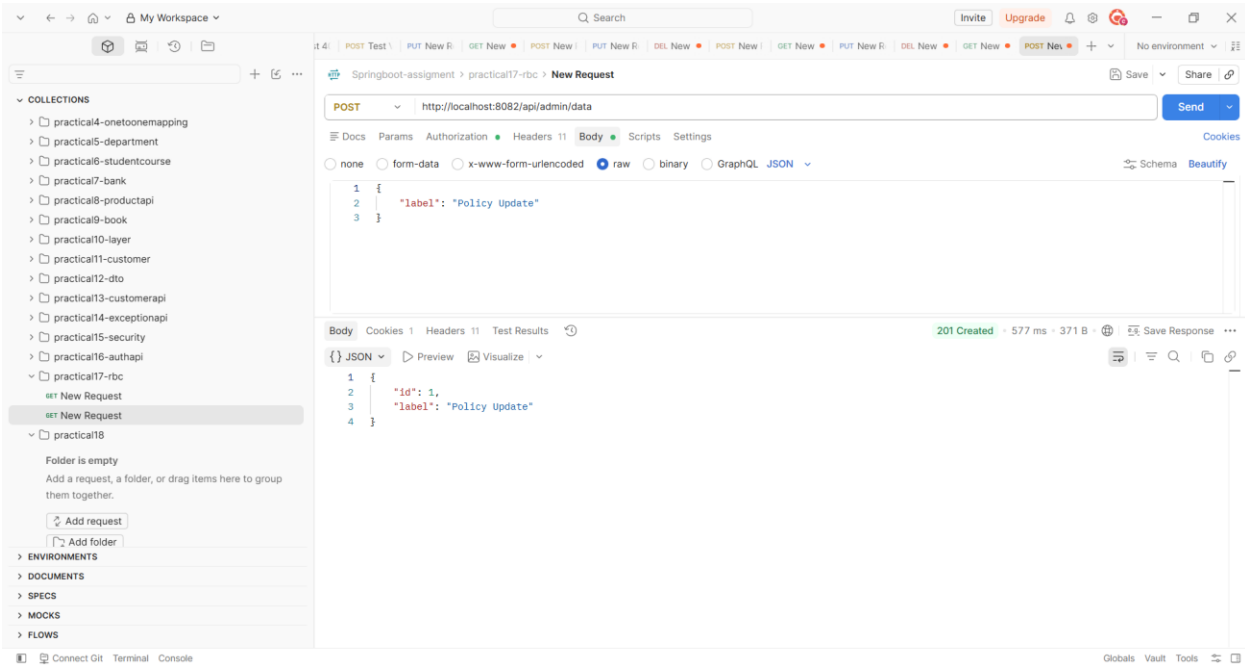
```
{
  "timestamp": "2026-08-14T16:04:45.389Z",
  "status": 403,
  "error": "Forbidden",
  "path": "/api/admin/data"
}
```

Get Method – admin

The screenshot shows the Postman interface with a new request configured. The request is a GET method to the URL `http://localhost:8082/api/admin/data`. The Authorization tab is selected, showing Basic Auth with the username `admin` and password `****`. The response body is displayed in JSON format, showing a 200 OK status with an empty array:

```
[ ]
```

Post Method



Practical No. 18: Securing REST API as an OAuth2 Resource Server with JWT

Objective:

To configure the Spring Boot application as an OAuth2 Resource Server that validates JSON Web Tokens (JWT) issued by an Authorization Server, protecting REST endpoints so that only requests carrying a valid Bearer token succeed — the token-based security mechanism covered in this chapter.

Class Name(s) with Attributes:

ResourceServerConfig (@Configuration — OAuth2 resource server)

- Bean: SecurityFilterChain configured with oauth2ResourceServer().jwt() SecureDataController (@RestController)
- Method: getSecureData() — requires valid JWT with configured scope

Step-by-Step Task Instructions:

1. Add spring-boot-starter-oauth2-resource-server dependency to pom.xml.
2. Configure spring.security.oauth2.resourceserver.jwt.issuer-uri (or jwk-set-uri) in application.properties, pointing to an Authorization Server.

Enroll – 25CI2110039 | Name – Kalariya Heli

3. In ResourceServerConfig, configure the SecurityFilterChain to require authentication for /api/secure/** using .oauth2ResourceServer(oauth2 -> oauth2.jwt()).
4. Obtain a sample JWT access token from the configured Authorization Server for testing.
5. Create GET /api/secure/data returning protected data, optionally checking a required scope/authority using @PreAuthorize("hasAuthority('SCOPE_read')").
6. Add POST /api/secure/data, PUT /api/secure/data/{id} and DELETE /api/secure/data/{id}, each requiring the same valid JWT (and, optionally, a 'write' scope for the mutating verbs) via .oauth2ResourceServer(oauth2 -> oauth2.jwt()).
7. In Postman, call each endpoint without a token, then with an Authorization: Bearer header, and finally with a deliberately invalid/expired token to observe all three outcomes for every verb.

ResourceServerConfig.java

```
package com.example.jwtapi.config;

import org.springframework.context.annotation.*;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableMethodSecurity
public class ResourceServerConfig {
    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http.csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/secure/**").authenticated()
                .anyRequest().permitAll())
            .oauth2ResourceServer(oauth2 -> oauth2.jwt(jwt -> {}));
        return http.build();
    }
}
```

SecureDataController.java

```
package com.example.jwtapi.controller;

import java.util.Map;
import org.springframework.http.*;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/secure")
public class SecureDataController {
    @GetMapping("/data")
    @PreAuthorize("hasAuthority('SCOPE_read')")
    // ...
}
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
public ResponseEntity<Map<String,String>> get() {
    return ResponseEntity.ok(Map.of("message","Protected resource data"));
}

@PostMapping("/data")
@PreAuthorize("hasAuthority('SCOPE_write')")
public ResponseEntity<Map<String,String>> create(@RequestBody Map<String,String> body) {
    return ResponseEntity.status(HttpStatus.CREATED).body(body);
}

@PutMapping("/data/{id}")
@PreAuthorize("hasAuthority('SCOPE_write')")
public ResponseEntity<Map<String,String>> update(@PathVariable int id,
    @RequestBody Map<String,String> body) {
    return ResponseEntity.ok(body);
}

@DeleteMapping("/data/{id}")
@PreAuthorize("hasAuthority('SCOPE_write')")
public ResponseEntity<Void> delete(@PathVariable int id) {
    return ResponseEntity.noContent().build();
}
}
```

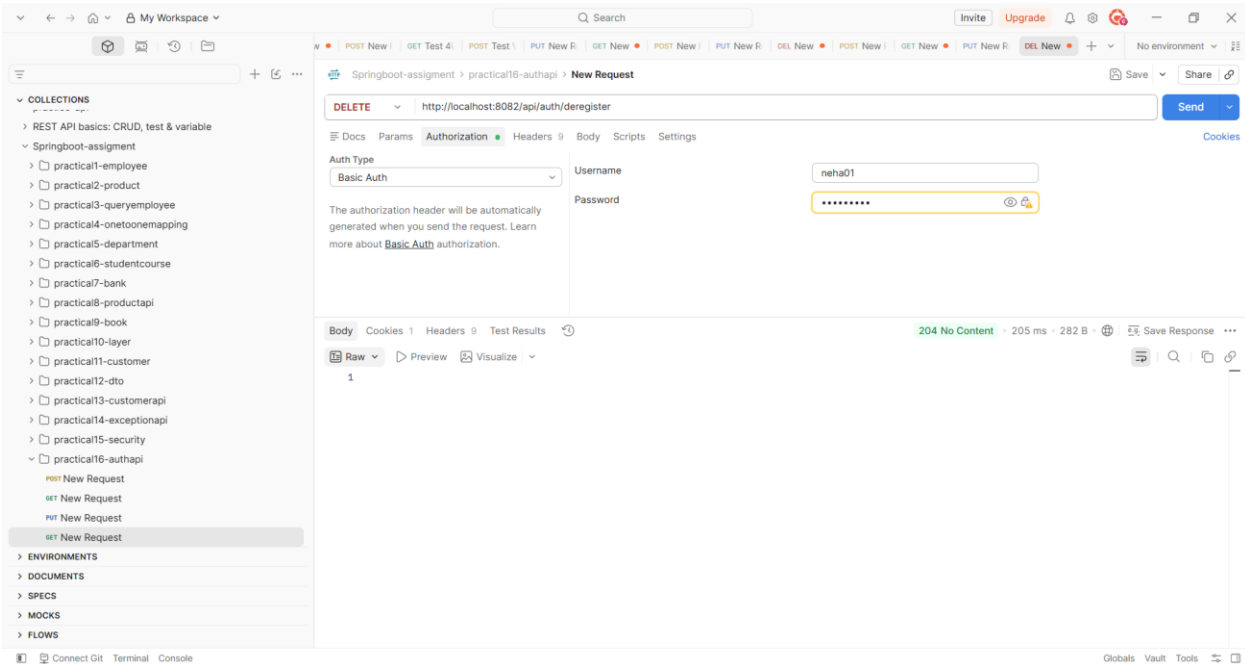
application.properties

```
spring.security.oauth2.resourceserver.jwt.issuer-uri=http://localhost:8080/realms/practical18
```

Get Method

The screenshot shows a REST client interface with a sidebar on the left containing a tree view of collections and environments. The main panel displays a 'New Request' for a GET method to the URL 'http://localhost:8082/api/admin/data'. The 'Authorization' tab is selected, showing 'Basic Auth' with a username of 'user' and a masked password. Below the request details, the 'Body' tab shows a JSON response with a 403 Forbidden status. The JSON body contains the following fields: 'timestamp' (2026-08-14T16:04:45.309Z), 'status' (403), 'error' (Forbidden), and 'path' (/api/admin/data). The interface also includes a top bar with search and navigation icons, and a bottom bar with status and tool icons.

Delete Method



Practical No. 19: OAuth2 Login with a Third-Party Provider (Google/GitHub) and Accessing a Secured REST API

Objective:

To implement the OAuth2 Authorization Code grant using Spring Security's OAuth2 Client support with a real provider (Google or GitHub), obtain an access token, and use that token to call a secured REST endpoint — completing this chapter's end-to-end OAuth2 flow using Postman's built-in OAuth 2.0 authorization support.

Class Name(s) with Attributes:

OAuth2ClientConfig (application.properties based client registration)

- spring.security.oauth2.client.registration.google.client-id
- spring.security.oauth2.client.registration.google.client-secret
- spring.security.oauth2.client.registration.google.scope UserInfoController (@RestController)
- Method: getUserProfile(@AuthenticationPrincipal OAuth2User principal) :

Map

Step-by-Step Task Instructions:

1. Register a new OAuth application on the Google Cloud Console (or GitHub Developer Settings) to obtain a Client ID and Client Secret; set the authorized redirect URI to `http://localhost:8080/login/oauth2/code/google`.
2. Add `spring-boot-starter-oauth2-client` dependency to `pom.xml`.
3. Configure `spring.security.oauth2.client.registration.google.client-id`, `client-secret` and `scope` (`openid`, `profile`, `email`) in `application.properties`.
4. Create `UserInfoController` with GET `/api/user/profile` that reads the authenticated `OAuth2User` principal and returns selected attributes (`name`, `email`) as JSON.
5. Run the application, browse to `http://localhost:8080/oauth2/authorization/google` to complete the login/consent flow, and confirm redirection back with an authenticated session.
6. Add POST `/api/user/notes`, PUT `/api/user/notes/{id}` and DELETE `/api/user/notes/{id}` as simple sessionauthenticated endpoints (any user who completed OAuth2 login may manage their own notes), to demonstrate that the OAuth2-authenticated principal can perform full CRUD, not just reads.
7. In Postman, open the Authorization tab, select Type = OAuth 2.0, and enter the Auth URL, Token URL, Client ID, Client Secret, and Redirect URI matching the provider's values; click 'Get New Access Token', then call GET `/api/user/profile` using the retrieved token.
8. Reuse the same retrieved token to call POST, PUT and DELETE on `/api/user/notes` and confirm each succeeds only while the token is valid.

Code

SecurityConfig.java

```
package com.example.oauthclient.config;

import org.springframework.context.annotation.*;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {
    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http.csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/", "/login/**", "/oauth2/**").permitAll()
                .requestMatchers("/api/user/**").authenticated()
                .anyRequest().permitAll())
            .oauth2Login(oauth -> {});
        return http.build();
    }
}
```

```
}  
}
```

UserInfoController.java

```
package com.example.oauthclient.controller;  
  
import java.util.*;  
import java.util.concurrent.atomic.AtomicInteger;  
import org.springframework.http.*;  
import org.springframework.security.core.annotation.AuthenticationPrincipal;  
import org.springframework.security.oauth2.core.user.OAuth2User;  
import org.springframework.web.bind.annotation.*;  
  
@RestController  
@RequestMapping("/api/user")  
public class UserInfoController {  
    private final Map<Integer,String> notes = new LinkedHashMap<>();  
    private final AtomicInteger ids = new AtomicInteger(1);  
  
    @GetMapping("/profile")  
    public Map<String,Object> profile(@AuthenticationPrincipal OAuth2User principal) {  
        return Map.of("name", principal.getAttribute("name"),  
                      "email", principal.getAttribute("email"));  
    }  
  
    @PostMapping("/notes")  
    public ResponseEntity<Map<String,Object>> create(@RequestBody Map<String,String> body) {  
        int id = ids.getAndIncrement();  
        notes.put(id, body.get("text"));  
        return ResponseEntity.status(HttpStatus.CREATED)  
            .body(Map.of("id",id,"text",body.get("text")));  
    }  
  
    @PutMapping("/notes/{id}")  
    public ResponseEntity<Map<String,Object>> update(@PathVariable int id,  
        @RequestBody Map<String,String> body) {  
        notes.put(id, body.get("text"));  
        return ResponseEntity.ok(Map.of("id",id,"text",body.get("text")));  
    }  
  
    @DeleteMapping("/notes/{id}")  
    public ResponseEntity<Void> delete(@PathVariable int id) {  
        notes.remove(id);  
        return ResponseEntity.noContent().build();  
    }  
}
```

application.properties

```
spring.security.oauth2.client.registration.google.client-id=YOUR_CLIENT_ID  
spring.security.oauth2.client.registration.google.client-secret=YOUR_CLIENT_SECRET  
spring.security.oauth2.client.registration.google.scope=openid,profile,email
```

Practical No. 20: Cross-Origin Resource Sharing (CORS) Configuration for Secured REST APIs

Objective:

To configure CORS at the application level so that a secured REST API can be safely consumed by a browserbased frontend hosted on a different origin — a practical security configuration concern addressed alongside authentication/authorization in this chapter.

Class Name(s) with Attributes:

CorsConfig (@Configuration implements WebMvcConfigurer)

- Method: addCorsMappings(CorsRegistry registry) SecurityConfig (extended)
- .cors(Customizer.withDefaults()) enabled in the SecurityFilterChain S

tep-by-Step Task Instructions:

1. Create CorsConfig implementing WebMvcConfigurer and override addCorsMappings() to allow origin http://localhost:3000 (a sample frontend) on /api/** with GET, POST, PUT, DELETE methods.
2. Update SecurityConfig's SecurityFilterChain to enable .cors(Customizer.withDefaults()) so Spring Security honours the CORS configuration instead of blocking cross-origin calls.
3. Restart the application and, from Postman, add an Origin: http://localhost:3000 header to a request to /api/secure/data to simulate a cross-origin browser call.
4. Verify the response includes the Access-Control-Allow-Origin header matching the allowed origin.
5. Repeat the same Origin-header test for POST /api/secure/data, PUT /api/secure/data/{id} and DELETE /api/secure/data/{id} to confirm the CORS configuration covers all four HTTP verbs, not only GET.
6. Change the Origin header to a disallowed value (e.g., http://evil.com) and confirm the request is blocked/CORS headers are absent.
7. Document the difference between a same-origin Postman call (no CORS headers needed) and a simulated cross-origin call (CORS headers required).

Code

CorsConfig.java

```
package com.example.corsapi.config;
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.*;
```

```
@Configuration
public class CorsConfig implements WebMvcConfigurer {
    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
            .allowedOrigins("http://localhost:3000")
            .allowedMethods("GET", "POST", "PUT", "DELETE")
            .allowedHeaders("*")
            .allowCredentials(true);
    }
}
```

SecurityConfig.java

```
package com.example.corsapi.config;

import org.springframework.context.annotation.*;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.userdetails.*;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {
    @Bean
    UserDetailsService users() {
        UserDetails user = User.withDefaultPasswordEncoder()
            .username("user").password("user123").roles("USER").build();
        return new InMemoryUserDetailsManager(user);
    }

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.csrf(csrf -> csrf.disable())
            .cors(Customizer.withDefaults())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/**").authenticated()
                .anyRequest().permitAll())
            .httpBasic(Customizer.withDefaults());
        return http.build();
    }
}
```

SecureDataController.java

```
package com.example.corsapi.controller;

import java.util.Map;
import org.springframework.http.*;
import org.springframework.web.bind.annotation.*;
```

Enroll – 25CI2110039 | Name – Kalariya Heli

```
@RestController
@RequestMapping("/api/secure")
public class SecureDataController {
    @GetMapping("/data")
    public ResponseEntity<Map<String,String>> get() {
        return ResponseEntity.ok(Map.of("text","Secured data"));
    }

    @PostMapping("/data")
    public ResponseEntity<Map<String,String>> create(@RequestBody Map<String,String> body) {
        return ResponseEntity.status(HttpStatus.CREATED).body(body);
    }

    @PutMapping("/data/{id}")
    public ResponseEntity<Map<String,String>> update(@PathVariable int id,
        @RequestBody Map<String,String> body) {
        return ResponseEntity.ok(body);
    }

    @DeleteMapping("/data/{id}")
    public ResponseEntity<Void> delete(@PathVariable int id) {
        return ResponseEntity.noContent().build();
    }
}
```

Get Method

The screenshot shows the Postman interface with a 'New Request' tab selected. The URL is set to 'http://localhost:8082/api/secure/data' and the method is 'GET'. The 'Headers' tab is active, showing a table with one header: 'Origin' with value 'http://localhost:3000'. The 'Body' tab is also visible, showing a JSON response: '{ "text": "Secured data" }'. The status bar at the bottom indicates a '200 OK' response with a 394 ms response time and 517 B of data.

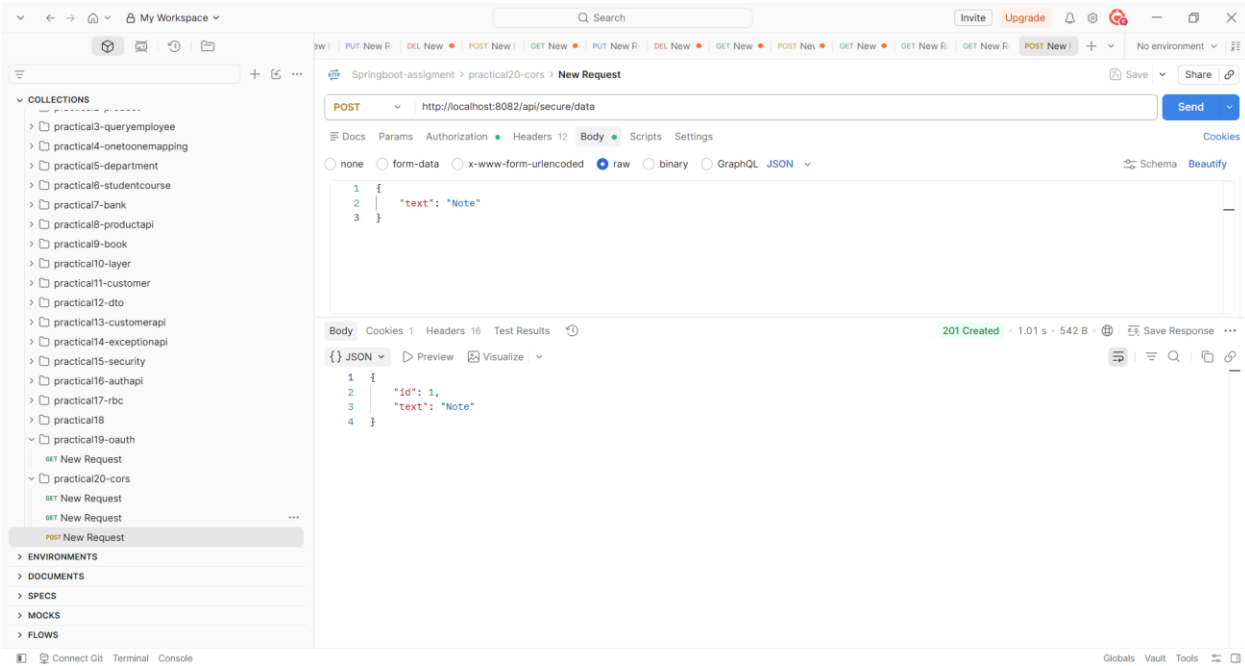
Key	Value	Description
Origin	http://localhost:3000	

Key	Value	Description

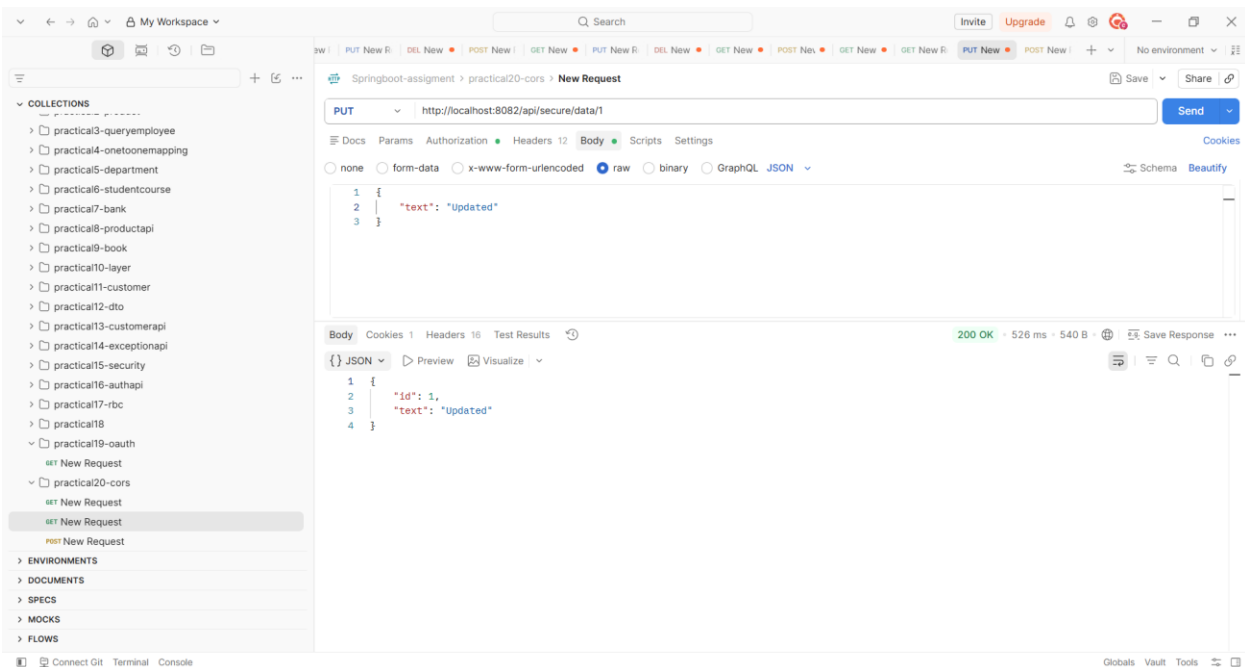
Body: { "text": "Secured data" }

200 OK • 394 ms • 517 B

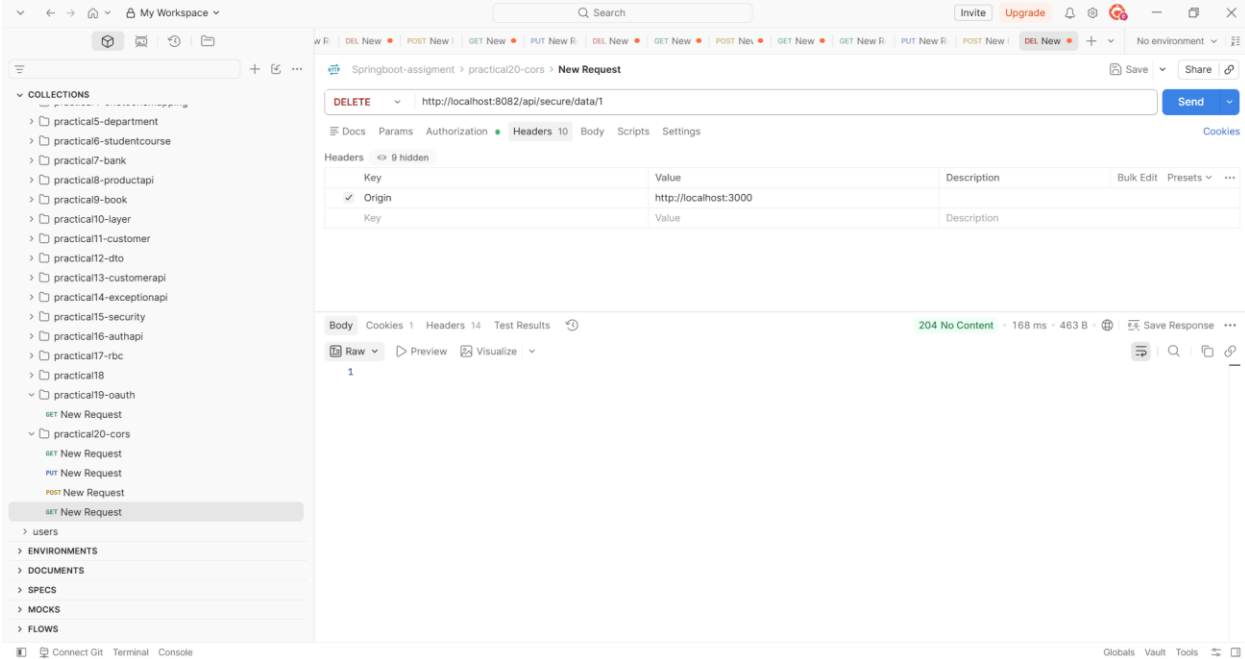
Post Method



Put Method



Delete Method



Method	API Endpoint	Expected Result
GET	http://localhost:8082/api/secure/data	200 OK, Access-Control-Allow-Origin: http://localhost:3000
POST	http://localhost:8082/api/secure/data	201 Created, CORS header present
PUT	http://localhost:8082/api/secure/data/1	200 OK, CORS header present
DELETE	http://localhost:8082/api/secure/data/1	204 No Content, CORS header present
GET	http://localhost:8082/api/secure/data	Disallowed origin; Access-Control-Allow-Origin should not allow http://evil.com